

Kommunikationsprotokoll och drivrutiner

Kommunikations- protokoll och drivrutiner

*Från eget protokoll till en testad
CAN-drivrutin i C++*

Erik Pihl

12 september 2026

Kommunikationsprotokoll och drivrutiner

Kursbok. Ett kapitel per föreläsning.

Boken tar dig från ett självpåhittat protokoll, där varje mekanism skrivs för hand, till en drivrutin mot en CAN-kontroller i ett FPGA som en parallellklass konstruerar samtidigt. Den sista tredjedelen kopplar ihop de två halvorna på riktig hårdvara.

Brödtexten är på svenska. All kod, alla kommentarer i koden, alla register- och signalnamn är på engelska — precis som i kursens övriga material och i koden du skriver.

Text och figurer: Erik Pihl.

Bokens text och figurer, liksom kursmaterialet den är satt från, är licensierade under Creative Commons Erkännande 4.0 Internationell (CC BY 4.0). Du får kopiera, sprida och bearbeta materialet i vilket syfte som helst, även kommersiellt, förutsatt att du anger upphovsperson och licens. Licensen finns på <https://creativecommons.org/licenses/by/4.0/deed.sv>. Koden i bokens listningar får

även användas under MIT-licensen, som är licensen för all kod i kursens repository. Där finns också lektionsmaterialet och lösningsförslagen till övningarna.

Koden är C++17. Drivrutinen håller sig till en delmängd som går att kompilera friställt för en AVR32DB28, alltså utan heap, undantag och RTTI. AVR, AVR-Dx, Cyclone V och DE0-CV är varumärken som tillhör sina respektive ägare.

Satt med Lua $\LaTeX$ i TeX Gyre Pagella och DejaVu Sans Mono.

Den här utgåvan: 12 september 2026.

Innehåll

Förord	ix
1 Frames	1
1.1 Problemet med en byte-ström	1
1.2 Framens fält	2
Sammanfattning	4
Övningar	5
2 Byte-parsning	7
2.1 Från buffer till ström	7
2.2 Robusthet.	9
Sammanfattning	10
Övningar	10
3 Bussar, routing och tillförlitlighet	13
3.1 Bussen och routingen.	13
3.2 En deterministisk felmodell	14
3.3 Tillförlitlig leverans	15
Sammanfattning	17
Övningar	17
4 CAN, registerkartan och arkitekturen	19
4.1 Vad hårdvaran tar över.	19
4.2 Kontrollern, och varför ett lager ligger emellan	20
4.3 Registerkartan	20
4.4 Lagerarkitekturen.	23
Sammanfattning	24
Övningar	25
5 Framen i kod, och den första testsviten	27
5.1 <code>driver::can::Frame</code>	27
5.2 Testpyramiden och de utdelade sviterna	29

Sammanfattning	30
Övningar	31
6 Interfacet och stubben	33
6.1 Interfacet	33
6.2 Stubben	34
Sammanfattning	35
Övningar	36
7 Komponenttest genom stubben	39
7.1 Buggen ett enhetstest inte kan se	39
7.2 <code>app::EchoNode</code>	40
7.3 Att läsa komponenttestet	41
Sammanfattning	42
Övningar	43
8 SPI och transaktionsprotokollet	45
8.1 SPI på fyra ledningar	45
8.2 Transaktionen	45
8.3 Vad det kostar i tid	47
Sammanfattning	47
Övningar	48
9 Transportsömmen och registeråtkomsten	51
9.1 Varför ännu en söm.	51
9.2 Konstanterna.	52
9.3 De två hjälpfunktionerna	53
Sammanfattning	55
Övningar	55
10 Drivrutinen	57
10.1 <code>send()</code>	57
10.2 Packningen.	58
10.3 <code>receive()</code>	59
10.4 <code>hasError()</code> och <code>clearError()</code>	60
10.5 Att vänta på en sändning	60
10.6 Checklista	61
Sammanfattning	61
Övningar	62

11	Testning utan hårdvara	65
11.1	Varför två.	65
11.2	ScriptedTransport.	66
11.3	BankTransport.	66
11.4	Hela stacken på en laptop.	67
11.5	Från fallerande testfall till bugg.	68
11.6	Vad en grön svit inte bevisar.	68
	Sammanfattning.	68
	Övningar.	69
12	Hårdvaran, transporten och factoryn	71
12.1	volatile.	71
12.2	SPI-periferin.	72
12.3	MVIO: två spänningar i samma chip.	73
12.4	Friställd C++.	74
12.5	Factoryn.	74
12.6	main.	75
12.7	En applikation som går att felsöka med.	76
	Sammanfattning.	76
	Övningar.	77
13	Bring-up	79
13.1	Kopplingen.	79
13.2	Stegen.	80
13.3	Felsökning.	81
	Sammanfattning.	82
	Övningar.	82
14	Bussanalys	85
14.1	Varför en analysator.	85
14.2	Att läsa en trace.	86
14.3	Att leta efter förenklingarna.	86
14.4	Bussbelastning.	86
14.5	En frame, hela vägen.	87
	Sammanfattning.	87
	Övningar.	87
15	Repetition	89
15.1	Samma frågor, två svar.	89
15.2	Vad duggan täcker.	90
15.3	De dyraste misstagen.	90

15.4	Efter kursen	91
A	Lösningar	93
	Kapitel 1 — Frames	93
	Kapitel 2 — Byte-parsning	94
	Kapitel 3 — Bussar, routing och tillförlitlighet.	96
	Kapitel 4 — CAN, registerkartan och arkitekturen	97
	Kapitel 5 — Framen i kod.	99
	Kapitel 6 — Interfacet och stubben	100
	Kapitel 7 — Komponenttest genom stubben.	102
	Kapitel 8 — SPI och transaktionsprotokollet.	104
	Kapitel 9 — Transportsömmen.	106
	Kapitel 10 — Drivrutinen	107
	Kapitel 11 — Testning utan hårdvara	109
	Kapitel 12 — Hårdvaran och factoryn	110
	Kapitel 13 — Bring-up.	112
	Kapitel 14 — Bussanalys	114
B	Om duggan	117
C	Övningsdugga	119
D	Svar på övningsduggan	123

Förord

Den här boken handlar om två sätt att lösa samma problem.

Första delen löser det i mjukvara. Du bestämmer själv var en frame börjar, hur lång den är, hur den skyddas mot fel, och vad som ska hända när ett svar uteblir. Varje mekanism blir din egen, och varje mekanism blir kod du får skriva och underhålla. Det är lärorikt på ett sätt som är svårt att ersätta: man förstår vad ett längdfält är till för först när man glömt det.

Andra delen kastar allt det och låter hårdvaran göra jobbet. CAN är från 1986, det sitter i varje bil byggd sedan mitten av nittiotalet, och det löser nästan exakt de problem du just löst själv — fast i kisel, och bättre. Det som återstår för mjukvaran är att prata med kontrollern.

Tredje delen kopplar ihop det med verkligheten: en mikrokontroller, ett FPGA, fyra ledningar mellan dem, och en buss som en bussanalysator kan läsa.

Det som gör kursen ovanlig

CAN-kontrollern du skriver drivrutin mot finns inte färdig. Den konstrueras i VHDL av en parallellklass, samtidigt som du skriver mjukvaran mot den.

Ingen av klasserna kan alltså kompilera mot den andras kod. Det enda som binder ihop de två halvorna är två dokument — en registerkarta och ett SPI-protokoll — och de är projektets egentliga kravspecifikation.

Observera: Att arbeta mot ett kontrakt i stället för mot en implementation är inte en kursövning. Det är hur inbyggd utveckling fungerar när hårdvara och mjukvara byggs parallellt, vilket är det normala fallet och inte undantaget. Det mesta av det som är svårt i den här kursen är svårt av den anledningen.

Bokens upplägg

Ett kapitel per föreläsning, femton stycken, i tre delar:

Kapitel	Del
1–3	Ett eget protokoll. Frames, byte-parsning, bussar, routing och tillförlitlighet.
4–12	CAN-drivrutinen. Registerkartan, arkitekturen, interfacet, stubben, SPI, transportsömmen, drivrutinen, testningen och hårdvaran.
13–15	Hårdvaran. Bring-up, bussanalys, och repetition inför duggan.

Varje kapitel öppnar med vad det innehåller, avslutas med en sammanfattning, och har därefter sina övningar. **Lösningarna till varje övning som inte ber om kod står i appendix A** — alla resonemangs-, räkne- och avkodningsuppgifter, och de delar av kodövningarna som är resonemang. Koden som kodövningarna ber om besvaras inte där: för den är den utdelade testsviten svaret, och koden publiceras i kursrepot efter respektive föreläsning.

Övningsduggan med fullständiga svar ligger sist, i appendix.

Förkunskaper

Boken förutsätter klasser, arv, virtuella funktioner, abstrakta basklasser och factory-mönstret. Den repeterar dem inte. Den förutsätter också grundläggande vana vid Git, en terminal och make.

Inget om CAN, om hårdvaruregister eller om testning förutsätts.

Ytterligare litteratur

En bok till hör till kursen, och den behövs inte för att klara den: *CAN — bussen, framen och kontrollern*, kursens egen bok om CAN som protokoll. Den tar bussen, framen, bitstopppningen, CRC-15, arbitreringen, bittajmingen och felhanteringen, med övningar och svar, och är att betrakta som det här kapitel 4:s långa version. Den ligger i ett eget repo, eftersom den delas med hårdvarukursen och därför inte nämner någon kurs.

KAPITEL 1

Frames

FÖRELÄSNING 1 · ETT EGET PROTOKOLL

Varför en byte-ström inte är kommunikation, och vad som måste läggas till.

I DET HÄR KAPITLET

- Vad ett kommunikationsprotokoll är, och vilket problem det löser.
- Varför en ostrukturerad byte-ström inte går att tolka.
- Framens fält: SOF, LEN, TYPE, DST, SRC, SEQ, DATA och CHK.
- Serialisering och deserialisering, och endianness som felkälla.

1.1 Problemet med en byte-ström

Två system kopplas ihop med en seriell länk — UART, RS-485, SPI, vad som helst. Det ena skickar bytes, det andra tar emot dem. Det är allt länken gör.

Anta att den sändande noden vill skicka temperaturen 23 grader och luftfuktigheten 45 procent. Den skickar två bytes:

17 2D

Mottagaren får två bytes. Och nu kommer frågorna, i tur och ordning:

- **Var börjar meddelandet?** Om mottagaren kopplades in mitt i en överföring är 17 kanske den andra byten i något annat.
- **Hur långt är det?** Kommer det fler bytes, eller var det allt?
- **Vad betyder de?** Att 17 är temperaturen och 2D fukten är en överenskommelse som inte finns någonstans i strömmen.
- **Kom de fram hela?** En störning som ändrade 17 till 16 ser ut precis som en temperatur på 22 grader.
- **Vem skickade dem, och till vem?** På en buss med fler än två noder är det inte självklart.

Ingen av frågorna går att besvara ur bytesen själva. Svaren måste läggas till, och det som lägger till dem är ett **protokoll**.

1.1.1 Frame kontra ström

En **frame** är en avgränsad, självbeskrivande enhet i strömmen. Den säger var den börjar, hur lång den är, vad den innehåller, vem den kommer från, vem den är till, och om den kom fram hel.

Strömmen är fortfarande bara bytes — det är den alltid. Skillnaden är att bytesen nu är ordnade efter en överenskommelse som båda sidor känner till, och som går att kontrollera.

Observera: Det här är den enda idén i kapitlet, och resten av boken är konsekvenser av den. Både det egna protokollet i kapitel 2 och CAN i kapitel 4 löser samma fem frågor. De skiljer sig i *var* lösningen ligger: i din kod, eller i kisel.

1.2 Framens fält

Kursens protokoll använder följande struktur. Siffran är fältets längd i bytes:

SOF (2) | LEN (1) | TYPE (1) | DST (1) | SRC (1) | SEQ (2) | DATA (N) | CHK (2)

Fält	Bytes	Vad det löser
SOF	2	<i>Var börjar meddelandet?</i> Ett fast mönster, 0xA5F7, som markerar start.
LEN	1	<i>Hur långt är det?</i> Antalet databytes som följer.
TYPE	1	<i>Vad betyder det?</i> Meddelandets sort.
DST	1	<i>Till vem?</i> Mottagarens adress.
SRC	1	<i>Från vem?</i> Avsändarens adress.
SEQ	2	Löpnummer, som gör det möjligt att para ihop svar med fråga och att känna igen en dubblett.
DATA	N	Nyttolasten, LEN bytes lång.
CHK	2	<i>Kom det fram helt?</i> Summan av alla föregående bytes.

Frametyperna i kursens protokoll:

Typ	Värde
Ping	0x00
Pong	0x01
StatusRequest	0x02
StatusResponse	0x03
Ack	0x04
Nack	0x05

1.2.1 Varför SOF är två bytes

En byte räcker inte. Vilket värde man än väljer kommer det förr eller senare att dyka upp mitt i en nyttolast, och då hittar mottagaren en frame som inte finns.

Två bytes gör det osannolikt snarare än omöjligt — 0xA5F7 kan fortfarande förekomma i data. Det är därför SOF ensamt aldrig är tillräckligt: mottagaren måste också kontrollera att längden är rimlig och att checksumman stämmer innan den tror på framen. Kapitel 2 bygger den kontrollen.

1.2.2 Varför LEN behövs när DATA ändå tar slut

Därför att mottagaren läser en byte i taget och inte kan se framåt. Utan ett längdfält vet den aldrig om nästa byte är data eller checksumma.

Alternativet vore ett slutmärke, men då måste det mönstret vara omöjligt i nyttolasten — vilket kräver att man kodar om datan. CAN löser samma problem med bitstopppning (se kursens CAN-bok); här räcker ett längdfält.

1.2.3 Checksumman

Kursens checksumma är enkel: **16-bitars summering av alla bytes fram till CHK-fältet.**

```
std::uint16_t computeChecksum(const std::uint8_t* buf, const std::size_t chkOffset)
    noexcept
{
    std::uint16_t checksum{};
    for (std::size_t i{}; i < chkOffset; ++i)
    {
        checksum += buf[i];
    }
    return checksum;
}
```

Den upptäcker enstaka bitfel och de flesta korta felburstar. Den upptäcker däremot inte att två bytes bytt plats, och inte att en byte ökat med ett medan en annan minskat med ett — summan är densamma. En CRC gör det bättre, och det är vad CAN använder.

Observera: Checksumman säger att framen kom fram *hel*. Den säger ingenting om att den kom fram *alls*. Skillnaden mellan integritet och leveransgaranti är hela kapitel 3s andra hälft, och den är värd att hålla isär från början.

1.2.4 Serialisering och endianness

Att serialisera är att skriva ut framens fält i rätt ordning i en buffer. Fält som är bredare än en byte — SOF, SEQ och CHK — måste skrivas i en bestämd byteordning, och kursens protokoll använder **big-endian**: den mest signifikanta byten först.

```
SEQ = 0x7F05  -> 7F 05      (big-endian, det protokollet säger)
                -> 05 7F      (little-endian, det din x86 gör i minnet)
```

Det här är den klassiska buggen i all protokollkod, och den är obehaglig av två skäl. Den är osynlig när båda sidor har samma fel, och den ser ut som slumpmässig korruption när de inte har det: 0x0005 blir 0x0500, alltså 1280 i stället för 5.

Skriv därför aldrig ut en flerbytesvariabel genom att kopiera minnet. Skriv ut den byte för byte, med explicita skiftningar:

```
// Big-endian: most significant byte first, whatever the host's byte order is.
buf[offset + 0U] = static_cast<std::uint8_t>(value >> 8U);
buf[offset + 1U] = static_cast<std::uint8_t>(value);
```

1.2.5 Ett komplett exempel

En StatusRequest från adress 0x17 till adress 0x25, med sekvensnummer 0x7F05 och utan nyttolast:

```
A5 F7 00 02 25 17 7F 05 02 5E
|---| | | | |---|---|
SOF LEN TYPE      SEQ  CHK
      DST--^ ^--SRC
```

Checksumman: $A5 + F7 + 00 + 02 + 25 + 17 + 7F + 05 = 606 = 0x025E$.

Notera att DST står före SRC. Ordningen är godtycklig men måste vara densamma på båda sidor, och att blanda ihop dem ger ett system där varje nod envist svarar sig själv.

Sammanfattning

- En seriell länk levererar bytes och ingenting annat. Var ett meddelande börjar, hur långt det är, vad det betyder, om det kom fram helt och vem det kom från är frågor strömmen inte besvarar.
- En **frame** är en avgränsad, självbeskrivande enhet som besvarar dem. Kursens frame har åtta fält: SOF, LEN, TYPE, DST, SRC, SEQ, DATA och CHK.
- **SOF** är två bytes därför att en byte förr eller senare dyker upp i nyttolasten. SOF ensamt räcker ändå inte — längd och checksumma måste också stämma.

- **LEN** behövs därför att mottagaren läser en byte i taget och inte kan se framåt.
- **Checksumman** ger integritet, inte leveransgaranti. Den säger att ramen kom fram hel, inte att den kom fram.
- Flerbytesfält skrivs **big-endian**, byte för byte med explicita skiftningar — aldrig genom att kopiera minnet.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koderna publiceras i kursrepot efter föreläsningen.

Övning 1.1 Bygg en frame

RÄKNING

En nod med adress 0x17 skickar en StatusRequest till en temperatursensor med adress 0x25. Sekvensnumret är 0x7F05.

- Skriv upp hela ramen byte för byte, på hexadecimal form.
- Sensorn svarar med ett StatusResponse som innehåller temperaturen 0x16 (en byte). Skriv upp den ramen också.
- Hur många bytes är den kortast möjliga ramen i det här protokollet?

Övning 1.2 Fälten och deras syfte

RESONEMANG

För vart och ett av fälten SOF, LEN, TYPE, DST, SRC, SEQ och CHK: beskriv i en mening vad som går sönder om fältet tas bort.

Två av dem går att ta bort utan att protokollet slutar fungera i ett system med exakt två noder. Vilka, och varför?

Övning 1.3 Endianness

RESONEMANG

En grupp serialiserar sekvensnumret genom att kopiera minnet direkt:

```
std::memcpy(&buf[Offset::Seq], &frame.seqNr, sizeof(frame.seqNr));
```

- Vad hamnar i bufferten på en x86-maskin om seqNr är 0x7F05?
- Koden fungerar ändå felfritt i deras egna tester. Varför?
- När går den sönder?

Övning 1.4 Checksummans gränser

RESONEMANG

Kursens checksumma är en summa av bytes.

- a) En störning byter plats på två databytes. Upptäcks det?
- b) En störning ökar en byte med 1 och minskar en annan med 1. Upptäcks det?
- c) Ge ett exempel på en ändring i framen ovan som checksumman *inte* upptäcker.

KAPITEL 2

Byte-parsning

FÖRELÄSNING 2 · ETT EGET PROTOKOLL

Verkligheten: data kommer en byte i taget, och ibland kommer fel bytes.

I DET HÄR KAPITLET

- Varför en parser behövs när framen inte längre ligger färdig i en buffer.
- Parseern som tillståndsmaskin, tillstånd för tillstånd.
- Återhämtning: skräpdata före SOF, tappade bytes, trasig checksumma.
- Var valideringen hör hemma, och varför inte i tillståndsmaskinen.

2.1 Från buffer till ström

I kapitel 1 antog vi att en komplett frame låg i en buffer. Det gör den aldrig.

En seriell länk levererar en byte i taget, när den kommer, och en nod som väntar på nästa byte får inte stanna och vänta — den har annat att göra. Mottagningen måste alltså vara **inkrementell**: ta emot en byte, gör vad som går att göra med den, och återvänd.

Det är precis vad en tillståndsmaskin är till för.

2.1.1 Tillstånden

Ett tillstånd per fält, plus ett som betyder klar:

```
WaitForSof1 -> WaitForSof2 -> WaitForLen  -> WaitForType -> WaitForDst
              -> WaitForSrc  -> WaitForSeq1 -> WaitForSeq2 -> WaitForPayload
              -> WaitForChk1 -> WaitForChk2 -> Ready
```

Parseerns publika gränssnitt är litet:

```
class Parser final
{
public:
```

```

/** Feed the parser one byte. Returns false on a parse error. */
bool processByte(std::uint8_t byte) noexcept;

/** True when a complete frame has been framed. */
bool isFrameReady() const noexcept;

/** Validate and copy out the framed data. */
bool extractFrame(Frame& frame) const noexcept;

/** Discard any partial frame and start over. */
void reset() noexcept;
};

```

2.1.2 Två ansvar, medvetet åtskilda

Lägg märke till att `isFrameReady()` och `extractFrame()` är två olika saker, och att det är avsiktligt:

- `isFrameReady()` säger att **framningen** är klar — rätt antal bytes har kommit in, på rätt platser.
- `extractFrame()` säger att ramen är **giltig** — SOF stämmer, längden är rimlig, typen finns, och checksumman går ihop.

En frame kan vara färdigframad och ändå trasig. Det är själva normalfallet när något gått fel på ledningen, och därför måste de två frågorna ställas var för sig.

```

bool Parser::extractFrame(Frame& frame) const noexcept
{
    if (!isFrameReady()) { return false; }

    // Deserialize validates SOF, payload length, frame type and checksum, so a
    // frame that was
    // framed correctly but arrived corrupted is rejected here.
    return frame.deserialize(myBuf, static_cast<std::size_t>(myParsedBytes));
}

```

Valideringen ligger alltså i `Frame::deserialize()` och inte i tillståndsmaskinen. Det är inte en slump: tillståndsmaskinen vet var fälten *ligger*, ramen vet vad de *betyder*, och att blanda ihop de två ansvaren ger en parser som måste ändras varje gång protokollet gör det.

Observera: `extractFrame()` är `const` och kan därför inte nollställa parsern. Det är ett medvetet val: den som anropar den vet om den vill försöka igen eller kasta bort ramen, och gör `reset()` själv. Samma mönster återkommer i drivrutinen: en metod rapporterar, anroparen bestämmer.

2.2 Robusthet

En parser som bara fungerar när allt går rätt är inte en parser. Tre fall måste hanteras, och alla tre inträffar på riktigt.

2.2.1 Skräpdata före SOF

Noden kopplas in mitt i en pågående överföring, eller en störning lägger till bytes. Strömmen ser ut så här:

```
32 74 00 FF A5 F7 03 03 25 17 7F 05 10 20 30 02 C2
|-----| |-----|
    skräp                ramen
```

Lösningen faller ut ur tillståndsmaskinen gratis: i `WaitForSof1` kastas varje byte som inte är `0xA5`. Parsern står stilla tills strömmen börjar se ut som en frame.

Ett fall är värt att tänka igenom särskilt: parsern står i `WaitForSof2` och får en byte som inte är `0xF7`. Det naiva svaret är att gå tillbaka till `WaitForSof1` och kasta byten. Men om byten var `0xA5` har man just kastat starten på en riktig frame. Rätt svar är att stanna i `WaitForSof2`: den byte man nyss såg kan vara den nya frames första.

2.2.2 Tappade bytes

En byte försvinner. Då blir alla följande fält förskjutna ett steg, längdfältet läser en databyte som längd, och parsern väntar på fel antal bytes.

Det finns ingen elegant räddning, och det ska inte finnas någon: ramen *är* förlorad. Poängen är att parsern ska upptäcka det och komma tillbaka till ett känt läge, inte att den ska gissa. Checksumman fallerar, `extractFrame()` returnerar `false`, anroparen anropar `reset()`, och nästa `0xA5F7` i strömmen startar om.

2.2.3 Orimlig längd

Det här är fallet som är lätt att missa, och det enda i kapitlet som är en säkerhetsbugg och inte bara en felhantering.

Längdfältet är en byte och kan alltså vara upp till 255. Parserns buffert rymmer `MaxPayloadLen` bytes. Tar parsern emot en längd som är större än så, och sedan lydigt skriver in så många databytes, skriver den utanför sin egen buffert.

```
case State::WaitForLen:
{
    // Reject a length the buffer cannot hold, else the payload would overflow ↪
    ↪ myBuf.
    if (MaxPayloadLen < byte) { return reportParseError(); }
```

```

myBuf[myParsedBytes++] = byte;
myState                  = State::WaitForType;
break;
}

```

Regel: Varje längd som kommer utifrån ska kontrolleras mot buffertens storlek *innan* den används, inte efter. Regeln gäller överallt i den här boken, och den gäller TX_DLC i kapitel 10 av exakt samma skäl.

2.2.4 Två frames efter varandra

När en frame är klar och extraherad måste parsern nollställas innan nästa kan tas emot. Glöms det står den kvar i Ready och kastar varje ny byte.

Det är därför `reset()` finns i det publika gränssnittet, och det är därför noden i kapitel 3 anropar den efter varje extraherad frame.

Sammanfattning

- Data kommer en byte i taget. Mottagningen måste därför vara inkrementell, och en tillståndsmaskin med ett tillstånd per fält är det naturliga sättet.
- **Framning och validering är två frågor.** `isFrameReady()` säger att rätt antal bytes kommit in; `extractFrame()` säger att de betyder något. En färdigframad frame kan mycket väl vara trasig.
- Valideringen hör hemma i `Frame::deserialize()`, inte i tillståndsmaskinen: maskinen vet var fälten ligger, framen vet vad de betyder.
- Skräpdata före SOF hanteras av att `WaitForSof1` kastar allt som inte är `0xA5`. I `WaitForSof2` ska parsern **stanna kvar** om byten är `0xA5` igen.
- Ett längdfält utifrån måste kontrolleras mot buffertens storlek innan det används. Annars är det en buffertöverskrivning, inte en felhantering.
- Parsern måste nollställas mellan frames.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koderna publiceras i kursrepot efter föreläsningen.

Övning 2.1 Hitta ramen

AVKODNING

En nod tar emot följande byte-ström:

```
32 74 00 FF A5 F7 03 03 25 17 7F 05 10 20 30 02 C2
```

- a) Vid vilket index börjar ramen?
- b) Vilka bytes tillhör den, från SOF till och med CHK?
- c) Vad ska parsern göra med bytesen före?
- d) Vilken typ har ramen, och vilka adresser går den mellan?

Övning 2.2 SOF-fällan

RESONEMANG

Parsern står i `WaitForSof2` och tar emot `0xA5`.

- a) Vad gör den naiva implementationen, och vad går förlorat?
- b) Vad ska den göra i stället?
- c) Konstruera en byte-ström där skillnaden avgör om en giltig frame hittas eller inte.

Övning 2.3 Tappad byte

AVKODNING

Samma frame som i övning 2.1, men byten `0x20` tappas på vägen.

- a) Vilket fält läser parsern som checksumma?
- b) Kommer `isFrameReady()` att bli sann?
- c) Vad returnerar `extractFrame()`, och varför?
- d) I vilket tillstånd står parsern efteråt, och vad krävs för att den ska hitta nästa frame?

Övning 2.4 Längdfältet

RESONEMANG

En parser kontrollerar inte längdfältet mot buffertens storlek.

- a) Konstruera en byte-ström som får den att skriva utanför sin buffert.
- b) Varför räcker det inte att kontrollera längden i `extractFrame()`?
- c) Vilken allmän regel följer av det, och var i kapitel 10 gäller samma regel igen?

Bussar, routing och tillförlitlighet

Flera noder på en delad ledning, och allt som krävs när den inte är perfekt.

I DET HÄR KAPITLET

- Den delade bussen, och varför broadcast är normen i inbyggda system.
- Routing på DST och SRC, och varför den ligger i noden och inte i bussen.
- En felmodell som är *deterministisk*, och varför det är hela poängen.
- ACK och NACK, timeout, omsändning — och dubbletterna som följer.

3.1 Bussen och routingen

Två noder kan kopplas direkt till varandra. Fem kan inte — det blir tio förbindelser, och antalet växer kvadratiskt. Lösningen är en **delad buss**: alla noder på samma ledning.

Det får en omedelbar konsekvens: **alla hör allt**. En buss är broadcast, och det är inget man väljer bort. Vill man att ett meddelande ska nå en viss nod får man skriva det i meddelandet.

3.1.1 Var routingen ligger

Bussens enda jobb är att leverera varje byte till varje nod. Den läser inte adresser, den känner inte till frames, den vet inte ens att det finns ett protokoll.

Routingen ligger i noden. Varje nod tar emot varje byte, matar den till sin parser, och när en komplett frame extraherats jämför den framens DST med sin egen adress:

```
void Node::tick() noexcept
{
    if (!myParser.isFrameReady()) { return; }

    Frame frame{};
    if (!myParser.extractFrame(frame)) { myParser.reset(); return; }
    myParser.reset();
}
```

```

if (frame.dstAddr != myAddr) { return; } // not for us
handleFrame(frame);
}

```

Att lägga routing i noden i stället för i bussen är inte en förenkling för kursens skull. Det är hur en riktig buss fungerar: ledningen kan inte läsa adresser.

3.1.2 DST och SRC

DST säger vem ramen är till. SRC säger vem den kom från, och behövs för att mottagaren ska kunna svara — utan den vet den inte vart svaret ska.

SEQ parar ihop svar med fråga. En nod som skickat tre förfrågningar och får ett svar måste kunna avgöra vilken av dem det svarar på.

Observera: CAN gör det här helt annorlunda: där finns ingen destinationsadress alls, och identifieraren beskriver meddelandets *innehåll* i stället för dess mottagare. Jämförelsen görs i kapitel 4, och den är en av kursens mest lärorika.

3.2 En deterministisk felmodell

En buss som alltid levererar varje byte i rätt ordning är inte en buss, utan ett funktionsanrop med extra steg. Verkliga bussar tappar bytes, ändrar bitar och fördröjer leveranser.

Problemet med att testa felhantering är att verkliga fel är slumpmässiga. Ett test som ibland fallerar säger ingenting, och ett fel som inträffade en gång går inte att återskapa.

Lösningen är att göra felen deterministiska. Bussimplementationen i kursens simulering räknar bytes och gör exakt samma sak varje körning:

```

// Deterministic fault injection: byte five is dropped, every run, every time.
if (5U == ++myByteCounter) { return; }

```

Tre fel räcker för att täcka det mesta:

Fel	Vad som händer
Tappad byte	Alla följande fält förskjuts. Parsern läser fel längd, väntar på fel antal bytes, och faller till slut på checksumman.
Ändrad bit	Framen är strukturellt korrekt — rätt längd, rätt fältplacering — men checksumman går inte ihop. Det är precis det fall checksumman finns för.
Fördröjning	Framen kommer fram, hel och riktig, men senare än väntat. Ingenting går sönder, och det är poängen: en korrekt parser bryr sig inte om <i>när</i> bytesen kommer.

Det tredje fallet är värt att stanna vid. En parser som fungerar när bytesen kommer tätt men går sönder när de kommer glest har ett dolt tidsberoende, och det är en bugg som aldrig visar sig förrän systemet flyttas till långsammare hårdvara.

3.3 Tillförlitlig leverans

Checksumman ger **integritet**: den som tar emot en frame vet att den är hel. Den ger ingen **leveransgaranti**: sändaren vet fortfarande inte om något kom fram.

Skillnaden är hela det här avsnittet, och de fyra mekanismer som följer bygger på varandra i tur och ordning.

3.3.1 ACK och NACK

Mottagaren svarar. Kom ramen fram hel skickar den ett Ack; var den trasig på ett sätt som ändå går att svara på skickar den ett Nack.

Reglerna för kvittensramen:

- DST och SRC byter plats mot den kvitterade ramen.
- SEQ är **samma** som i den kvitterade ramen — det är så sändaren vet vad som kvitteras.
- LEN är noll.

Att SEQ följer med är inte en detalj. Utan det vet en sändare som har två frames i luften inte vilken av dem som kom fram.

Observera: Ett Nack går bara att skicka när ramen är trasig på ett sätt som lämnar avsändare och sekvensnummer läsbara. Är checksumman fel kan man inte lita på *något* fält — inklusive SRC — så i praktiken är utebliven kvittens det vanliga sättet ett fel visar sig.

3.3.2 Timeout

Och därmed: vad gör sändaren när ingenting kommer tillbaka?

Den kan inte vänta för alltid, och den kan inte veta skillnaden mellan en förlorad frame, en förlorad kvittens och en långsam mottagare. Den enda utvägen är att bestämma sig efter en viss tid: en **timeout**.

Att välja timeout-värde är en avvägning utan rätt svar:

För kort	För lång
Onödiga omsändningar	Långsam felupptäckt
Fler dubbletter	Bussen står still och väntar
Bussen belastas mer när den redan är belastad	Systemet känns hängt

3.3.3 Omsändning

Vid timeout skickas framen om, med **samma** SEQ. Det är avgörande: ett nytt sekvensnummer skulle göra omsändningen till ett nytt meddelande, och mottagaren skulle inte kunna känna igen den som en upprepning.

Antalet försök måste vara begränsat. En sändare som försöker i all evighet mot en nod som är avstängd gör ingenting annat.

3.3.4 Dubbletter

Och här stänger cirkeln. Omsändningen är precis det som skapar dubbletter.

Det farliga fallet är inte att framen tappas — då hände ingenting — utan att **kvittensen** tappas:

A -> B	StatusRequest, SEQ 0x0007	B tar emot och behandlar den
B -> A	Ack, SEQ 0x0007	kvittensen tappas
A	timeout	
A -> B	StatusRequest, SEQ 0x0007	omsändning
B		...behandlar den en andra gång?

Ur B:s synvinkel är det två identiska frames. Behandlar den båda har kommandot utförts två gånger, och är kommandot "öka börvärdet med ett" blir resultatet fel.

Lösningen är att komma ihåg. Varje nod sparar senast behandlade SEQ per avsändare. Kommer samma SEQ igen:

- Kör **inte** applikationslogiken igen.
- Skicka **ändå** kvittensen igen.

Den andra punkten är den som glöms. Sändaren väntar fortfarande på sin kvittens, och tystnad leder bara till ännu en omsändning.

Observera: Fyra mekanismer, i en kedja där var och en löser problemet den föregående skapade: checksumma upptäcker fel, ACK bekräftar leverans, timeout upptäcker utebliven ACK, omsändning åtgärdar den — och dubblettetdetektering städar upp efter omsändningen. Ingen av dem går att utelämna, och det är därför kedjan är värd att kunna utantill.

3.3.5 Och sedan gör hårdvaran det åt oss

Allt i det här avsnittet är kod du skriver, testar och underhåller. Från kapitel 4 ligger det mesta av det i kisel: CAN gör ramning, checksumma och kollisionshantering i hårdvara, och kvitterar dessutom i en enda bit inuti framen.

Det gör inte kapitlet mindre viktigt. Det gör det till den bakgrund mot vilken CAN går att förstå — och till en del av vad duggan examinerar.

Sammanfattning

- En delad buss är broadcast. Alla noder hör allt, och **routingen ligger i noden**, inte i bussen — ledningen kan inte läsa adresser.
- DST säger vem framen är till, SRC vem den kom från så att svar är möjliga, och SEQ parar ihop svar med fråga.
- En felmodell ska vara **deterministisk**. Ett fel som inträffar på samma byte varje körning går att testa; ett slumpmässigt gör det inte.
- Checksumman ger **integritet**, inte **leveransgaranti**.
- Kedjan: checksumma upptäcker fel → ACK bekräftar leverans → timeout upptäcker utebliven ACK → omsändning åtgärdar → dubblettetdetektering städar upp efter omsändningen.
- En omsändning har **samma** SEQ. Ett nytt vore ett nytt meddelande.
- En upptäckt dubblett ska **inte** köra applikationslogiken igen, men **ska** kvitteras igen. Tystnad ger bara ännu en omsändning.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koderna publiceras i kursrepot efter föreläsningen.

Övning 3.1 Routing

RESONEMANG

- Sker routing i bussen eller i noden? Motivera med vad en fysisk ledning kan och inte kan.
- En nod tar emot en frame med korrekt checksumma, men DST är inte dess adress. Vad ska den göra?
- Varför ska den *inte* skicka Nack i det läget?

Övning 3.2 Felmodellen

RESONEMANG

För vart och ett av felen — tappad byte, ändrad bit, fördröjning — svara på:

- Blir framen strukturellt korrekt?
- Vilken mekanism upptäcker felet?
- Vad står parsern i för tillstånd efteråt?

Ett av de tre felen ska inte orsaka något problem alls. Vilket, och vad betyder det om det ändå gör det?

Övning 3.3 Förlorad kvittens

AVKODNING

Nod A skickar en `StatusRequest` med `SEQ 0x0007` till nod B. B tar emot den, behandlar den, och skickar `Ack`. Kvittensen tappas.

- a) Vad gör A härnäst, och efter vad?
- b) Vad ser B då?
- c) Vad ska B göra, och vad ska B inte göra?
- d) Vad hade hänt om B varit tyst i stället?

Övning 3.4 Timeout-värdet

RESONEMANG

- a) Vad händer om timeout är för kort? Ge två konsekvenser.
- b) Vad händer om den är för lång?
- c) Vilken information skulle behövas för att välja värdet rätt, och varför har en nod sällan den?

Övning 3.5 Kedjan

RESONEMANG

Räkna upp de fem mekanismerna i tillförlitlighetskedjan, i ordning. För var och en: vilket problem löser den, och vilket nytt problem skapar den?

KAPITEL 4

CAN, registerkartan och arkitekturen

FÖRELÄSNING 4 · CAN-DRIVROUTINEN

Hårdvaran tar över, och projektet börjar.

I DET HÄR KAPITLET

- Vilka mekanismer från kapitel 3 som CAN utför i kisel, och var.
- Kontrollern du skriver drivrutin mot, och varför ett registerlager ligger emellan.
- Registerkartan: vad varje register betyder och hur varje statusbit nollställs.
- Lagerarkitekturen, och vem som får veta vad.

4.1 Vad hårdvaran tar över

CAN löser samma problem som kapitel 1–kapitel 3, i kisel:

Mekanism	Vem gör den i CAN
Ramsynkronisering	Kontrollern: SOF-biten, och sju recessiva bitar som slutmärke.
Längdfält	Kontrollern: DLC, fyra bitar, 0–8 databytes.
Checksumma	Kontrollern: CRC-15, beräknad och kontrollerad av samma motor.
Kollisionshantering	Bussen själv, genom arbitring — utan bussmästare och utan förlorad tid.
Kvittens	Kontrollern: en enda bit inuti framen, som varje mottagare drar ned.
Adressering	Finns inte. Identifieraren beskriver innehållet, och varje nod filtrerar själv.
Sekvensnummer	Finns inte. Behövs det får applikationen lägga det i nyttolasten.
Timeout och omsändning	En fullständig kontroller gör det i hårdvara. <i>Den här gör det inte</i> — se §4.3.4.

Det som återstår för mjukvaran är alltså att lämna över en identifierare, en längd och några databytes — och att läsa av hur det gick.

Observera: CAN som protokoll — bussen, framen, bitstopningen, CRC-15, arbitreringen, bittajmningen och felhanteringen — står i sin helhet i kursens CAN-bok. Det här kapitlet förutsätter dess kapitel 1, 2 och 7. Läs dem först; resten av boken är fördjupning du kan ta när den behövs.

4.2 Kontrollern, och varför ett lager ligger emellan

Kontrollern är konstruerad i VHDL av en parallellklass. Den är byggd för en anropare i samma klockdomän, och signalerar därför med pulser som är **en klockcykel** långa — 20 ns vid 50 MHz.

En processor som pollar över en seriebuss ser systemet några tiotals mikrosekunder i taget i bästa fall. En puls på 20 ns är alltså borta tusentals cykler innan nästa pollning kommer.

Därför ligger det två lager mellan kontrollern och din kod:



Registerbanken är lagret som gör konstruktionen användbar från mjukvara. Den fångar pulserna och håller kvar dem som nivåer du själv måste nollställa. Varje "skriv 0x1 här för att kvittera" i registerkartan är den mekanismen.

4.3 Registerkartan

Tretton register, vart och ett 32 bitar. **Indexet** är det som går ut på ledningen; offsetkolumnen är alltid $\text{index} \times 4$ och är ett arv från en äldre, minnesmappad variant av samma konstruktion.

Idx	Register	Offset	Åtk.	Beskrivning
0	STATUS	0x00	R	Bit 0: TX klar. Bit 1: RX giltig. Bit 2: Fel.
1	TX_ID	0x04	R/W	11-bitars sändar-ID (bitarna 10–0).
2	TX_DLC	0x08	R/W	Data Length Code (bitarna 3–0, värde 0–8).
3	TX_DATA_LO	0x0C	R/W	Sändardata byte 0–3 (byte 0 = MSB).
4	TX_DATA_HI	0x10	R/W	Sändardata byte 4–7.
5	TX_SEND	0x14	W	Skriv 0x1 för att utlösa sändning.
6	RX_ID	0x18	R	Mottagen identifierare.
7	RX_DLC	0x1C	R	Mottagen DLC.
8	RX_DATA_LO	0x20	R	Mottagen data byte 0–3.
9	RX_DATA_HI	0x24	R	Mottagen data byte 4–7.
10	RX_ACK	0x28	W	Skriv 0x1 för att kvittera.
11	ERROR_FLAGS	0x2C	R/W	Felregistret; skriv 0x0 för att nollställa.
12	TX_ABORT	0x30	W	Skriv 0x1 för att tvinga tillbaka TX klar.

Index 13–15 är reserverade: en läsning ger 0x00000000, en skrivning ignoreras.

4.3.1 STATUS: de tre bitarna allt kretsar kring

Bit	Betydelse	Sätts av	Nollställs av
0	TX klar	reset; en avslutad sändning; en stigande flank på felnivån medan biten är låg; en skrivning till TX_ABORT	en accepterad skrivning till TX_SEND
1	RX giltig	en mottagen frame, som samtidigt låser RX_*	en skrivning av 0x1 till RX_ACK
2	Fel	en stigande flank på kontrollerns felnivå	en skrivning av 0x0 till ERROR_FLAGS

Bitarna 31–3 läses alltid som 0.

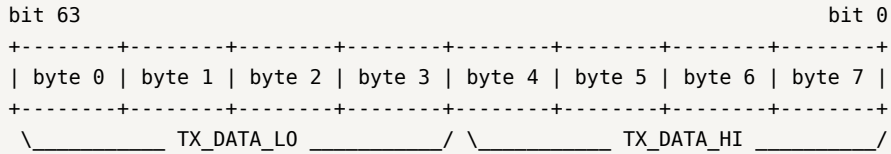
Läs tabellen en gång till. Varje bit har en uttrycklig nollställning, och den nollställningen är drivrutinens ansvar. Glöms RX_ACK står bit 1 kvar för alltid och du läser samma frame i en oändlig loop. Glöms att TX_SEND nollställer bit 0 undrar du varför den andra sändningen aldrig accepteras.

4.3.2 Regler som är lätta att missa

- TX_SEND och RX_ACK kräver att **bit 0 är satt**; övriga bitar ignoreras.
- ERROR_FLAGS nollställs **bara av ett helt nollställt ord**. 0x1 gör ingenting, och 0x000000FF gör ingenting trots att bit 0 är satt. Motsatt regel mot de två ovan.
- TX_ID maskerar till elva bitar, TX_DLC till fyra. En skrivning av 0xFFFFFFFF läses tillbaka som 0x000007FF respektive 0x000000F.
- Skrivningar till STATUS och RX_* ignoreras tyst.
- Läsningar av TX_SEND och RX_ACK ger 0x00000000.

4.3.3 Databyteordning

TX_DATA och RX_DATA är 64 bitar breda, med **databyte 0 i de mest signifikanta bitarna**:



LO och HI syftar på registerparets ordning, inte på bitsignifikans: **TX_DATA_LO bär de första fyra databytesen**. Det är projektets vanligaste läsfel, och det syns inte förrän en frame kommer fram bakvänd.

En frame med dlc = 2 och data AA BB packas alltså som TX_DATA_LO = 0xAABB0000 och TX_DATA_HI = 0x00000000.

4.3.4 De tre begränsningarna

Hårdvaran är förenklad mot riktig CAN på många punkter. Tre av dem syns ända upp i din kod, och måste hanteras snarare än döljas.

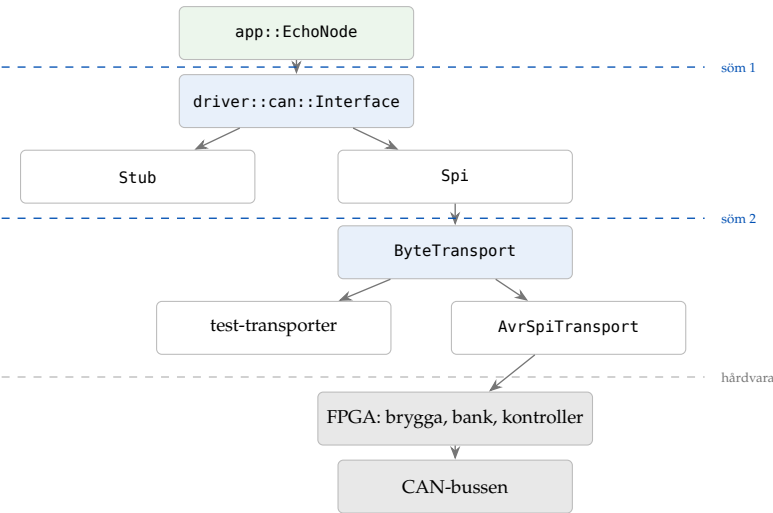
En förlorad arbitrerings ser ut som en lyckad sändning. STATUS bit 0 kommer tillbaka i båda fallen. Biten betyder *sändarporten är fri igen*, inte *framen kom fram*. Vill du veta utfallet: vänta på bit 0, och läs **sedan** ERROR_FLAGS.

ERROR_FLAGS är en bit för fyra orsaker — förlorad arbitrerings, stoppfel, misslyckad CRC och mottagen fjärrframe. Du kan se *att* något gick fel, aldrig *vad*. Retry-logik som skiljer på dem går inte att skriva mot det här gränssnittet.

Ingen buffring på mottagarsidan. Kontrollern håller exakt en frame. Kommer nästa innan den förra kvitterats skrivs den över, och den nyaste vinner. Polla ofta — och vet att det vid någon trafikintensitet ändå inte räcker.

Observera: Hårdvaran validerar ingenting. En TX_DLC på 0xF lagras och skickas vidare precis som den är. Att avvisa ogiltiga frames är **drivrutinens** ansvar, och bara drivrutinens: två kopior av samma regel är två saker som ska hållas i synk.

4.4 Lagerarkitekturen



Figur 4.1 Drivrutinsstacken. Varje vågrät linje är en gräns någon inte får titta igenom.

Två sömmar bär hela konstruktionen, och de löser olika problem:

Söm	Gör vad testbart
driver::can::Interface	Applikationen. EchoNode kan köras mot en Stub och behöver aldrig veta att hårdvara finns.
driver::can::ByteTransport	Drivrutinen själv. Samma Spi-kod som flashas till målet kan köras på din laptop mot en transport du skriptar.

Den andra sömmen är inte uppenbar, och den kommer i kapitel 9. Utan den går Spi bara att köra på riktig hårdvara — och den finns först i kapitel 13, näst sist i kursen, på ett kort som samtidigt felsöks av någon annan.

Regeln som följer, och som en granskare ska leta efter: ingen fil ovanför driver/can/interface.hpp får innehålla ordet SPI.

4.4.1 Sömmen har redan lönat sig en gång

Argumentet för ett interface brukar vara hypotetiskt — *tänk om hårdvaran byts ut*. Här är det inte hypotetiskt.

En tidigare version av den här konstruktionen nådde registren via minnesmappad I/O på basadressen 0xFF200000. Det förutsätter en processor med en adressbuss ut mot FPGA-väven, och mikrokontrollern i det här systemet har ingen sådan buss. Registren nås över SPI i stället.

Allt ovanför Interface hade överlevt den ändringen **oförändrat**. Det är hela argumentet, demonstrerat på riktig kod.

4.4.2 Sändningssekvensen

Att skicka en frame (id 0x123, dlc 2, data AA BB) är fem registerskrivningar och en pollning:

```

1. läs   STATUS           -> bit 0 satt? annars: upptaget, returnera false
2. skriv TX_ID            = 0x00000123
3. skriv TX_DLC           = 0x00000002
4. skriv TX_DATA_LO       = 0xAABB0000
5. skriv TX_DATA_HI       = 0x00000000
6. skriv TX_SEND          = 0x00000001    -> STATUS bit 0 går låg
   ...
7. polla STATUS           -> bit 0 tillbaka: porten är fri
8. läs   ERROR_FLAGS      -> 0 = framen gick ut, 1 = något gick fel

```

Och att ta emot en:

```

1. läs   STATUS           -> bit 1 satt? annars: ingen frame, returnera false
2. läs   RX_ID, RX_DLC
3. läs   RX_DATA_LO, RX_DATA_HI, packa upp, maskera mot RX_DLC
4. skriv RX_ACK           = 0x00000001    -> STATUS bit 1 går låg

```

TX_SEND **måste** skrivas sist. Den skrivningen betyder inte "spara ett värde" utan "börja sända nu", och kontrollern läser då de övriga registren. Skriv triggern först och du skickar förra framens innehåll — vilket vid bring-up'en ser ut som en nod som ligger exakt en frame efter.

Sammanfattning

- CAN gör ramning, längd, checksumma, kollisionshantering och kvittens i kisel. Det som återstår för mjukvaran är identifierare, längd, databytes — och att läsa av hur det gick.
- CAN har **ingen destinationsadress**. Identifieraren beskriver innehållet, och varje nod filtrerar själv.
- Kontrollern signalerar med encykelspulser. **Registerbanken** gör om dem till klibbiga bitar med **uttrycklig nollställning** — och den nollställningen är drivrutinens ansvar.
- TX_SEND och RX_ACK kräver bit 0 satt; ERROR_FLAGS nollställs bara av ett *helt nollställt* ord. Motsatta regler, och en vanlig bugg.
- TX_DATA_LO bär databyte 0–3, med byte 0 i bitarna 31–24.
- Tre begränsningar når upp i koden: en förlorad arbitrering ser ut som en lyckad sändning, felflaggan är en bit för fyra orsaker, och mottagarsidan buffrar inte.
- Hårdvaran validerar ingenting. Valideringen ligger i drivrutinen, på exakt ett ställe.

- Två sömmar: Interface gör applikationen testbar, ByteTransport gör drivrutinen testbar.

Övningar

Övningarna görs med papper och penna, och lösningarna står i [appendix A](#).

Övning 4.1 Vem gör vad

RESONEMANG

Fyll i, för varje mekanism från kapitel 3: vem utför den i ett CAN-system, och var? Ramsynkronisering, längdfält, adressering, sekvensnummer, checksumma, kvittens, kollisionshantering, timeout och omsändning, dubblettdetektering.

Två av raderna har inget självklart svar. Vilka, och varför?

Övning 4.2 Packa en frame

RÄKNING

Du ska skicka en frame med `id = 0x2A7`, `dLC = 5` och databytesen `01 02 03 04 05`. Skriv upp de fem registervärdena som ska skrivas, i den ordning de ska skrivas, på hexadecimal form. Varför spelar ordningen roll?

Övning 4.3 Packa upp en frame

RÄKNING

Du läser `RX_ID = 0x00000481`, `RX_DLC = 0x00000003`, `RX_DATA_LO = 0xDEADBE00` och `RX_DATA_HI = 0x00000000`.

- Vilken identifierare kom framen från?
- Vilka databytes innehåller den, i ordning?
- Vad ska `frame.data[3]` till `frame.data[7]` innehålla efter `receive()`?
- Vilket register måste skrivas innan nästa frame kan tas emot, och med vilket värde?

Övning 4.4 Fyra påståenden

RESONEMANG

Tre av följande beskriver en bugg. Vilka, och vad går fel?

- "Efter `send()` pollar vi STATUS bit 0 och returnerar `true` när den kommer tillbaka — då vet vi att framen kom fram."
- "`clearError()` skriver `0x1` till `ERROR_FLAGS` för att nollställa felbiten."
- "`receive()` kopierar alltid alla åtta bytes från `RX_DATA_LO/HI` till `frame.data`."
- "`send()` kontrollerar STATUS bit 0 innan den skriver `TX_SEND`."

Övning 4.5 Lagergränsen

RESONEMANG

Sant eller falskt, med motivering i en mening:

- a) `app::EchoNode` får inkludera `driver/can/spi.hpp`.
- b) `driver::can::Spi` får inkludera `driver/can/interface.hpp`.
- c) `driver::can::Stub` behöver en `ByteTransport`.
- d) `ByteTransport` behöver känna till registerkartan.
- e) Ett test av `EchoNode` behöver en `ByteTransport`.

KAPITEL 5

Framen i kod, och den första testsviten

FÖRELÄSNING 5 · CAN-DRIVROUTINEN

Elva bytes data, och de tester som avgör om resten av kursen går att lita på.

I DET HÄR KAPITLET

- `driver::can::Frame`: en aggregate, inte ett objekt, och varför.
- Var validering hör hemma när hårdvaran inte validerar någonting.
- Testpyramiden, och varför testning är avgörande just i den här kursen.
- Att *läsa* en testsvit — du skriver inga.

5.1 `driver::can::Frame`

Registerkartan i kapitel 4 säger exakt vad hårdvaran kan skicka och ta emot, och det är inte mycket: en identifierare, en längd och upp till åtta databytes. Klassen speglar det:

```
namespace driver::can
{
    /**
     * @brief A single CAN frame: identifier, data length code and payload.
     */
    struct Frame
    {
        /** Maximum number of payload bytes in a CAN frame. */
        static constexpr std::uint8_t MaxDataLen{8U};

        std::uint16_t id{};           /**< CAN identifier, 11 bits (0x000 - 0x7FF)↵
        ↵ . */
        std::uint8_t dlc{};           /**< Data length code, 0 - 8. */
        std::uint8_t data[MaxDataLen]{}; /**< Payload. */
    };
}
```

```
};

/**
 * @brief Check whether a frame is valid, i.e. whether the hardware can send it.
 */
[[nodiscard]] constexpr bool isValid(const Frame& frame) noexcept
{
    return (frame.id <= 0x7FFU) && (frame.dlc <= Frame::MaxDataLen);
}

} // namespace driver::can
```

Det är hela filen. Ingen .cpp.

5.1.1 Varför en aggregate

Frame har medvetet ingen konstruktor, destruktor eller medlemsfunktion. Skälen, i tur och ordning:

Den är data, inte beteende. En frame är något som flyttas: från applikationen till drivrutinen, från drivrutinen till fyra register, från registren ut på bussen.

Aggregate-initiering är läsbar. Med aggregaten fungerar `Frame frame{0x123U, 2U, {0xAAU, 0xBBU}}`. Lägg till en enda användardeklarerad konstruktor och den raden slutar kompilera. Testerna du möter från och med nästa kapitel är fulla av sådana rader.

Default member initialization gör `Frame f{}` säker. `{}` efter varje medlem betyder att en default-konstruerad frame har nollställda fält. Utan dem hade `receive()` kunnat lämna ifrån sig skräp i de bytes den inte fyllde.

5.1.2 static constexpr, inte #define

`MaxDataLen` har en **typ**, en **räckvidd** och finns i **debuggern**. En `#define` har inget av det, och gäller dessutom från raden den står på till filens slut, tvärs igenom varje header som inkluderas efteråt.

Den används dessutom som arraystorlek, vilket gör att arrayens storlek och gränskontrollen i `isValid()` garanterat är samma tal.

5.1.3 Varför `isValid()` är en fri funktion

Två skäl. Den håller aggregaten intakt — en medlemsfunktion är första steget mot en klass med invarianter, och nästa steg är en konstruktor. Och den hör till *drivrutinen*, inte till datan: regeln kommer från den här hårdvarans registerbredder, inte från CAN som begrepp.

Regel: Hårdvaran validerar ingenting. Drivrutinen **måste** därför validera — och **bara** drivrutinen. Frestelsen att lägga en kontroll till i applikationen ”för säkerhets skull” ska motstås: två kopior av samma regel är två saker som ska hållas i synk, och den dag `MaxDataLen` ändras kommer bara den ena att ändras med.

5.2 Testpyramiden och de utdelade sviterna

5.2.1 Du skriver inga tester

Varje testsvit i den här kursen är **utdelad**. Ditt jobb är att köra dem, läsa dem när de fallerar, och få dem att passera.

Det är ett medvetet upplägg. Att skriva bra tester är ett eget hantverk, och det kommer i en senare kurs. Här får du i stället se vad tester är *till för*, på ett projekt där de gör verklig nytta — och den som har sett en testsvit hitta en bugg i sin egen kod har en betydligt konkretare bild av vad ett bra test är än den som ombeds skriva ett först.

5.2.2 Varför testning avgör den här kursen

I tidigare projekt gick det att köra programmet och titta. Här går det inte:

- **Hårdvaran finns inte än.** Den byggs parallellt. Första gången du kan köra mot den är kapitel 13, tredje passet från slutet.
- **Hårdvaran är inte din.** När den finns felsöks den samtidigt av någon annan. En bugg kan ligga i din kod, i deras VHDL, eller i en kopplingstråd.
- **Det mesta går att köra utan hårdvara.** Det är ingen tröstpremie utan en designkonsekvens: sömmarna i kapitel 4 finns just för det.

Regeln: allt som kan köras på laptopen ska vara grönt innan det körs på hårdvara. Bring-uppen ska leta efter kopplingsfel, inte efter bitpackningsfel.

5.2.3 Pyramidens tre nivåer

Nivå	Var	Vad
Enhetstest	kapitel 5, kapitel 6	En klass i taget: <code>Frame</code> , <code>Stub</code> .
Komponenttest	kapitel 7, kapitel 11	Flera delar ihop, utan hårdvara: <code>EchoNode</code> genom stubben; <code>SpI</code> mot en simulerad registerbank.
Integrationstest	kapitel 13, kapitel 14	Riktig hårdvara, riktiga ledningar.

Bredden är en rekommendation om *antal*: många snabba tester längst ned, få dyra längst upp. Ett projekt med tre snabba tester och en person som kopplar om sladdar är pyramiden upp och ned.

5.2.4 Att läsa ett testfall

Varje utdelat testfall har tre delar:

```
TEST(Frame, isValidRejectsTooLargeDlc)
{
    // Arrange.
    driver::can::Frame frame{};
    frame.dlc = 9U;

    // Act & Assert.
    EXPECT_FALSE(driver::can::isValid(frame));
}
```

Arrange säger vilket utgångsläge kravet gäller i, Act säger vad som ska hända, och **Assert är kravet**.

Lägg också märke till värdena: 8 och 9, 0x7FF och 0x800. Inte 4 och 0x400. **Buggar bor på gränserna** — en jämförelse som ska vara $\leq$ men skrevs $<$ fungerar för varenda värde utom ett.

5.2.5 Att läsa en fallerande svit

```
FAILED: EXPECT_EQ(frame.data[0], 0xAAU) at test_frame.cpp:42
```

Arbetsgången, och **utan att öppna din egen kod i steg 1–3**:

1. Läs **testnamnet**. Det säger vilket krav som är brutet.
2. Läs **Arrange**. Den säger under vilka förutsättningar.
3. Läs **Assert-raden** som fallerade. Den säger vad som förväntades.
4. Först därefter: öppna din kod.

Steg 1–3 tar en minut och gör steg 4 kort. Att hoppa direkt till steg 4 är den vanligaste anledningen till att en enkel bugg tar en timme.

Observera: Den utdelade testsviten är ofta en **mer exakt kravspecifikation** än prosan i projektbeskrivningen. Prosan är skriven av en människa och kan vara tvetydig; sviten kompilerar och kör. Undrar du exakt vad en metod ska returnera i ett gränsfall: öppna testfilen innan du gissar.

Sammanfattning

- Frame är en **aggregate**: ingen konstruktor, ingen destruktor, inga metoder. Det håller aggregate-initieringen läsbar och gör klassen till ren data.

- `MaxDataLen` är `static constexpr` — typ, räckvidd, synlig i debuggern, och samma tal som arrayens storlek.
- `isValid()` är en fri funktion, och regeln uttrycks **en gång**. Hårdvaran validerar inget, så drivrutinen måste — och bara den.
- Du skriver inga tester i kursen. Du kör dem, läser dem och får dem att passera.
- Testning avgör kursen därför att hårdvaran varken finns eller är din förrän näst sist.
- Läs ett fallerande testfall i ordningen `testnamn` → Arrange → Assert, och öppna din egen kod sist.
- Testsviten är oftare en exaktare kravspecifikation än prosan.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koderna publiceras i kursrepot efter föreläsningen.

Övning 5.1 Förutsäg

RESONEMANG

Svara innan du kompilerar, och kontrollera sedan.

- Vad är `sizeof(driver::can::Frame)` på din maskin? Varför är det inte 11?
- Kompilerar `driver::can::Frame f{0x123U, 2U, {0xAAU, 0xBBU}}?` Vad blir `f.data[7]`?
- Vad händer om du lägger till en egen konstruktor och sedan försöker kompilera raden i b)?

Övning 5.2 constexpr

KOD

Lägg till följande i en testfil:

```
static_assert(driver::can::isValid(driver::can::Frame{0x7FFU, 8U, {}}));
static_assert(!driver::can::isValid(driver::can::Frame{0x800U, 0U, {}}));
```

- Vad bevisar det att raderna kompilerar?
- Vad kostar de vid körning?

Övning 5.3 Läs sviten

RESONEMANG

Öppna den utdelade `test_frame.cpp` och svara utan att köra den.

- Vilka gränsvärden testas för `id` respektive `d1c`? Varför just de?
- Vilket testfall skulle falla om `isValid()` skrev `<` i stället för `<=`? Vilka skulle fortfarande vara gröna?
- Finns det något krav i det här kapitlet som sviten *inte* kontrollerar?

Övning 5.4 Bryt koden

KOD

Ändra `isValid()` så att `id`-gränsen blir `<` i stället för `<=`, och kör sviten.

- a) Vilket testfall fallerar, och vad står i utskriften?
- b) Följ arbetsgången i §5.2. Hur lång tid tog det att hitta felet?
- c) Återställ koden.

Övning 5.5 Var ligger valideringen

RESONEMANG

En gruppmedlem föreslår att också `EchoNode` ska kontrollera `d1c` innan den anropar `send()`, "för säkerhets skull".

- a) Vad är argumentet för?
- b) Vad är argumentet emot?
- c) Vad händer den dag `MaxDataLen` ändras?

KAPITEL 6

Interfacet och stubben

FÖRELÄSNING 6 · CAN-DRIVROUTINEN

Sömnen som gör att applikationen aldrig behöver veta var den är.

I DET HÄR KAPITLET

- `driver::can::Interface`: fyra metoder, och motiveringen till varje `noexcept`, `const` och `[[nodiscard]]`.
- Vad som medvetet *inte* finns i interfacet, och testet som avgör det.
- `driver::can::Stub`: en testdubblare i minnet.
- Stubb, mock, och var gränsen för en testdubblares realism går.

6.1 Interfacet

```
namespace driver::can
{
    struct Frame;

    class Interface
    {
    public:
        virtual ~Interface() noexcept = default;

        [[nodiscard]] virtual bool send(const Frame& frame)      = 0;
        [[nodiscard]] virtual bool receive(Frame& frame)        = 0;
        [[nodiscard]] virtual bool hasError() const noexcept    = 0;
        virtual void clearError() noexcept                      = 0;
    };
} // namespace driver::can
```

Varje del av den, en i taget:

Virtuell destruktör. Objekt kommer att raderas genom en `Interface*`. Utan `virtual` anropas bara basklassens destruktör, och en `Spi` som håller resurser läcker dem — odefinierat beteende, och en av de få buggarna i den här designen som inget test fångar. = default därför att det inte finns något att göra; `noexcept` därför att destruktörer aldrig ska kasta.

= **0 på alla fyra.** Klassen är ren abstrakt: den går inte att instansiera och har ingen implementation att ärvas av misstag. Den är ett kontrakt och ingenting annat.

[[nodiscard]] på tre av fyra. `send()` returnerar `false` när hårdvaran är upptagen. En anropare som ignorerar det tror att framen gick iväg — den vanligaste drivrutinsbuggen som finns, här förvandlad till en kompilatorvarning.

const på hasError(). Att fråga om ett fel ändrar inte drivrutinen, och metoden går därmed att anropa på en `const Interface&`.

noexcept på de två felmetoderna. De anropas från felhanteringsvägar, och en felhanteringsväg som själv kan kasta är svår att resonera om. De gör dessutom ingenting som *kan* misslyckas.

6.1.1 Vad som inte finns här

Ingen `init()`. Ingen `setBaudRate()`. Ingen `readRegister()`.

Testet är: kan både Stub och Spi implementera metoden meningsfullt? `readRegister()` kan de inte — stubben har inga register. En sådan metod hade tvingat stubben att ljuga, och tvingat applikationen att veta att register finns.

6.2 Stubben

Stub är final, ärver `Interface`, och simulerar CAN i minnet:

Metod	Kontrakt
Konstruktorn	<code>myLastFrame tom, myInitialized = true, myHasData = false.</code>
<code>send()</code>	<code>false</code> om oinitialiserad. Annars: lagra framen, sätt <code>myHasData</code> , returnera <code>true</code> .
<code>receive()</code>	<code>false</code> om oinitialiserad eller utan data. Annars: kopiera ut framen, nollställ <code>myHasData</code> , returnera <code>true</code> .
<code>hasError()</code>	Alltid <code>false</code> .
<code>clearError()</code>	Gör ingenting.
<code>inject()</code>	Lagrar en frame för senare <code>receive()</code> .

Kopierings- och flyttoperationerna tas bort i den publika delen. En `Stub` representerar en (simulerad) hårdvaruresurs, och två kopior av samma resurs är inte två resurser.

6.2.1 Varför inject() inte ligger i interfacet

Den finns bara för testernas skull: den simulerar att en frame kommit in på bussen. En `Spi` kan inte implementera den — den kan inte trollda fram trafik som inte finns.

Konsekvensen är mönstret som återkommer genom hela boken:

```

driver::can::Stub stub{};           // konkret typ: har inject()
app::EchoNode node{stub, 0x200U};   // tar en Interface&: har den inte
stub.inject(frame);                 // testet talar med stubben direkt
node.run();                         // komponenten vet ingenting om det

```

Testet håller i den **konkrete** typen, komponenten ser bara **interfacet**.

6.2.2 Vad stubben inte simulerar

Inte fel. Inte upptagen hårdvara. Ingen kö — en andra `send()` skriver över den första. Ingen arbitrering, ingen CRC.

Det är avsiktligt. Stubbens jobb är att låta `EchoNode` köras, inte att vara en CAN-simulator.

Observera: Att veta var gränsen för en testdubblares realism går är en färdighet i sig. **En stubb som växer tills den är en andra implementation av hårdvaran är en stubb som själv behöver testas.** Den som vill testa hur drivrutinen betar sig när hårdvaran är upptagen behöver en annan dubblare, en söm längre ned — och den kommer i kapitel 11.

6.2.3 Stubb, inte mock

En **stubb** svarar på anrop med förprogrammerade svar; testet kontrollerar vad den anropande koden gjorde med svaren. En **mock** kontrollerar dessutom att den blev anropad rätt — rätt metod, rätt argument, rätt antal gånger.

Stub är en stubb. `ScriptedTransport` i kapitel 11 ligger närmare en mock.

6.2.4 Sömmen har redan lönat sig

Argumentet för ett interface brukar vara hypotetiskt. Här är det inte det: en tidigare version av konstruktionen nådde registren via **minnesmappad I/O** på `0xFF200000`, vilket förutsätter en adressbuss ut mot FPGA-väven som mikrokontrollern inte har.

Allt ovanför Interface överlevde bytet till SPI **oförändrat**. Den gamla varianten hade `Fpga` och `volatile`-pekare där vi har `Spi` och `ByteTransport`. Ingen av dem är fel; det är två implementationer under samma söm.

Sammanfattning

- Interfacet är ren abstrakt och har fyra metoder. Destruktorn måste vara `virtual`, annars läcker en implementation som raderas genom en basklasspekare.
- `[[nodiscard]]` på `send()` gör den vanligaste drivrutinsbuggen — ett ignorerat returvärde — till en kompilatorvarning.
- Testet på om en metod hör hemma i interfacet: **kan både Stub och Spi implementera den meningsfullt?**

- `inject()` ligger därför i `Stub`, inte i `Interface`. Testet håller i den konkreta typen; komponenten ser bara interfacet.
- Stubben simulerar medvetet varken fel, upptagen hårdvara eller kö. En stubb som växer till en andra hårdvaruimplementation behöver själv testas.
- Sömnen har redan betalat sig: bytet från minnesmappad I/O till SPI lämnade allt ovanför `Interface` oförändrat.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koden publiceras i kursrepot efter föreläsningen.

Övning 6.1 Förutsäg kompilatorn

RESONEMANG

Svara först, kompilera sedan.

- Du tar bort `virtual` från destruktorn, skapar en `Stub` med `new` och raderar den genom en `Interface*`. Kompilerar det? Vad blir fel?
- Du tar bort `[[nodiscard]]` från `send()` och skriver `driver.send(frame);`. Vad säger kompilatorn, med och utan attributet?
- Du stavar fel på en metod men skriver `override`. Och om du utelämnar `override`?

Fråga c) är skälet till att `override` är obligatoriskt i projektet.

Övning 6.2 En metod för mycket

RESONEMANG

Någon föreslår att lägga till i interfacet:

```
[[nodiscard]] virtual std::uint32_t readStatusRegister() const noexcept = 0;
```

”Den är praktisk vid felsökning.”

- Vad skulle `Stub` returnera från den?
- Vad händer med `EchoNode` om någon börjar använda den?
- Vad händer den dag registerkartan ändras?
- Var hör metoden i stället hemma?

Övning 6.3 Polymorfismen

KOD

I den utdelade `test_stub.cpp` finns en hjälpfunktion som tar en `driver::can::Interface&` i stället för en `Stub&`.

- a) Innehåller den ordet `Stub` någonstans?
- b) Varför är den skriven mot `interface`?
- c) I kapitel 11 anropas samma funktion med en `Spi`. Vad i din kod måste stämma för att det ska gå?

Övning 6.4 Stubbens gränser

RESONEMANG

- a) Räkna upp fyra saker stubben inte simulerar.
- b) Välj en av dem och beskriv hur du skulle kunna lägga till den.
- c) Vad skulle det kosta? När blir en testdubblare så komplicerad att den själv behöver testas?

Komponenttest genom stubben

FÖRELÄSNING 7 · CAN-DRIVROUTINEN

Flera delar tillsammans, fortfarande utan hårdvara.

I DET HÄR KAPITLET

- Skillnaden mot enhetstest, med en bugg bara komponenttestet kan se.
- `app::EchoNode`, och dependency injection.
- Lagergränsen uttryckt i en enda rad kod.
- Vad komponenttestet ändå inte bevisar.

7.1 Buggen ett enhetstest inte kan se

Ett **enhetstest** testar en enhet isolerad. Ett **komponenttest** testar flera delar som samverkar, fortfarande utan hårdvara: här `app::EchoNode` plus `driver::can::Stub`, kopplade genom `driver::can::Interface`.

Skillnaden syns tydligast i vilken sorts bugg de fångar. Anta att `run()` skrivs så här:

```
Frame frame{};
if (!myDriver.receive(frame)) { return; }

Frame response{};
response.id = myResponseId;
response.dlc = myDriver.receive(frame) ? frame.dlc : 0U;    // fel: anropar receive-
    ↪ () igen
```

Varje enhetstest är fortfarande grönt. Stub gör exakt det den lovar, och `Frame` likaså. Buggen ligger i **interaktionen**: det andra anropet konsumerar tillståndet det första lämnade.

Det är hela poängen. Interaktionsbuggar — ett anrop för mycket, ett anrop i fel ordning, ett returvärde som ignoreras — är den vanligaste sortens fel i drivrutinskod, och de bor mellan enheterna, inte i dem.

7.2 app::EchoNode

En applikationskomponent som gör en sak: läser en frame och ekar tillbaka den med ett annat ID.

```
namespace driver::can { class Interface; }

namespace app
{
class EchoNode
{
public:
    explicit EchoNode(driver::can::Interface& driver, std::uint16_t responseId) ↪
        ↪ noexcept;

    void run() noexcept;
    [[nodiscard]] std::uint32_t echoCount() const noexcept;

private:
    driver::can::Interface& myDriver; /**< Driver used for all traffic. Not owned.↪
        ↪ */
    std::uint16_t myResponseId;
    std::uint32_t myEchoCount;
};
} // namespace app
```

7.2.1 Lagergränsen, i en rad

Lägg märke till vad headern *inte* inkluderar: *stub.hpp*, *spi.hpp*, *byte_transport.hpp*. Bara en framåtdeklaration.

Den raden är arkitekturens gräns uttryckt i kod. En granskare som ser `#include "driver/can/spi.hpp"` här ska stoppa PR:en direkt. Regeln, som gäller hela projektet: ingen fil ovanför *driver/can/interface.hpp* får innehålla ordet *SPI*.

7.2.2 Dependency injection

EchoNode **skapar inte** sin drivrutin. Den får den. Alternativet är värt att titta på för att se vad det kostar:

```
class EchoNode
{
public:
    EchoNode() : myDriver{} {} // skapar sin egen Spi
private:
```

```
driver::can::Spi myDriver; // nej
};
```

Den varianten är omöjlig att testa: det finns inget sätt att ge den en stubb, alltså inget sätt att köra den utan ett FPGA på skrivbordet. Den binder dessutom applikationen till en konkret drivrutinstyp, vilket gör hela Interface meningslös.

Varför en referens och inte en pekare? En referens kan inte vara null och kan inte bytas efter konstruktion. Båda är egenskaper vi vill ha.

Varför inte `std::unique_ptr`? Därför att EchoNode inte *äger* drivrutinen. Ägandet ligger hos factoryn i kapitel 12, och drivrutinens livstid är längre än komponentens. Doxygen-kommentaren säger uttryckligen "Not owned", och det är värt att skriva ut.

7.2.3 Två detaljer i run()

Loopen går till `frame.dlc`, inte till `MaxDataLen`. Att kopiera åtta bytes hade "fungerat" eftersom resten ändå är nollor — men `response.dlc` säger hur många som gäller, och kod som kopierar mer än den säger sig kopiera slutar stämma när någon annan förutsättning ändras. Exakt samma resonemang som maskeringen i kapitel 10.

`myEchoCount` räknas upp bara när `send()` lyckades. Räknaren betyder "ekade frames", inte "försök".

7.3 Att läsa komponenttestet

```
TEST(EchoNode, echoesInjectedFrameWithResponseId)
{
    // Arrange.
    driver::can::Stub stub{};
    app::EchoNode node{stub, 0x200U};

    driver::can::Frame incoming{};
    incoming.id      = 0x123U;
    incoming.dlc     = 2U;
    incoming.data[0] = 0xAAU;
    incoming.data[1] = 0xBBU;
    stub.inject(incoming);

    // Act.
    node.run();

    // Assert.
    driver::can::Frame echoed{};
```

```

    EXPECT_TRUE(stub.receive(echoed));
    EXPECT_EQ(echoed.id, 0x200U);
    EXPECT_EQ(node.echoCount(), 1U);
}

```

Testet har två roller samtidigt: det **matar in** trafik med `inject()`, och det **inspekterar** utgående trafik med `receive()` — båda genom samma testdubblare.

Vad den utdelade sviten täcker:

Fall	Vad det bevisar
Ingen injicerad frame	<code>echoCount()</code> oförändrad.
En injicerad frame	Rätt id, dlc och data ekas; räknaren går upp med ett.
Två frames, två <code>run()</code>	Räknaren blir 2.
En frame, <code>run()</code> två gånger	Räknaren blir 1 — inte 2. Fångar buggen i §7.1.
<code>dlc = 0</code> och <code>dlc = 8</code>	Gränsvärdespar.
<code>send()</code> misslyckas	Räknaren går inte upp.

7.3.1 Samma mönster, ett lager ned

Det här är det enda du behöver bära med dig härifrån. I kapitel 11 gör den utdelade sviten **exakt samma sak**, en söm längre ned: BankTransport ersätter Stub, Spi är det som testas, och testet skriptar vad hårdvaran svarar i stället för vilka frames som kommer in.

Att det *är* samma mönster är ingen tillfällighet — det är vad man får av att lägga sömmarna på rätt ställen.

7.3.2 Vad det inte bevisar

Att EchoNode samverkar rätt med *någon* implementation av Interface. Ingenting om Spi, som ännu inte finns, och ingenting om huruvida registerkartan lästs rätt.

Sammanfattning

- Ett komponenttest fångar **interaktionsbuggar** — ett anrop för mycket, i fel ordning, med ignorerat returvärde — som varje enhetstest är grönt trots.
- EchoNode får sin drivrutin genom konstruktorn. Skapar den sin egen är den omöjlig att testa och Interface meningslös.
- En **referens**, inte en pekare: kan inte vara null, kan inte bytas. Och inte en `unique_ptr`: komponenten äger inte drivrutinen.
- Headern framåtdeklarerar Interface och inkluderar ingen konkret drivrutin. Den raden *är* lagergränsen.
- Loopen går till dlc, inte till MaxDataLen. Räknaren räknar lyckade ekon, inte försök.

- Testet matar in trafik och inspekterar utgående trafik genom samma dubblare. Samma mönster återkommer en sön längre ned.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Kod publiceras i kursrepot efter föreläsningen.

Övning 7.1 Skriv buggen, se testet fånga den

KOD

Ändra tillfälligt `run()` så att den anropar `receive()` två gånger.

- Vilka testfall fallerar, och vilka är fortfarande gröna?
- Kör bara enhetstesterna för `Stub` och `Frame`. Fallerar något av dem?
- Skriv ned i två meningar vad övningen visade.

Övning 7.2 Lagergränsen

RESONEMANG

Bryter raden mot arkitekturens lagergräns? Motivera i en mening.

- `#include "driver/can/interface.hpp"` i `echo_node.hpp`.
- `#include "driver/can/stub.hpp"` i `test_echo_node.cpp`.
- `#include "driver/can/stub.hpp"` i `echo_node.cpp`.
- `if (myDriver.hasError()) { myDriver.clearError(); } i run()`.
- `static_cast<driver::can::Spi&>(myDriver).readReg(0U); i run()`.

Rad d) bryter inte mot någon gräns — men är den en bra idé? Vem borde hantera fel?

Övning 7.3 Ägande

RESONEMANG

- Vad händer om du byter `driver::can::Interface& myDriver` mot `driver::can::Interface myDriver`? Vad säger kompilatorn, och varför?
- Vad händer om du skapar en `EchoNode` med en `Stub` som går ur scope innan `run()` anropas? Kompilerar det?
- Varför är drivrutinen en referens och inte en `std::unique_ptr`?

Övning 7.4 Läs sviten

RESONEMANG

Öppna den utdelade `test_echo_node.cpp`.

- Vilket fall skulle fortfarande vara grönt om `run()` glömde räkna upp räknaren?
- Hur får testet in en frame, och hur läser det tillbaka den ekade? Varför finns de metoderna inte i `Interface`?
- Vilket krav uttrycker fallet som anropar `run()` två gånger på en injicerad frame?

SPI och transaktionsprotokollet

FÖRELÄSNING 8 · CAN-DRIVROUTINEN

Kontraktets andra halva: hur registren faktiskt nås.

I DET HÄR KAPITLET

- SPI på fyra ledningar, och varför full duplex spelar roll.
- Transaktionen: en kommandobyte plus fyra databytes, alltid exakt fem.
- Låsning vid läsning, verkställande vid skrivning, avbrott på SS.
- Vad det kostar i tid, och varför SPI-länken är flaskhalsen.

8.1 SPI på fyra ledningar

Ledning	Riktning	Betydelse
SCK	master → slav	Klocka. Ingenting händer utan flanker på den.
MOSI	master → slav	Masterns data.
MISO	slav → master	Slavens data.
SS	master → slav	Aktiv låg. Ramar transaktionen.

Den viktigaste egenskapen: **SPI är full duplex**. Varje klockpuls skiftar samtidigt en bit ut på MOSI och en bit in på MISO. Det finns inget sätt att "bara läsa" — för att få en byte in måste man skicka en byte ut.

Därför skickar en läsning fyra dummybytes 0x00, och därför är de fyra MISO-byten under en skrivning meningslöst skräp.

Parametrarna: SPI mode 0 (klockan vilar låg, data samplas på stigande flank), MSB först, SCK högst 1 MHz.

8.2 Transaktionen

Varje transaktion är exakt fem bytes: en kommandobyte följd av fyra databytes, med SS låg hela vägen.

Bit:	7	6	5	4	3	2	1	0
	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
	W	0	0	0	register index			
	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

W = 1 betyder skrivning, W = 0 läsning. Bitarna 6–4 är reserverade och ska vara 0. Bitarna 3–0 är registerindexet, 0–12, ur registerkartan i kapitel 4.

8.2.1 Skrivning

Mastern skickar registervärdet i de fyra databyten, **MSB först**. Slaven verkställer det hopsatta värdet **först när den femte byten är hel**.

8.2.2 Läsning

I slutet av kommandobyten **läser slaven registrets aktuella värde en gång**, in i sin svarsskiftare. De fyra databyten klockar sedan ut det värdet.

Läsningsregeln är en del av kontraktet, inte en implementationsdetalj. De fyra byten hör alltid till *ett* sammanhängande prov av registret, taget i ett ögonblick. Utan den regeln hade en STATUS-läsning kunnat bli ihopsydd av två tidpunkter — TX-klar-biten från före en sändning och RX-giltig-biten från efter. Det hade varit en bugg som uppträder ungefär en gång på tusen och aldrig går att återskapa.

8.2.3 SS-reglerna

Tre regler, och alla tre har kostat någon en kväll:

- **En transaktion per låg SS-period.** Mastern måste släppa SS hög mellan transaktioner.
- **Back-to-back fungerar inte, och misslyckas tyst.** En sjätte byte som kommer medan SS fortfarande är låg *kastas* — den tolkas inte som en ny kommandobyte. Att slaven är ledig räcker alltså inte.
- **Avbrott har inga sidoeffekter.** Går SS hög före femte byten överges transaktionen helt: ingenting verkställs, ingen trigger går. Det är vad som gör slaven självläkande efter en glitch.

Det är därför ByteTransport i kapitel 9 har `select()` och `deselect()` som egna metoder: ramningen är drivrutinens beslut, inte transportens.

8.2.4 MISO utanför en läsning

Byten mastern klockar ut **under kommandobyten** är meningslös. Slaven laddar sin svarsskiftare i *slutet* av kommandobyten, så det som lämnar under den är rester från förra gången. **Kasta den.**

8.2.5 Ett komplett exempel

Att skicka en frame (id 0x123, dlc 2, data AA BB):

```
Skriv TX_ID      : 81 00 00 01 23
Skriv TX_DLC     : 82 00 00 00 02
Skriv TX_DATA_LO : 83 AA BB 00 00
Skriv TX_DATA_HI : 84 00 00 00 00
Skriv TX_SEND    : 85 00 00 00 01
Läs  STATUS     : 00 xx xx xx xx  -> MISO ger 00 00 00 00 under sändning,
                                   sedan 00 00 00 01 när porten är fri igen.
```

Läs av kommandobyten i varje rad: 0x81 är skrivbiten plus index 1, alltså TX_ID. 0x00 på sista raden är en läsning av index 0.

Notera 83 AA BB 00 00: **databyte 0 i den mest signifikanta positionen**. Det är samma packning på båda sidor av ledningen.

8.3 Vad det kostar i tid

En transaktion är 5 bytes = 40 bitar. Vid 1 MHz tar klockningen 40 μs , plus SS-hantering. Räkna med **40–50 μs per transaktion**:

Operation	Transaktioner	Ungefär
hasError()	1	45 μs
receive(), ingen frame	1	45 μs
receive(), en frame	6	270 μs
send()	6	270 μs

Sätt det mot CAN-sidan: en maximal CAN-frame är omkring 130 bitar, alltså **130 μs** vid 1 Mbit/s.

En send() tar längre tid än framen den skickar. SPI-länken, inte CAN-bussen, är flaskhalsen i den här konstruktionen.

Och pollningen kan missa frames. Kontrollern buffrar inte: kommer två frames tätt inpå varandra och din receive() tar 270 μs , hinner den andra skriva över den första. Det är hårdvarans dokumenterade begränsning, inte din bugg — men det är din kod som märker den.

Sammanfattning

- SPI är **full duplex**: varje klockpuls skiftar en bit åt vardera hållet. En läsning måste därför skicka dummybytes.

- Varje transaktion är **exakt fem bytes**: kommandobyte plus fyra databytes, MSB först, med SS låg hela vägen.
- Vid läsning **läses** registervärdet en gång, i slutet av kommandobyten. De fyra byten hör alltid till ett sammanhängande prov.
- Byten som klockas in under kommandobyten är skräp. Kasta den.
- SS måste gå hög mellan transaktioner. Två transaktioner under en låg period misslyckas *tyst*.
- En transaktion kostar 40–50 μs . En `send()` är sex av dem, alltså längre tid än den CAN-frame den skickar.

Övningar

Papper och penna. Lösningarna står i [appendix A](#).

Övning 8.1 Avkoda

AVKODNING

Följande bytes går ut på MOSI. För var och en: läsning eller skrivning, vilket register, och vilket värde?

- a) 81 00 00 07 FF
- b) 85 00 00 00 01
- c) 8B 00 00 00 00
- d) C2 00 00 00 03

Två av dem är ovanliga på var sitt sätt. Vilka, och varför?

Övning 8.2 Koda

RÄKNING

Skriv upp den kompletta femBytestransaktionen för var och en:

- a) Läs RX_DLC.
- b) Skriv TX_DLC = 4.
- c) Trigga en sändning.
- d) Kvittera en mottagen frame.
- e) Nollställ felregistret.

Notera särskilt skillnaden mellan d) och e).

Övning 8.3 Vad kommer tillbaka

RESONEMANG

- a) Under en skrivning klockar mastern in fem bytes på MIS0. Hur många betyder något?
- b) Under en läsning av STATUS, hur många av de fem betyder något, och vilka?
- c) En drivrutin sätter ihop returvärdet av *alla fem* mottagna byten i stället för de fyra sista. Vad händer, och hur skulle det se ut i felsökningen?

Övning 8.4 SS och avbrott

RESONEMANG

Vad händer i vart och ett av fallen?

- a) SS går hög efter tre av fem bytes i en skrivning till TX_ID.
- b) Mastern håller SS låg och skickar tio bytes i sträck: 81 00 00 01 23 82 00 00 00 02.
- c) Mastern skickar kommandobyten 0x91.

Fall b) är det farliga. Varför, och hur skulle symptomet se ut vid bring-upen?

Övning 8.5 Tidsbudget

RÄKNING

Vid 1 MHz SCK:

- a) Hur lång tid tar en `receive()` som hittar en frame?
- b) Hur lång tid tar en maximal CAN-frame på bussen vid 1 Mbit/s?
- c) En annan nod sänder frames så fort den kan. Vad händer med dem som kommer medan din `receive()` pågår? Är det din bugg?

Transportsömmen och registeråtkomsten

FÖRELÄSNING 9 · CAN-DRIVROUTINEN

Protokollet blir kod, och drivrutinen blir körbar på en laptop.

I DET HÄR KAPITLET

- ByteTransport: varför Interface inte räckte.
- Registerindex, statusmasker och kommandobiten som namngivna konstanter.
- `readReg()` och `writeReg()`, rad för rad.
- Teckenutvidgningsbuggen, som är projektets vanligaste.

9.1 Varför ännu en söm

Interface gjorde Stub och Spi utbytbara, och därmed EchoNode testbar. Men Spi själv ligger *under* den sömmen. Utan något mer går den bara att köra på riktig hårdvara.

Räkna på vad det skulle betyda. Spi blir klar i kapitel 10. Hårdvaran finns i kapitel 13. Däremellan skulle du ha en oprövad klass som ingen kört en enda gång — och första gången den körs görs det på ett kort som samtidigt felsöks av en annan grupp, över fyra ledningar som kanske sitter rätt.

Det är inte ett bra sätt att hitta ett bitskiftningsfel.

```
namespace driver::can
{
/**
 * @brief Moves single bytes to and from the CAN controller, and frames ↵
 *         ↵ transactions.
 *
 *
 *         Knows nothing about registers, command bytes or CAN: it only carries ↵
 *         ↵ bytes.
```

```

    */
class ByteTransport
{
public:
    virtual ~ByteTransport() noexcept = default;

    virtual void select() noexcept = 0;    /**< Assert SS; begin a transaction. ↵
    ↵ */
    virtual void deselect() noexcept = 0;  /**< Release SS; end the transaction. ↵
    ↵ */

    [[nodiscard]] virtual std::uint8_t transfer(std::uint8_t byte) noexcept = 0;
};
} // namespace driver::can

```

Tre metoder. Det är hela lagret.

Varför `transfer()` returnerar en `byte`. SPI är full duplex; en signatur som vore `void transfer(std::uint8_t)` skulle ljuga om vad hårdvaran gör.

Varför `select()/deselect()` är egna metoder. SS-ramningen är en del av *protokollet*, inte av hur en byte överförs. Transporten ska inte behöva veta att en transaktion är fem bytes lång. Håll den dum, så gör den bara en sak — och är lätt att implementera rätt både på målet och i ett test.

Varför `noexcept` överallt. Lagret körs på ett friställt mål utan undantag. Att skriva det i signaturen redan nu gör det omöjligt att smyga in något som kastar.

9.2 Konstanterna

```

class Spi final : public Interface
{
    // ...
private:
    /** Register indices, as sent in the command byte. */
    static constexpr std::uint8_t RegStatus{0U};
    static constexpr std::uint8_t RegTxId{1U};
    static constexpr std::uint8_t RegTxDlc{2U};
    static constexpr std::uint8_t RegTxDataLo{3U};
    static constexpr std::uint8_t RegTxDataHi{4U};
    static constexpr std::uint8_t RegTxSend{5U};
    static constexpr std::uint8_t RegRxId{6U};
    static constexpr std::uint8_t RegRxDlc{7U};
    static constexpr std::uint8_t RegRxDataLo{8U};
    static constexpr std::uint8_t RegRxDataHi{9U};

```

```

static constexpr std::uint8_t RegRxAck{10U};
static constexpr std::uint8_t RegErrorFlags{11U};
static constexpr std::uint8_t RegTxAbort{12U};

/** STATUS register bit masks. */
static constexpr std::uint32_t StatusTxReady{1U << 0U};
static constexpr std::uint32_t StatusRxValid{1U << 1U};
static constexpr std::uint32_t StatusError{1U << 2U};

/** Command byte: bit 7 set means write. */
static constexpr std::uint8_t CmdWrite{0x80U};

/** Value written to the write-triggered registers. */
static constexpr std::uint32_t Trigger{0x1U};

ByteTransport& myTransport; /**< Transport used for every register access. Not-
    owned. */
};

```

Index, inte offset. Registerkartans offsetkolumn är ett arv från den minnesmappade varianten. Det som går ut på ledningen är indexet, alltså offset/4. Skriv konstanterna som index direkt, så slipper du en division vid varje anrop och en klass av fel där någon dividerar två gånger eller ingen gång.

1U << 0U och inte 0x01U. Masken skrivs som en skiftning därför att registerkartan är skriven i bitpositioner. StatusRxValid{1U << 1U} går att jämföra rad för rad mot tabellen; 0x02U gör det inte.

9.3 De två hjälpfunktionerna

```

void Spi::writeReg(const std::uint8_t index, const std::uint32_t value) const noexcept
{
    myTransport.select();

    static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(CmdWrite |
    index)));
    static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value >> 24U)));
    static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value >> 16U)));
    static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value >> 8U)));
}

```

```

static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value)));

myTransport.deselect();
}

std::uint32_t Spi::readReg(const std::uint8_t index) const noexcept
{
    myTransport.select();

    static_cast<void>(myTransport.transfer(index));

    std::uint32_t value{};
    value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 24U;
    value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 16U;
    value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 8U;
    value |= static_cast<std::uint32_t>(myTransport.transfer(0U));

    myTransport.deselect();
    return value;
}

```

Läs `writeReg()` mot 81 00 00 01 23 i kapitel 8. Fyra saker att lägga märke till:

Varje `static_cast<void>` är avsiktlig. `transfer()` är `[[nodiscard]]`, och under en skrivning är returvärdet meningslöst. Casten säger till både kompilatorn och läsaren att det är känt.

I `readReg()` kastas den första byten. Slaven laddar sin svarsskiftare i *slutet* av kommandobyten. Fyra `transfer()` bidrar till värdet, inte fem.

Kommandobyten är bara `index` vid läsning. Bit 7 låg betyder läsning; ingen `CmdWrite`.

Metoderna är `const`. De ändrar inte `Spi`-objektets tillstånd — referensen i sig ändras inte. Att hårdvaran ändras är en annan sak; `const` i C++ talar om objektet, inte om världen. Det är därför `hasError()` `const` kan anropa `readReg()`.

9.3.1 Teckenutvidgningsbuggen

Det här är projektets vanligaste bugg, och den är värd en egen rubrik:

```

value |= myTransport.transfer(0U) << 24U; // fel
value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 24U; // rätt

```

`transfer()` returnerar en `std::uint8_t`. I den första raden befordras den till int — 32 bitar *med tecken* — och skiftas 24 steg. För ett värde med bit 7 satt skriver det in i teckenbiten. Resultatet är odefinierat beteende, och i praktiken ett negativt tal som sedan konverteras till `std::uint32_t` med `0xFF`-bytes där det inte ska finnas några.

Symptomet: drivrutinen fungerar för små värden och går sönder så fort en byte har bit 7 satt. Alltså fungerar den för 0x123 och går sönder för 0xAABB.

Regel: Casta före skiftningen, inte efter. Samma regel gäller packningen i kapitel 10.

9.3.2 Varför hjälpfunktionerna är privata

De är implementationsdetaljer. Ingen utanför klassen ska kunna peta i registren, av samma skäl som Interface inte har en `readStatusRegister()`: den dag någon gör det i applikationskoden är arkitekturen bruten.

Den utdelade testsviten testar dem ändå — genom `send()` och `receive()`, och genom att inspektera vilka bytes som kom ut. Att testa en privat metod direkt är nästan alltid ett tecken på att den borde vara publik, eller på att testet testat fel sak.

9.3.3 Vad lagret inte gör

`readReg()` och `writeReg()` är rena översättningar av protokollet och innehåller ingen logik alls: ingen validering, ingen statuskontroll, inget begrepp om vad ett register betyder.

All betydelse ligger i lagret ovanför, och det är kapitel 10. Så fort `writeReg()` börjar innehålla "om index är `RegTxSend`, kontrollera först att..." har två lager smält ihop, och båda blev svårare att testa.

Sammanfattning

- Interface gör applikationen testbar; `ByteTransport` gör **drivrutinen** testbar. Utan den andra sömmen körs `Spi` första gången på hårdvara som samtidigt felsöks av någon annan.
- Transporten är medvetet dum: `select()`, `deselect()`, `transfer()`. Den vet inte att en transaktion är fem bytes lång — ramningen tillhör protokollet.
- Registerkonstanterna är **index**, inte offsetar, och maskerna skrivs som skiftningar så att de går att jämföra rad för rad mot registerkartan.
- `readReg()` kastar byten som klockas in under kommandobyten. Fyra av fem bidrar.
- **Casta före skiftningen.** En `std::uint8_t` befordras annars till `int`, och en byte med bit 7 satt skiftad 24 steg skriver i teckenbiten.
- Hjälpfunktionerna är privata och innehåller ingen logik. All betydelse ligger i lagret ovanför.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koderna publiceras i kursrepot efter föreläsningen.

Övning 9.1 Förutsäg

RÄKNING

Svara innan du kompilerar.

- Vad blir kommandobyten för en skrivning till `RegTxDataHi`? För en läsning av samma?
- Vilken sekvens av fem bytes producerar `writeReg(RegTxDataLo, 0xAABB0000U)`?
- Vad returnerar `readReg()` om transporten svarar FF 12 34 56 78 på de fem `transfer()`-anropen?
- Vad returnerar den om du av misstag räknar med alla fem mottagna byten?

Övning 9.2 Teckenutvidgningen

KOD

Skriv `readReg()` en gång **fel**, med skiftningen före casten. Låt transporten svara så att den första databyten är `0x80`.

- Vad blir `value`, och varför?
- För vilka värden fungerar den felaktiga koden ändå?
- Hur skulle symptomet se ut om buggen låg kvar till bring-up'en? Vilka id-värden hade fungerat?

Övning 9.3 En transport att felsöka med

KOD

Skriv en `PrintTransport` som implementerar `ByteTransport` och skriver ut varje anrop:

```
select
-> 81  <- 00
-> 00  <- 00
-> 00  <- 00
-> 01  <- 00
-> 23  <- 00
deselect
```

Jämför utskriften rad för rad mot exemplet i kapitel 8. Den är inte en testdubblare utan ett felsökningsverktyg, och den är värd att behålla hela projektet.

Övning 9.4 Lagergränsen

RESONEMANG

Sant eller falskt, med motivering:

- `byte_transport.hpp` får inkludera `frame.hpp`.
- `ByteTransport` borde ha en metod `writeRegister(index, value)`.
- `ByteTransport` borde göra hela femBytestransaktionen i ett anrop.
- `readReg()` borde vara publik så att testerna kan anropa den direkt.

Påstående c) låter mest rimligt och är ändå fel. Varför?

KAPITEL 10

Drivrutinen

FÖRELÄSNING 10 · CAN-DRIVRUTINEN

De fyra metoderna, och begränsningarna de inte får dölja.

I DET HÄR KAPITLET

- `send()`: validering, statuskontroll, fem skrivningar, en trigger — i den ordningen.
- Packning och upppackning över registerpar.
- `receive()`, och varför den maskerar mot dlc trots att hårdvaran redan nollar.
- `clearError()`, som skriver noll och inte ett.

10.1 `send()`

```
bool Spi::send(const Frame& frame)
{
    if (!isValid(frame))                { return false; }
    if ((readReg(RegStatus) & StatusTxReady) == 0U) { return false; }

    writeReg(RegTxId,    frame.id);
    writeReg(RegTxDlc,    frame.dlc);
    writeReg(RegTxDataLo, packDataLo(frame));
    writeReg(RegTxDataHi, packDataHi(frame));
    writeReg(RegTxSend,   Trigger);

    return true;
}
```

10.1.1 Ordningen är inte godtycklig

`TX_SEND` **måste** skrivas sist. Den skrivningen betyder inte "spara ett värde" utan "börja sända nu", och kontrollern läser då `TX_ID`, `TX_DLC` och de två dataregistren. Skriv triggern först

och du skickar förra framens innehåll — vilket vid bring-upen ser ut som en nod som ligger exakt en frame efter.

Statuskontrollen måste komma **före** skrivningarna. Skulle du skriva TX_ID medan en sändning pågår skriver du över data kontrollern håller på att använda.

10.1.2 Två false, två betydelser

Interfacet har bara en bool att uttrycka båda med:

Orsak	Vad anroparen bör göra
Ogiltig frame	Rätta ramen. Väntan hjälper aldrig.
Hårdvaran upptagen	Vänta och försök igen.

Det är en medveten förenkling, men värd att vara medveten om. Två saker följer:

Ordningen spelar roll. Valideringen först, så att en ogiltig frame avvisas *utan* att en enda transaktion går ut på bussen. Den utdelade testsviten kontrollerar just det.

Det blir ett problem exakt en söm högre upp. Den här loopen är en oändlig loop för en frame med dlc = 9:

```
while (!driver.send(frame))    // hänger för alltid om ramen är ogiltig
{
}
```

Rätt hantering är att validera innan man går in i loopen, eller att räkna försök och ge upp.

10.2 Packningen

Databyte 0 ligger i de **mest signifikanta** bitarna, och TX_DATA_L0 bär de **första** fyra bytesen.

```
/**
 * @brief Pack four payload bytes into one register word, byte 0 in the most ~
 * ~ significant bits.
 */
[[nodiscard]] std::uint32_t pack(const std::uint8_t* const data,
                                const std::uint8_t offset) noexcept
{
    return (static_cast<std::uint32_t>(data[offset + 0U]) << 24U)
        | (static_cast<std::uint32_t>(data[offset + 1U]) << 16U)
        | (static_cast<std::uint32_t>(data[offset + 2U]) << 8U)
        | (static_cast<std::uint32_t>(data[offset + 3U]));
}
```

Samma cast-regel som i kapitel 9: **casta före skiftningen**.

De två vanligaste packningsfelen:

Fel	Symptom
TX_DATA_LO och TX_DATA_HI förväxlade	Framen kommer fram med byte 4–7 först. Syns direkt i en bussanalysator, och inte alls i simulering om testet också har fel.
Skiftningarna i omvänd ordning	AA BB CC DD kommer fram som DD CC BB AA.

Båda ger en frame som *kommer fram* — bara med fel innehåll. Det är den obehagligaste sortens fel, för allt *ser* ut att fungera.

10.3 receive()

```
bool Spi::receive(Frame& frame)
{
    if ((readReg(RegStatus) & StatusRxValid) == 0U) { return false; }

    frame.id = static_cast<std::uint16_t>(readReg(RegRxId) & 0x7FFU);
    frame.dlc = static_cast<std::uint8_t>(readReg(RegRxDlc) & 0xFU);

    unpack(readReg(RegRxDataLo), frame.data, 0U);
    unpack(readReg(RegRxDataHi), frame.data, 4U);

    for (std::uint8_t i{frame.dlc}; i < Frame::MaxDataLen; ++i)
    {
        frame.data[i] = 0U;
    }

    writeReg(RegRxAck, Trigger);
    return true;
}
```

10.3.1 Maskeringen mot dlc

Loopen ser överflödig ut. Registerkartan säger ju att kontrollern nollställer sin dataackumulator vid varje framestart, så de bytesen *är* redan noll.

Gör det ändå, av tre skäl:

- 1. Nollorna är kontrollerns garanti, inte framens innehåll.** En drivrutin som kopierar åtta bytes rakt av rapporterar upp till sex bytes som aldrig sändes. Att de råkar vara noll gör dem inte till data.
- 2. Garantin kan försvinna.** Den kommer från en implementationsdetalj i hårdvarans VHDL. Ändras den, eller lånar du ett kort från en annan grupp, är din drivrutin fortfarande rätt.

3. **dlc är kontraktet.** `frame.dlc = 2` betyder "två bytes gäller". Kod som lämnar ifrån sig ett objekt vars fält säger emot varandra gör nästa läsare osäker.

10.3.2 RX_ACK sist

Kvittensen frigör `RX_*`-registren och nollställer `STATUS` bit 1. Skriv den **efter** att du läst allt — annars kan nästa frame skriva över registren mitt i din läsning.

Glöm den helt, och bit 1 står kvar för alltid: varje `receive()` returnerar `true` med samma frame, i en oändlig loop. Det är ett symptom värt att känna igen.

10.4 `hasError()` och `clearError()`

```
bool Spi::hasError() const noexcept
{
    return (readReg(RegStatus) & StatusError) != 0U;
}

void Spi::clearError() noexcept
{
    writeReg(RegErrorFlags, 0U);
}
```

Två rader, och en av dem är en fälla.

`clearError()` skriver `0x0`, inte `0x1`. `ERROR_FLAGS` är en lås-och-nollställ-på-noll-konstruktion: bara en skrivning där hela ordet är noll nollställer läsningen. `0x1` gör ingenting, och `0x000000FF` gör ingenting trots att bit 0 är satt.

Det är motsatt regel mot `TX_SEND` och `RX_ACK`, som kräver att bit 0 *är* satt. Att de tre registren har olika regler är inte elegant, men det står i kartan, och det är kartan som gäller.

Symptomet om du skriver `0x1`: felbiten går aldrig ned, `hasError()` returnerar `true` för alltid efter första felet, och all felhantering ovanför slutar fungera.

10.5 Att vänta på en sändning

`send()` **väntar inte.** Den returnerar så fort triggern är skriven, och det är rätt: en drivrutinsmetod som blockerar i upp till 130 μ s på en 4 MHz-MCU äter hela systemets tidsbudget.

Hur väntar anroparen då? Interfacet har varken `txReady()` eller `waitForSend()` — avsiktligt, eftersom Stub inte kunnat implementera dem meningsfullt. Det anroparen har är att `send()` returnerar `false` medan sändaren är upptagen. **Väntan är alltså ett nytt försök:**

```
driver.clearError();
```

```

if (!isValid(frame)) { return false; } // never going to be accepted, however ↪
    ↪ long we wait

std::uint16_t attempts{};
while (!driver.send(frame))
{
    if (++attempts >= MaxAttempts) { return false; }
}

// The port was free when the frame was accepted, so the previous transmission had↪
    ↪ finished.
if (driver.hasError())
{
    driver.clearError();
    // Lost arbitration, stuff error, bad CRC or a remote frame - the latch does ↪
        ↪ not say which.
}

```

En förlorad arbitrerings ser ut som en lyckad sändning. STATUS bit 0 kommer tillbaka i båda fallen, och betyder *porten är fri*, inte *ramen kom fram*. Vill du veta utfallet måste du läsa ERROR_FLAGS efter att biten kommit tillbaka — och även då får du veta *att* något gick fel, aldrig *vad*.

10.6 Checklista

- ☐ send() validerar **först**, och skickar ingen transaktion för en ogiltig frame.
- ☐ send() kontrollerar StatusTxReady innan den skriver något.
- ☐ TX_SEND skrivs **sist**.
- ☐ TX_DATA_L0 bär databyte 0–3, med byte 0 i bitarna 31–24.
- ☐ Varje static_cast<std::uint32_t> står **före** skiftningen.
- ☐ receive() kontrollerar StatusRxValid först.
- ☐ receive() skriver RX_ACK **efter** alla läsningar.
- ☐ receive() nollställer frame.data ovanför frame.dlc.
- ☐ clearError() skriver 0x0.
- ☐ Ingen magisk siffra i hela filen.

Sammanfattning

- send(): validera, kontrollera status, skriv fyra dataregister, trigga **sist**. Triggern betyder "börja sända nu", och kontrollern läser då de övriga registren.

- `send()` returnerar `false` av två skäl, och bara det ena går över av sig självt. `while (!send(frame)) {}` hänger för alltid på en ogiltig frame.
- `TX_DATA_LO` bär databyte 0–3. Förväxling ger en frame som kommer fram bakvänd — allt ser ut att fungera.
- `receive()` maskerar mot dlc trots att hårdvaran redan nollar: nollorna är kontrollernas garanti, inte framens innehåll.
- `RX_ACK` skrivs sist. Glöms den läser du samma frame i en oändlig loop.
- `clearError()` skriver 0x0. Skriver du 0x1 går felbiten aldrig ned.
- `send()` väntar inte. Väntan är ett nytt försök, och sker hos anroparen.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koderna publiceras i kursrepot efter föreläsningen.

Övning 10.1 Bryt koden med flit

KOD

Gör en ändring i taget, kör mot din `PrintTransport`, och notera vad utskriften visar och hur felet skulle yttra sig på en riktig buss. Återställ efter varje.

- `TX_SEND` skrivs först i stället för sist.
- `TX_DATA_LO` och `TX_DATA_HI` byter plats.
- Skiftningarna inom `pack()` i omvänd ordning.
- `clearError()` skriver 0x1.
- `receive()` glömmar `RX_ACK`.
- `receive()` maskerar inte mot dlc.
- `send()` hoppar över statuskontrollen.

Rad f) är den svåraste: utskriften ser *rätt ut*. Varför, och vilket testfall i kapitel 11 fångar den ändå?

Övning 10.2 Räkna transaktioner

RÄKNING

Skriv ned utan att köra, och kontrollera sedan.

- En `send()` som lyckas?
- En `send()` som returnerar `false` på grund av upptagen hårdvara?
- En `send()` som returnerar `false` på grund av `dlc = 9`?
- En `receive()` som inte hittar någon frame?
- En `receive()` som hittar en?

Fråga c) är poängen: valideringen ligger först *just för att* en ogiltig frame inte ska kosta en enda transaktion.

Övning 10.3 Den oändliga loopen

KOD

- a) Kör `while (!driver.send(frame)) {}` med en giltig frame mot en transport som säger att hårdvaran är upptagen. Vad händer?
- b) Kör samma loop med `frame.dlc = 9`. Vad händer?
- c) Skriv om loopen så att båda fallen hanteras.

Övning 10.4 Designfrågor

RESONEMANG

Korta svar; de hör hemma i projektrapporten.

- a) `send()` returnerar `false` av två helt olika skäl. Vad hade du ändrat i `Interface` för att skilja dem åt, och vad hade det kostat `Stub`?
- b) Ge ett scenario där maskeringen mot `dlc` faktiskt gör skillnad.
- c) Din nod sänder en frame och förlorar arbitrerings. Beskriv steg för steg vad drivrutinen observerar — och vad den *inte* kan observera.
- d) Kontrollern buffrar inte mottagna frames. Vilken ändring i *din* kod minskar risken att tappa en frame mest, och vilken gräns kommer du ändå inte förbi?

Testning utan hårdvara

FÖRELÄSNING 11 · CAN-DRIVRUTINEN

Två testdubblare, två frågor, och hela stacken på en laptop.

I DET HÄR KAPITLET

- ScriptedTransport: skickade drivrutinen rätt bytes?
- BankTransport: beter sig drivrutinen rätt?
- Vilken bugg den ena fångar och den andra missar.
- Vad en grön testsvit inte bevisar.

11.1 Varför två

ByteTransport är ett litet interface — tre metoder — och just därför går det att implementera på två helt olika sätt, som svarar på två olika frågor.

Dubblare	Frågan	Vad den är
ScriptedTransport	<i>Skickade drivrutinen rätt bytes?</i>	Bandspelare: spelar upp förprogrammerade svar, spelar in allt som skickades.
BankTransport	<i>Beter sig drivrutinen rätt?</i>	Simulator: en registerbank i minnet, med hårdvarans semantik.

Båda är utdelade. De överlappar inte, och båda behövs:

Bugg	Scripted	Bank
clearError() skriver 0x1	Ja, direkt	Ja, indirekt
receive() maskerar inte mot dlc	Nej	Ja
TX_SEND skriven före dataregistren	Ja	Nej
TX_DATA_L0/HI förväxlade	Ja	Nej

Rad två och tre är skälet till att det inte räcker med en av dem.

11.2 ScriptedTransport

Den gör tre saker: spelar upp MISO-bytes ur ett skript, spelar in varje MOSI-byte, och räknar `select()/deselect()`-par.

```
TEST(Spi, writeRegSendsCommandByteThenValueMsbFirst)
{
    // Arrange.
    ScriptedTransport transport{};
    driver::can::Spi driver{transport};

    driver::can::Frame frame{};
    frame.id      = 0x123U;
    frame.dlc     = 2U;
    frame.data[0] = 0xAAU;
    frame.data[1] = 0xBBU;

    // Act.
    static_cast<void>(driver.send(frame));

    // Assert: the TX_DATA_LO transaction.
    const auto& sent{transport.sent()};
    EXPECT_EQ(sent[10], 0x83U); // CmdWrite | RegTxDataLo
    EXPECT_EQ(sent[11], 0xAAU);
    EXPECT_EQ(sent[12], 0xBBU);
    EXPECT_EQ(sent[13], 0x00U);
    EXPECT_EQ(sent[14], 0x00U);
}
```

Läs det som protokollet, inte som kod. De fem raderna i Assert-delen är 83 AA BB 00 00 ur kapitel 8, skrivet som ett krav. Hela den testfilen är i praktiken SPI-protokollet uttryckt i C++ — och därmed en exaktare kravspecifikation än prosan.

11.3 BankTransport

Den mer ambitiösa. Den tolkar kommandobyten, håller tretton register i minnet, och härmar hårdvarans semantik: maskeringen av TX_ID/TX_DLC, att en skrivning till TX_SEND nollställer TX-klar, att RX_ACK nollställer RX-giltig, och att bara ett nollställt ord nollställer ERROR_FLAGS.

Plus några metoder som bara testet använder, för att simulera vad kontrollern gör:

```
void completeTransmission() noexcept;           /**< TX ready goes high again. ↪
    ↪ */
```

```

void deliverFrame(const driver::can::Frame& frame) noexcept; /**< RX valid goes ↵
    ↵ high. */
void raiseError() noexcept;                               /**< The controller latches an ↵
    ↵ error. */

```

Med dem kan ett testfall skriptas ett helt förlopp — till exempel en frame med dlc = 2 som följer på en med dlc = 8, vilket är exakt det fall maskeringsloopen i kapitel 10 finns för.

11.4 Hela stacken på en laptop

```

TEST(EchoNode, echoesThroughTheRealDriver)
{
    // Arrange.
    BankTransport bank{};
    driver::can::Spi driver{bank};
    app::EchoNode node{driver, 0x200U};

    driver::can::Frame incoming{};
    incoming.id      = 0x123U;
    incoming.dlc     = 2U;
    incoming.data[0] = 0xAAU;
    incoming.data[1] = 0xBBU;
    bank.deliverFrame(incoming);

    // Act.
    node.run();

    // Assert.
    EXPECT_EQ(node.echoCount(), 1U);
    EXPECT_EQ(bank.readRegister(1U), 0x200U); // TX_ID
    EXPECT_EQ(bank.readRegister(3U), 0xAABB0000U); // TX_DATA_L0
}

```

Jämför med testfallet i kapitel 7. Det är **samma testfall**, med Stub utbytt mot Spi + BankTransport. EchoNode är oförändrad.

Det är arkitekturen som betalar tillbaka: sötmen i Interface gjorde bytet möjligt, och sötmen i ByteTransport gjorde det möjligt *utan hårdvara*.

11.5 Från fallerande testfall till bugg

```
FAILED: EXPECT_EQ(sent[11], 0xAAU) at test_spi.cpp:87
```

Arbetsgången, och **utan att öppna din egen kod i steg 1–3**:

1. **Testnamnet.** `writeRegSendsCommandByteThenValueMsbFirst` — kravet gäller hur ett värde skrivs ut, MSB först.
2. **Indexet.** `sent[11]` är den andra byten i den tredje transaktionen. Räkna: 0–4 är STATUS-läsningen, 5–9 är `TX_ID`, 10–14 är `TX_DLC...` eller `TX_DATA_L0`? Att räkna fel här är vanligt; skriv ut hela inspelningen och titta.
3. **Det förväntade värdet.** `0xAA` är `frame.data[0]`. Kravet är alltså att databyte 0 hamnar först.
4. **Nu**, och först nu: `pack()`. Ligger `data[0]` i bitarna 31–24?

11.6 Vad en grön svit inte bevisar

Det här är viktigare än det låter.

En grön svit bevisar att din kod stämmer med **testförfattarens tolkning av kontraktet**. Den bevisar inte:

- Att **tolkningen** är rätt. Testsviten är skriven av en människa som läste samma dokument som du.
- Att **hårdvarugruppen** implementerat samma tolkning. Deras testbänkar är skrivna mot samma dokument, av en annan människa.
- Att **kopplingen** är rätt. En omkastad MOSI/MISO är osynlig för varje test på laptopen.
- Att **tidsbudgeten** håller. BankTransport svarar omedelbart; en riktig SPI-länk gör det inte.
- Att du **pollar tillräckligt ofta** för att inte tappa frames.

Allt det fångas bara av kapitel 13 och kapitel 14. Men allt *under* de punkterna ska vara grönt innan du kopplar in något: **en grön svit gör inte bring-upen enkel; den gör den möjlig**, genom att ta bort ett par hundra felkällor innan du står vid bänken.

Sammanfattning

- Två testdubblare, två frågor. ScriptedTransport kontrollerar *vilka bytes* som skickades; BankTransport kontrollerar *hur drivrutinen beter sig*.
- De överlappar inte. En utebliven maskering mot `dLc` fångas bara av den ena, en felordnad trigger bara av den andra.

- ScriptedTransport-filen är SPI-protokollet skrivet som kod — och därmed en exaktare kravspecifikation än prosan.
- Med BankTransport körs **hela stacken** på laptopen. Testfallet är detsamma som i kapitel 7, med Stub utbytt. EchoNode är oförändrad.
- Läs ett fallerande testfall i ordningen testnamn → index → förväntat värde → din kod.
- En grön svit bevisar att koden stämmer med testförfattarens *tolkning*. Kopplingen, tidsbudgeten och den andra gruppens tolkning står utanför.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koden publiceras i kursrepot efter föreläsningen.

Övning 11.1 Få sviten grön

KOD

Lägg in de utdelade filerna oförändrade och kör dem. Notera hur många testfall som fallerar första gången.

Arbeta igenom dem ett i taget, och för logg:

Testfall	Vad namnet sa	Var buggen låg	Minuter
----------	---------------	----------------	---------

Kolumnen ”minuter” brukar falla dramatiskt under uppgiftens gång, och skälet är att man slutar gissa.

Övning 11.2 Vilken dubblare fångar vad?

RESONEMANG

För var och en: ScriptedTransport, BankTransport, båda, eller ingen? Motivera i en mening, och **kontrollera sedan genom att införa buggen**.

- TX_DATA_L0 och TX_DATA_HI förväxlade.
- clearError() skriver 0x1.
- receive() maskerar inte mot dlc.
- receive() glömmer RX_ACK.
- readReg() räknar med kommandobystens MISO-svar.
- send() validerar inte dlc.
- MOSI och MISO omkastade i kopplingen.

Rad g) är med av ett skäl. Vilket?

Övning 11.3 Läs sviten som kravspecifikation

RESONEMANG

Öppna den utdelade `test_spi.cpp`.

- a) Vilket testfall motsvarar exemplet `83 AA BB 00 00` i kapitel 8?
- b) Vilket testfall kontrollerar att en ogiltig frame inte kostar en enda transaktion? Hur kontrollerar det den saken?
- c) Hur många testfall använder vardera dubblaren? Stämmer fördelningen med resonemanget i §11.1?

Övning 11.4 Hela stacken

KOD

Kör testfallet som kopplar `EchoNode` → `Spi` → `BankTransport`.

- a) Hur många rader har ändrats i `EchoNode` sedan kapitel 7?
- b) Vilken egenskap hos arkitekturen gjorde bytet gratis?
- c) Vad hade behövt ändras om `EchoNode` tagit en `driver::can::Spi` i konstruktorn?

Övning 11.5 Vad återstår

RESONEMANG

Skriv en lista på allt som fortfarande kan vara fel trots att hela sviten är grön. Sikta på minst fem punkter, varav minst en du hittat på egen hand.

Listan är din checklista inför kapitel 13.

KAPITEL 12

Hårdvaran, transporten och factoryn

FÖRELÄSNING 12 · CAN-DRIVRUTINEN

Ned till botten: riktiga register, och limmet som binder ihop allt.

I DET HÄR KAPITLET

- `volatile`: vad det är till för, och var det får stå.
- Mikrokontrollerns SPI-periferi, och de tre inställningar som är fällor.
- MPIO: hur kärnan går på 5 V medan SPI-ledningarna går på 3,3 V.
- Factoryn, och testet på om arkitekturen håller.

12.1 `volatile`

Hittills har koden bara rört vanligt minne. Den här filen rör **hårdvara**, och då slutar ett av kompilatorns grundläggande antaganden att gälla.

Kompilatorn antar att ingen annan än programmet ändrar en variabel. Med den utgångspunkten är följande loop oändlig, och optimeraren får ta bort den:

```
while ((SPI0.INTFLAGS & SPI_IF_bm) == 0U)
{
}
```

INTFLAGS läses en gång, resultatet läggs i ett register, och eftersom ingenting i loopen ändrar det är villkoret konstant. Med -O2 blir resultatet antingen en loop som aldrig avslutas eller en loop som optimeras bort helt.

Men INTFLAGS *ändras* — av periferin, utan att programmet gör något. `volatile` är hur man säger det: *läs om den här variabeln varje gång; slå inte ihop läsningar, flytta dem inte, ta inte bort dem.*

I `<avr/io.h>` är periferiregistren redan deklarerade `volatile`. Du behöver inte skriva kvalificeraren — men du behöver veta att den är där, och varför.

12.1.1 Vad `volatile` inte är

Det är **inte** en synkroniseringsmekanism. Det garanterar inte atomicitet och ingen ordning mot annat än andra `volatile`-åtkomster. I tråddad kod på en värddator är `volatile` nästan alltid fel verktyg; `std::atomic` är rätt.

Här handlar det om något annat: ett register som en periferi ändrar under näsan på programmet. Det är precis det fall `volatile` finns för.

12.1.2 Var det får stå

Bara i transportens implementationsfil.

Det är hela poängen med sömmen i kapitel 9. Allt ovanför — `Spi`, `Stub`, `Interface`, `EchoNode` — är vanlig, portabel C++ som kompilerar och körs på din laptop. En granskare som ser `volatile` i någon annan fil ska fråga varför.

12.2 SPI-periferin

Fyra register räcker för en polld master: ett för enable och klockinställningar, ett för läge, ett för statusflaggor, och ett för data.

```
void AvrSpiTransport::init() noexcept
{
    // Route SPI0 to PORTC (ALT1): PC0 = MOSI, PC1 = MISO, PC2 = SCK, PC3 = SS.
    PORTMUX.SPIROUTEA = PORTMUX_SPI0_ALT1_gc;

    // MOSI, SCK and SS are outputs; MISO is an input. SS idles high.
    PORTC.DIRSET = PIN0_bm | PIN2_bm | PIN3_bm;
    PORTC.DIRCLR = PIN1_bm;
    PORTC.OUTSET = PIN3_bm;

    // Master, MSB first, 4 MHz / 4 = 1 MHz, enabled.
    SPI0.CTRLA = SPI_MASTER_bm | SPI_PRESC_DIV4_gc | SPI_ENABLE_bm;

    // SPI mode 0, and software-controlled slave select.
    SPI0.CTRLB = SPI_MODE_0_gc | SPI_SSD_bm;
}
```

Fem rader, och tre av dem är fällor:

Pinnmuxen. Utan den raden ligger periferin på sin standardmux, och de fyra ledningar du kopplat är helt tysta. Symptomet är det mest förvirrande som finns: programmet kör, inga fel rapporteras, och logikanalysatorn visar ingenting alls. **Sätt den först.**

Slave select disable. Utan den övervakar periferin SS-pinnen, och en låg nivå där får mastern att tro att en annan master tagit över bussen: den lämnar masterläget. Men här är det *din egen kod* som drar SS låg vid varje `select()`. Utan biten skjuter drivrutinen sig själv i foten vid första transaktionen.

Prescalern. Protokollet garanterar ingenting över 1 MHz. Kör inte snabbare för att det går.

12.2.1 Transporten

```
void AvrSpiTransport::select() noexcept { PORTC.OUTCLR = PIN3_bm; }
void AvrSpiTransport::deselect() noexcept { PORTC.OUTSET = PIN3_bm; }

std::uint8_t AvrSpiTransport::transfer(const std::uint8_t byte) noexcept
{
    SPI0.DATA = byte;

    while ((SPI0.INTFLAGS & SPI_IF_bm) == 0U)
    {
    }

    return SPI0.DATA;
}
```

Läsningen av SPI0.DATA nollställer flaggan. Hoppa över den och nästa transfer() ser en flagga som redan står kvar, returnerar omedelbart, och läser en byte som inte hunnit komma in. Symptomet är data som ligger **en byte efter** genom hela transaktionen, vilket är svårt att se och lätt att missta för ett bitordningsfel.

Ingen fördröjning behövs kring SS. Protokollet kräver 60 ns setup och hold; en enda portskrivning på en 4 MHz-MCU är redan flera gånger långsammare än så.

12.3 MVIO: två spänningar i samma chip

Det här är den elegantaste detaljen i hela konstruktionen.

Problemet: MCU-kortet körs på 5 V. FPGA-kortets GPIO är 3,3 V och tål inte 5 V. Den vanliga lösningen är en nivåomvandlare på varje ledning — en komponent till, fyra kopplingar till, och fyra saker till som kan sitta fel.

Lösningen: AVR-DB-familjen har **MVIO** (*Multi-Voltage I/O*). PORTC matas inte från VDD som resten av chippet, utan från en egen pinne: VDDI02.

Koppla VDDI02 till FPGA-kortets 3,3 V-rail, och hela PORTC blir en 3,3-voltsdomän — medan kärnan och övriga portar kör oförändrat på 5 V. **Ingen nivåomvandlare.**

Det är därför pinnkonfigurationen är en del av kontraktet och inte varje grupps eget val: byter du till standardmuxen hamnar ledningarna i VDD-domänen, och då behövs nivåomvandlaren igen — eller så går FPGA-ingången sönder.

Observera: VDDIO2 måste vara matad **innan** PORTC gör någonting alls. Är den inte det är portens beteende odefinierat. Statusregistret för MVIO säger om den är det, och en kontroll av det vid uppstart är den enskilt mest värdefulla raden kod inför bring-upen: en bortglömd VDDIO2-koppling ser ut precis som en trasig SPI-implementation, och den kontrollen skiljer de två fallen åt på en sekund.

12.4 Friställd C++

Firmware byggs för ett **friställt** mål. En del av C++ följer inte med:

Finns inte	Vad du gör i stället
Heap (<code>new</code> , <code>std::vector</code>)	Statiskt allokerade objekt; allt har känd storlek.
Undantag	Returkoder — vilket hela interfacet redan bygger på.
RTTI (<code>dynamic_cast</code>)	Virtuella funktioner räcker; du castar aldrig nedåt.
<code><iostream></code>	UART-utskrift, eller en lysdiod.

Lägg märke till att **ingenting** av det påverkar designen. Interfacet returnerar `bool` i stället för att kasta. `Frame` är en aggregate med statisk storlek. Ingen klass använder heapen. Det var inte tur — projektets kodkonventioner är skrivna för ett friställt mål från början, och det här är kapitlet där det lönar sig.

12.5 Factoryn

Alla delar finns nu. Det som fattas är limmet:

<code>EchoNode</code>	behöver ett	<code>Interface&</code>
<code>Spi</code>	ÄR ett	<code>Interface</code> , och behöver en <code>ByteTransport&</code>
<code>AvrSpiTransport</code>	ÄR en	<code>ByteTransport</code>

Någon måste skapa objekten, i rätt ordning, och hålla dem vid liv så länge programmet kör. Den någon ska inte vara `main`, och definitivt inte `EchoNode`.

```
class Factory
{
public:
    Factory() noexcept;

    Factory(const Factory&)           = delete;
    Factory(Factory&&)                = delete;
    Factory& operator=(const Factory&) = delete;
    Factory& operator=(Factory&&)     = delete;
```

```

/**
 * @brief Get the CAN driver.
 *
 * @return Reference to the driver, valid for the lifetime of the factory.
 */
[[nodiscard]] driver::can::Interface& canDriver() noexcept;

private:
    AvrSpiTransport myTransport; /**< Byte transport. Declared before the driver ↵
    ↵ that uses it. */
    driver::can::Spi myDriver; /**< The CAN driver, bound to myTransport. */
};

```

Deklarationsordningen är ett krav, inte stil. Medlemmar konstrueras i den ordning de deklarerar, och myDriver tar en referens till myTransport i sin konstruktor. Byt plats på de två raderna och du binder en referens till ett objekt som inte konstruerats än. Kompilatorn säger ingenting.

Returtypen är Interface&, inte Spi&. Det är hela poängen. Anroparen får något den kan använda, inte något den kan känna igen.

Ingen heap. Objekten är medlemmar, factoryn ligger på stacken i main, och allt har statisk storlek. På ett friställt mål finns ingen std::unique_ptr att ta till — och den behövs inte heller. Smarta pekare i ett factory-exempel löser ett ägandeproblem som inte finns när antalet objekt är känt vid kompileringstillfället.

12.6 main

```

int main()
{
    Factory factory{};
    app::EchoNode node{factory.canDriver(), ResponseId};

    while (true)
    {
        node.run();
    }
}

```

Fem rader. main gör tre saker och inget mer: skapar factoryn, skapar applikationskomponenten med det factoryn lämnar ut, och kör.

Orden Spi, AvrSpiTransport, SPI0 och PORTC förekommer inte. **Om du kan läsa din main utan att veta vilken hårdvara programmet kör på har du fått arkitekturen rätt.**

12.6.1 Testet på designen

Byt en rad i factoryn:

```
private:
    driver::can::Stub myDriver;    // was: AvrSpiTransport + Spi
```

Kompilera för **värddatorn**. Det ska gå, och programmet ska köra — EchoNode ekar frames mot en array i minnet i stället för mot en CAN-buss.

Går det inte, pekar kompilatorfelet ut exakt var någon rad ovanför interfacet (`driver/can/interface.hpp`) har lärt sig något om hårdvaran som den inte borde veta.

Det är ett test du kan köra på tio sekunder, och det säger mer om arkitekturens kvalitet än någon kodgranskning.

12.7 En applikation som går att felsöka med

EchoNode är bra att bevisa arkitekturen med och dålig att felsöka med: den gör ingenting synligt. Inför kapitel 13 är det värt att bygga en till.

Lysdiod	Tänds när
Grön	En frame har tagits emot och kvitterats.
Gul	En frame har skickats (<code>send()</code> returnerade <code>true</code>).
Röd	<code>hasError()</code> returnerade <code>true</code> .

Plus en knapp som skickar en frame med ett känt ID och en känd nyttolast.

Det är ett kvarts arbete och det betalar sig direkt vid bänken: en röd lysdiod som tänds när du kopplar in transceivern säger något helt annat än en nod som inte gör något alls.

Håll den utanför EchoNode. Lysdioderna hör till applikationen, inte till drivrutinen, och drivrutinen ska fortsätta vara byggbar för värddatorn.

Sammanfattning

- `volatile` säger åt kompilatorn att inte optimera bort läsningar av något hårdvaran ändrar. Det är **inte** en synkroniseringsmekanism.
- `volatile` får stå i **en enda fil**: transportens implementation. Det är vad sömmen i kapitel 9 köpte.
- Tre inställningar är fällor: pinnmuxen (utan den är ledningarna tysta), slave select disable (utan den lämnar mastern masterläget vid första `select()`), och prescalern.
- `transfer()` måste **läsa** dataregistret efter varje överföring. Annars ligger all data en byte fel.

- MVIO låter PORTC köra på 3,3 V medan kärnan kör på 5 V — ingen nivåomvandlare. VDDIO2 måste vara matad innan porten gör något.
- Factoryns medlemmar deklareras i konstruktionsordning: transporten före drivrutinen som binder en referens till den.
- Testet på arkitekturen: byt Spi mot Stub i factoryn och kompilera för värddatorn. Går det inte har någon rad ovanför interfacet lärt sig för mycket.

Övningar

Lösningarna till uppgifterna som inte ber om kod står i [appendix A](#). Koderna publiceras i kursrepot efter föreläsningen.

Övning 12.1 Se optimeraren arbeta

KOD

På värddatorn, innan du skriver någon hårdvarukod:

```
int flag{};

void wait()
{
    while (flag == 0)
    {
    }
}
```

- Kompilera med `-O2 -S` och titta på assemblern. Vad blev loopen?
- Lägg till `volatile` på `flag` och gör om. Vad ändrades?
- Förklara i två meningar varför skillnaden spelar roll för statusflaggan i `transfer()`.

Övning 12.2 Förutsäg, ändra, observera

KOD

Gör en ändring i taget och notera symptomet. Återställ efter varje.

- Ta bort pinnmux-raden.
- Ta bort slave-select-disable ur lägesregistret.
- Sätt prescalern så att klockan blir dubbelt så snabb.
- Ta bort läsningen av dataregistret i `transfer()`.
- Glöm att sätta klockpinnen som utgång.

Rad d) är den lömskaste: det *ser* ut att fungera. Vad blir egentligen fel, och när märks det?

Övning 12.3 Arkitekturtestet

KOD

Byt transport och drivrutin mot en Stub i factoryn och kompilera för värddatorn.

- a) Gick det?
- b) Om inte: vilken fil klagade kompilatorn på, och vilken lagergräns hade brutits?
- c) Kör programmet. Vad gör EchoNode mot en Stub?

Övning 12.4 Friställt

KOD

Bygg firmware med `-fno-exceptions -fno-rtti`.

- a) Kompilerar allt?
- b) Sök igenom hela projektet efter `new`, `std::vector`, `std::string` och `dynamic_cast`. Hittar du något?
- c) Varför behövde designen inte ändras för det friställda målet?

Övning 12.5 Konstruktionsordningen

RESONEMANG

Byt plats på factoryns två medlemsdeklarationer.

- a) Kompilerar det? Varnar kompilatorn?
- b) Vad händer vid körning, och varför?
- c) Vilken allmän regel om medlemsordning följer av det?

KAPITEL 13

Bring-up

FÖRELÄSNING 13 · HÅRDVARAN

Två halvor, fyra ledningar, och en stege med sex pinnar.

I DET HÄR KAPITLET

- Kopplingen: SPI mellan korten, och CAN-sidan ut på en riktig buss.
- Bring-up-stegen, var och en med ett eget kvitto.
- Felsökning: från symptom till hypotes till mätning.
- Att felsöka något två grupper skrivit varsin halva av.

13.1 Kopplingen

13.1.1 SPI mellan korten

Fyra ledningar plus matning och jord, enligt kontraktets pinnkonfiguration. Två saker är värda att säga ut:

VDDI02 först. PORTC är MVIO-domänen och matas därifrån. Utan den kopplingen gör porten ingenting alls, och symptomet ser ut precis som en trasig SPI-implementation.

Gemensam jord är inte valfritt. Två kort utan gemensam jord kommunicerar inte, och symptomet är obegripligt.

13.1.2 CAN-sidan

FPGA-nodens CAN-sida är en **enledarbuss med öppen dränering** på tre GPIO-pinnar: tx_bus (vad noden driver), bus_en (om den driver alls) och rx_bus (bussens nivå in).

För att en bussanalysator ska kunna se trafiken måste den enledarbussen bli en riktig **differentiell** CAN-buss. Det är transceiverns jobb, och dess sändaringång ska vara **låg för en dominant bit**:

bus_en	tx_bus	Noden	Transceivers ingång
0	—	driver inte	1 (recessiv)
1	0	dominant	0
1	1	recessiv	1

Hur de två signalerna slås ihop till den enda ledningen är hårdvarusidans del av kopplingen; stäm av det innan du kopplar. En nod vars bus_en ignoreras driver bussen hela tiden och blockerar alla andra.

Terminering: 120 Ω i vardera änden, inte i mitten och inte fler än två. En felterminerad buss fungerar ofta ändå på en kort kabel, och slutar fungera precis när du börjat lita på den.

13.2 Stegen

Klättra dem i ordning. Varje steg har ett eget kvitto, och poängen är att **aldrig ha mer än en okänd sak åt gången**. Hoppa över ett steg och du felsöker fyra lager samtidigt.

Steg	Gör	Kvitto
0	Matning och jord	MVIO-statusen rapporterar att VDDI02 är inom intervallet.
1	MOSI till MISO, utan FPGA	Du får tillbaka exakt den byte du skickade. Prova 0x00, 0xFF, 0xAA, 0x55.
2	Koppla in FPGA; skriv och läs TX_ID	0xFFFFFFFF läses tillbaka som 0x000007FF.
3	Läs STATUS upprepat	0x00000001 — TX klar satt, övriga låga.
4	Koppla in transceivern; kör en send()	Aktivitet på bussen; STATUS bit 0 går låg och kommer tillbaka.
5	Låt en andra nod sända	receive() returnerar true med rätt innehåll.

Två av stegen förtjänar en kommentar.

Steg 1 är AvrSpiTransports enda test. Det är den enda klassen i projektet som inget automattest täcker, eftersom den pratar med ett fysiskt register. Fungerar loopbacken inte ligger felet i din kod eller i konfigurationen — FPGA:n är inte inkopplad och kan inte vara orsaken.

Steg 2 är det avgörande. Maskeringen till elva bitar bevisar att du pratar med *registerbanken* och inte med ett eko av din egen ledning. En loopback ger tillbaka det du skickade; en registerbank ger tillbaka det den lagrade.

13.3 Felsökning

Symptom	Trolig orsak	Kontrollera
Inget alls, inga klockpulser	Pinnmuxen inte satt	Steg 1
Porten gör ingenting	VDDIO2 inte matad	MVIO-statusen
Alla läsningar ger 0x00000000	MISO inte kopplad, eller fel pinne	Byt MOSI/MISO och se om det ändras
Alla läsningar ger 0xFFFFFFFF	MISO flytande	Kopplingen
Läsningarna ligger en byte fel	Dataregistret läses inte i <code>transfer()</code>	kapitel 12
Rätt för små tal, fel för stora	Teckenutvidgning i <code>readReg()</code>	kapitel 9
Andra transaktionen ignoreras	Två transaktioner under en låg SS	kapitel 8
Framen kommer fram bakvänd	TX_DATA_L0/HI förväxlade	kapitel 10
Samma frame om och om igen	RX_ACK skrivs inte	kapitel 10
<code>hasError()</code> alltid true	<code>clearError()</code> skriver 0x1	kapitel 10
Bussen konstant dominant	En nods <code>bus_en</code> ignorerad	Kopplingstabellen
Fungerar på kort kabel, inte lång	Terminering	120 Ω i vardera änden

13.3.1 Arbetsgången

1. Vilket steg var det sista som gav rätt kvitto?
2. Vad är den **enda** saken som tillkommit sedan dess?
3. Formulera en hypotes om vad som är fel.
4. Bestäm vilken mätning som skulle avgöra saken.
5. Gör mätningen.

Observera: Att byta sladdar tills det fungerar är inte felsökning, och det tar längre tid. Skillnaden mellan de två arbetssätten är ungefär en eftermiddag per fel, och den är den mest överförbara färdigheten i hela kursen.

13.3.2 Att felsöka någon annans halva

En bugg under bring-upen kan ligga i din kod, i den andra gruppens VHDL, eller i en kopplingstråd. Tre regler gör det hanterbart:

- **Misstänk din egen halva först.** Inte av ödmjukhet, utan för att du kan ändra den och mäta på den.
- **Bevisa innan du rapporterar.** "Det fungerar inte" är inte en felrapport. "Steg 2 ger 0x000007FF för TX_ID men 0x00000000 för TX_DLC" är det.

- **Ändra en sak i taget, och skriv ned vad.** Två personer som felsöker samma koppling samtidigt gör varandras mätningar meningslösa.

Sammanfattning

- VDDI02 och gemensam jord först. Utan dem ser en korrekt SPI-implementation ut som en trasig.
- FPGA-nodens enledarbuss blir en riktig differentiell CAN-buss genom en transceiver, vars sändaringång ska vara **låg för dominant**. Terminera med $120\ \Omega$ i vardera änden.
- Klättra stegen i ordning, och ha **aldrig mer än en okänd sak åt gången**.
- Steg 1 (loopback) är transportens enda test — den klassen täcks inte av något automattest.
- Steg 2 (maskeringen till elva bitar) bevisar att du pratar med registerbanken och inte med ett eko av din egen ledning.
- Felsök med hypotes och mätning, inte med sladdbyten. Misstänk din egen halva först, bevisa innan du rapporterar, ändra en sak i taget.

Övningar

Övning 13.1 Stegen

RESONEMANG

- Vad bevisar steg 2 som steg 1 inte gjorde?
- Varför räcker det inte att läsa tillbaka samma värde du skrev?
- Vilket steg är det enda testet av `AvrSpiTransport`, och varför finns inget automattest för den klassen?

Övning 13.2 Från symptom till hypotes

RESONEMANG

Du läser STATUS och får `0x00000000` varje gång.

- Räkna upp minst fyra möjliga orsaker.
- Ordna dem efter hur snabbt de går att utesluta.
- Vilken mätning skiljer de två första åt?

Övning 13.3 Vems fel?

RESONEMANG

Din nod skickar en frame, men ingenting syns på bussen.

- Ligger felet i din drivrutin, i FPGA:n eller i kopplingen?
- Hur avgör du det *utan* att gissa?
- Formulera den felrapport du skulle skicka till hårdvarugruppen, med det du vet så här långt.

Övning 13.4 Termineringen

RÄKNING

Du mäter resistansen över den strömlösa bussen.

- a) Vad ska du få med korrekt terminering?
- b) Du får $40\ \Omega$. Vad är fel?
- c) Du får $120\ \Omega$. Vad är fel, och när skulle det visa sig?

Bussanalys

FÖRELÄSNING 14 · HÅRDVARAN

Trafiken sedd utifrån, med ett verktyg som inte tror på någon av halvorna.

I DET HÄR KAPITLET

- Vad en bussanalysator är, och varför den är en tredje, opartisk part.
- Att läsa en frame i en trace: identifierare, DLC, data, tidsstämplar.
- Att skicka *till* din nod, och se kvittensen fungera.
- Att leta efter konstruktionens dokumenterade förenklingar — utifrån.

14.1 Varför en analysator

Fram till nu har varje observation gjorts av en av de två halvorna. Din drivrutin säger att den skickade en frame; hårdvarugruppens testbänk säger att kontrollern sände en. Båda kan ha fel på samma sätt, eftersom båda är skrivna mot samma dokument av två personer som kan ha läst det lika fel.

En **bussanalysator** är en tredje part som inte tror på någon av er. Den är en riktig CAN-kontroller på bussen, och den avkodar det som faktiskt ligger på ledningarna.

Det är därför labben ligger sist och inte först: det är först nu det finns trafik att titta på, och det är först nu svaret betyder något.

Observera: Analysatorn kvitterar dessutom dina frames. Det är värt att veta: en ensam nod på bussen får aldrig någon kvittens, och *en fullständig* CAN-kontroller hade då skickat om i all evighet. Konstruktionen i den här kursen läser aldrig tillbaka ACK-luckan, så den märker ingenting — men skillnaden blir synlig i tracen så fort analysatorn kopplas in.

14.2 Att läsa en trace

Ställ in analysatorn på samma bithastighet som konstruktionen, och kontrollera att termineringen fortfarande är $120\ \Omega$ i vardera änden — inte tre motstånd för att det nu är tre noder.

Skicka en frame med känt ID och känd nyttolast från din nod, och leta upp den:

Kolumn	Vad du kontrollerar
Identifierare	Stämmer den med vad du skrev till TX_ID?
DLC	Stämmer den med TX_DLC?
Data	Stämmer bytesen, <i>och i rätt ordning</i> ? Det är här en förväxling av TX_DATA_L0/HI antligen blir synlig.
Tidsstämpel	Hur ofta kommer de? Stämmer det med hur ofta din kod anropar send()?

Sedan åt andra hållet: skicka en frame *från* analysatorn till din nod. Returnerar receive() true? Stämmer innehållet? Och — det som är lätt att glömma — gick STATUS bit 1 ned när du skrev RX_ACK?

14.3 Att leta efter förenklingarna

Konstruktionen är förenklad mot riktig CAN på flera punkter. Några av dem går att se utifrån, och det är den intressantaste delen av labben.

Förenkling	Vad du kan leta efter
Ingen ACK-återläsning	Koppla bort analysatorn och sänd med bara din nod på bussen. Vad säger felflaggan? Vad <i>skulle</i> en riktig nod ha gjort?
Inga felmarar	Vad händer om analysatorn skickar en?
Ingen automatisk omsändning	En frame som gick fel skickas inte om av hårdvaran. Syns det?
Ingen mottagningskö	Låt analysatorn skicka två frames tätt. Kommer båda fram till din applikation?

Att hitta en avvikelse är inte att hitta ett fel. Uppgiften är att kunna säga *vilken dokumenterad förenkling* som förklarar vad du ser — och vilka avvikelser som *inte* har någon sådan förklaring. Det är de senare som är intressanta, för båda klasserna.

14.4 Bussbelastning

Analysatorn visar bussbelastningen. Jämför med tidsbudgeten i kapitel 8:

- Vad är belastningen med din nod som enda sändare?
- En send() tar sex SPI-transaktioner, alltså omkring $270\ \mu\text{s}$. En maximal CAN-frame är omkring $130\ \mu\text{s}$. **Vad begränsar dig egentligen — bussen eller länken?**

Svaret på den sista frågan är hela poängen med att mäta i stället för att anta, och det är det enda stället i kursen där du kan se det med egna ögon.

14.5 En frame, hela vägen

Avsluta med att spåra en enda frame genom hela stacken, och namnge varje lager och vem som byggde det:

```
app::EchoNode          -> du
driver::can::Interface -> du
driver::can::Spi        -> du
driver::can::ByteTransport -> du
AvrSpiTransport         -> du
===== fyra ledningar =====
spi_slave               -> utdelad
spi_reg_bridge          -> hårdvarugruppen
register_bank           -> hårdvarugruppen
can_controller          -> hårdvarugruppen
===== transceiver =====
CAN-bussen             -> fysik
```

Tio lager, två klasser, ett kontrakt — och en frame som kom fram.

Sammanfattning

- En bussanalysator är en **tredje part som inte tror på någon av halvorna**. Fram till nu har varje observation gjorts av en av dem, och båda kan ha läst kontraktet lika fel.
- Analysatorn kvitterar dina frames. En ensam nod får aldrig någon kvittens — och den här konstruktionen märker inte ens det, eftersom den aldrig läser tillbaka ACK-luckan.
- I tracen blir en förväxling av TX_DATA_L0/HI äntligen synlig, eftersom den ger en frame som kommer fram med rätt längd och fel ordning.
- Att hitta en avvikelse är inte att hitta ett fel. Uppgiften är att koppla den till en dokumenterad förenkling — eller konstatera att den saknar förklaring.
- En `send()` tar längre tid än framen den skickar. SPI-länken är flaskhalsen, inte CAN-bussen, och här kan du mäta det.

Övningar

Övning 14.1 Läs din egen trafik

KOD

Skicka en frame med känt ID och känd nyttolast.

- a) Hitta den i tracen. Stämmer identifierare, DLC och data?
- b) Stämmer databytesens *ordning*?
- c) Vilka fält ser du i tracen som registerkartan *inte* exponerar?

Övning 14.2 Åt andra hållet

KOD

Skicka en frame från analysatorn till din nod.

- a) Returnerar `receive()` `true`?
- b) Vad hände med STATUS bit 1, före och efter din `RX_ACK`?
- c) Skicka två frames tätt efter varandra. Kommer båda fram till applikationen? Varför?

Övning 14.3 Kvittensen

RESONEMANG

- a) Kvitterar analysatorn dina frames? Hur ser du det?
- b) Koppla bort den och sänd med bara din nod på bussen. Vad säger felflaggan?
- c) Vad skulle en fullständig CAN-kontroller ha gjort i det läget, och varför gör inte den här det?

Övning 14.4 Flaskhalsen

RÄKNING

- a) Vad är bussbelastningen med din nod som enda sändare?
- b) Hur lång tid tar en `send()` i din drivrutin, och hur lång tid tar framen på bussen?
- c) Vad begränsar dig — bussen eller SPI-länken? Stämmer mätningen med räkningen i kapitel 8?

KAPITEL 15

Repetition

FÖRELÄSNING 15 · HÅRDVARAN

Vad som examineras, och hur de två halvorna hänger ihop.

I DET HÄR KAPITLET

- De två halvornas gemensamma frågor, ställda bredvid varandra.
- Vad duggan täcker, del för del.
- De femton misstagen som kostat mest tid i det här projektet.

15.1 Samma frågor, två svar

Hela boken kretsar kring fem frågor. Första delen besvarade dem i mjukvara, andra delen lät hårdvaran göra det:

Fråga	Eget protokoll	CAN
Var börjar meddelandet?	SOF, två bytes	SOF-bit; sju recessiva bitar som slutmärke
Hur långt är det? Vem är det till?	LEN, en byte DST och SRC	DLC, fyra bitar, 0–8 databytes Ingen adress. Identifieraren beskriver innehållet
Kom det fram helt?	Summachecksumma	CRC-15, plus bitstoppning och formatregler
Kom det fram alls?	ACK, timeout, omsändning, dubblettetektering	En ACK-bit inuti framen

Att kunna fylla i den tabellen ur minnet, och motivera varje rad, är ungefär vad duggan kräver av den första halvan.

15.2 Vad duggan täcker

15.2.1 Del A — det egna protokollet

Framedesign och checksumberäkning för hand. Parsning som tillståndsmaskin. Adressering, routing och sekvensnummer. Byte-strömmar som ska tolkas och analyseras. Felmodellen: tappade bytes, korrupt data, fördröjning. Kedjan ACK → timeout → omsändning → dubletter.

15.2.2 Del B — CAN och drivrutinen

CAN-ID, prioritet, DLC och arbitrering. Vilka mekanismer hårdvaran utför, och var. Registerkartan: vad varje register betyder, och hur varje statusbit nollställs. Databyteordningen över TX_DATA_L0/HI. SPI-transaktionen: kommandobyten, de fyra databyten, SS-reglerna, och att koda och avkoda en transaktion för hand. Lagerarkitekturen. De tre begränsningarna.

Registerkartan delas ut med duggan — indexen behöver inte memoreras. Att *läsa* kartan och veta vad den betyder behövs desto mer.

15.3 De dyraste misstagen

Var och en av dem har kostat någon en kväll. De står i den ordning de brukar inträffa.

#	Misstag	Symptom
1	Parseern kontrollerar inte längdfältet mot buffertens storlek	Buffertöverskrivning; allt möjligt annat går sönder
2	Parseern går tillbaka till <code>WaitForSof1</code> på ett andra <code>0xA5</code>	Enstaka frames missas, oförklarligt
3	Flerbytesfält serialiseras genom att kopiera minnet	Fungerar tills den andra sidan byggs med annan endianness
4	Dubblett kvitteras inte igen	Sändaren skickar om i all evighet
5	<code>TX_DATA_LO</code> och <code>TX_DATA_HI</code> förväxlade	Framen kommer fram — bakvänd
6	Cast efter skiftningen i <code>readReg()</code>	Fungerar för små värden, går sönder vid bit 7
7	<code>TX_SEND</code> skrivs före dataregistren	Noden ligger exakt en frame efter
8	<code>clearError()</code> skriver <code>0x1</code>	Felbiten går aldrig ned; all felhantering slutar fungera
9	<code>RX_ACK</code> glöms	Samma frame läses i en oändlig loop
10	<code>receive()</code> maskerar inte mot <code>d1c</code>	Upp till sex bytes rapporteras som aldrig sändes
11	<code>while (!send(frame)) {}</code> på en ogiltig frame	Hänger för alltid
12	<code>Pinnmuxen</code> inte satt	Programmet kör, ledningarna är tysta, inga fel rapporteras
13	Dataregistret läses inte i <code>transfer()</code>	All data ligger en byte fel
14	<code>VDDIO2</code> inte kopplad	Ser ut precis som en trasig SPI-implementation
15	Factoryns medlemmar i fel deklarationsordning	Referens till ett objekt som inte konstruerats

Observera: Elva av de femton hade fångats av ett test på laptopen. Tre av dem — 12, 13 och 14 — hade det inte, och det är precis de tre som bor i den enda klass inget automattest täcker. Det är ingen slump, och det är hela argumentet för stegen i kapitel 13.

15.4 Efter kursen

Två saker fortsätter direkt härifrån.

Testning. Genom hela projektet har du kört och läst tester som någon annan skrivit. Nästa steg är att bestämma vad de ska kontrollera — vilket är ett svårare problem än det låter, och det som avgör om en testsvit är värd något.

Ditt eget repo. Drivrutinen du byggt är en komplett, testad, lagerdelad drivrutinsstack mot riktig hårdvara, utvecklad mot ett kontrakt och verifierad utan tillgång till den andra halvan. Det är ett rimligt första objekt i en portfölj, och det är en mer realistisk beskrivning av inbyggd utveckling än de flesta kursprojekt.

Lösningar

Här finns lösningarna till **varje övning som inte ber om kod** — alla resonemangs-, räkne- och avkodningsuppgifter, och de delar av kodövningarna som är resonemang. Lösningarna är fullständiga: där en uppgift ber om en uträkning står stegen, och där den ber om ett resonemang står argumentet och inte bara slutsatsen.

De övningar som ber om kod besvaras inte här. För dem är den utdelade testsviten svaret: den säger, testfall för testfall, exakt vad koden ska göra. Koden publiceras dessutom i kursrepot efter respektive föreläsning.

Läs lösningen efter att du försökt själv. En lösning man läst är inte samma sak som en lösning man kommit fram till, och det är den skillnaden övningarna finns för.

Kapitel 1 — Frames

Övning 1.1 Bygg en frame

sidan 5

a) $LEN = 0$, $TYPE = 0x02$, $DST = 0x25$, $SRC = 0x17$, $SEQ = 0x7F05$. Checksumman blir $A5 + F7 + 00 + 02 + 25 + 17 + 7F + 05 = 606 = 0x025E$:

```
A5 F7 00 02 25 17 7F 05 02 5E
```

b) Adresserna byter plats, $LEN = 1$, $TYPE = 0x03$, en databyte $0x16$. Checksumman blir $630 = 0x0276$:

```
A5 F7 01 03 17 25 7F 05 16 02 76
```

c) Tio bytes: de åtta headerbytesen plus två checksummabytes, med $LEN = 0$.

Övning 1.2 Fälten och deras syfte

sidan 5

Utan **SOF** går det inte att veta var en frame börjar i strömmen. Utan **LEN** vet mottagaren inte när payloaden tar slut, eftersom den läser en byte i taget. Utan **TYPE** går det inte att avgöra vad ramen betyder. Utan **DST** vet ingen vem den är till, utan **SRC** kan mottagaren inte svara, och utan **SEQ** går svar inte att para ihop med fråga och dubletter inte att känna igen. Utan **CHK** upptäcks inga överföringsfel.

De två som går att ta bort i ett system med exakt två noder är **DST** och **SRC**: med bara två parter är avsändare och mottagare givna av vem som tog emot. Det är också precis vad som gör dem outhärliga så snart en tredje nod tillkommer.

Övning 1.3 Endianness

sidan 5

- a) 05 7F — x86 lagrar little-endian, alltså lägsta byten först, medan protokollet kräver 7F 05.
- b) Båda sidor har samma fel. Serialiseringen skriver 05 7F och deserialiseringen läser tillbaka det som 0x7F05, så felen tar ut varandra så länge båda ändarna är byggda ur samma kod på samma arkitektur.
- c) Så fort den andra sidan är något annat: en annan arkitektur, en annan implementation, eller en bussanalysator som läser strömmen enligt specifikationen. Då blir 0x7F05 till 0x057F.

Övning 1.4 Checksummans gränser

sidan 6

- a) Nej. En summa är oberoende av ordningen, så två ombytta bytes ger samma summa.
- b) Nej. Summan är oförändrad, eftersom det som lades till på ett ställe drogs bort på ett annat.
- c) Till exempel: byt DST och SRC med varandra i ramen från övning 1.1, eller ändra 0x25 till 0x26 och 0x17 till 0x16. Båda ger identisk checksumma. En CRC skulle upptäcka bådadera, vilket är skälet till att CAN använder en.

Kapitel 2 — Byte-parsning

Övning 2.1 Hitta framen

sidan 11

- a) Vid index 4 (nollindexerat): 32 74 00 FF är skräp, och A5 F7 på index 4–5 är SOF.
- b) A5 F7 03 03 25 17 7F 05 10 20 30 02 C2 — tretton bytes, alltså tio header- och footer-bytes plus tre databytes.
- c) Kasta dem. De tas emot i `WaitForSof1` och förkastas en och en, eftersom ingen av dem är 0xA5.
- d) `TYPE = 0x03`, alltså `StatusResponse`, från `SRC = 0x17` till `DST = 0x25`.

Övning 2.2 SOF-fällan

sidan 11

- a) Den naiva implementationen går tillbaka till `WaitForSof1` och kastar byten. Då kastar den den byte som just kan ha varit den nya framens första.
- b) Stanna kvar i `WaitForSof2`. Byten var 0xA5, alltså en möjlig SOF-start i sin egen rätt.
- c) Till exempel A5 A5 F7 ...: här är den andra 0xA5 framens verkliga början. Den naiva parsern kastar den, hamnar i `WaitForSof1`, ser 0xF7 — som inte är 0xA5 — och missar hela framen.

Övning 2.3 Tappad byte

sidan 11

- a) `LEN` är fortfarande 3, så parsern samlar 10 30 02 som payload och läser därefter C2 som den *första* checksummabyten.
- b) Inte på den här strömmen. Parsern står i `WaitForChk2` och väntar på en byte som aldrig kommer; först när nästa byte anländer blir framningen klar.
- c) `false`. Framningen kan lyckas, men `Frame::deserialize()` räknar om checksumman och den stämmer inte.
- d) `WaitForChk2` tills nästa byte kommer. För att hitta nästa frame måste anroparen anropa `reset()`, varefter parsern söker efter 0xA5 igen.

Övning 2.4 Längdfältet

sidan 11

- a) Sätt LEN till ett värde större än MaxPayloadLen, till exempel A5 F7 FF 00 Parseern samlar då 255 databytes i en buffert som rymmer färre.
- b) För sent. Skadan sker under insamlingen, alltså i processByte(), långt innan någon hinner anropa extractFrame().
- c) Att varje längd som kommer utifrån ska kontrolleras mot buffertens storlek *innan* den används. Samma regel gäller TX_DLC i kapitel 10: hårdvaran validerar inte, så drivrutinen måste avvisa $dlc > 8$ innan den skriver något.

Kapitel 3 — Bussar, routing och tillförlitlighet**Övning 3.1** Routing

sidan 17

- a) I noden. En fysisk ledning levererar spänningsnivåer; den kan varken läsa adresser eller veta att det finns frames. Varje nod måste därför själv jämföra DST med sin egen adress.
- b) Ignorera den. Framen är inte felaktig — den är adresserad till någon annan.
- c) Därför att Nack betyder *något gick fel*. På en buss med n noder skulle varje frame utlösa $n - 2$ falska negativa kvittenser, och sändaren skulle inte kunna skilja dem från ett verkligt fel.

Övning 3.2 Felmodellen

sidan 17

- a) **Tappad byte:** nej — alla följande fält förskjuts, och LEN läses ur fel byte. **Ändrad bit:** ja — strukturen är intakt, bara innehållet är fel. **Fördröjning:** ja — ingenting i framen påverkas.
- b) De två första fångas av checksumman, i deserialize(). Den tredje ska inte fångas av något, eftersom ingenting är fel.
- c) Efter en tappad byte eller en ändrad bit står parseern kvar med en förkastad frame och måste nollställas; efter en fördröjning står den där den ska.
Fördröjningen ska alltså inte orsaka något problem. Gör den ändå det har parseern ett dolt tidsberoende — den antar något om *när* bytes kommer — och det är en bugg som inte visar sig förrän systemet flyttas till långsammare hårdvara.

Övning 3.3 Förlorad kvittens

sidan 18

- a) A får ingen Ack, timern löper ut, och A skickar om framen med **samma** SEQ.
- b) B ser en frame den redan behandlat: samma avsändare, samma sekvensnummer.
- c) B ska **skicka Ack igen** och **inte** köra applikationslogiken en andra gång. Kommandot är redan utfört.
- d) A hade fått timeout igen och skickat om igen, i en cykel som bara bryts av att försöksräknaren tar slut. Det är därför den uteblivna kvittensen är det farliga felet, inte den uteblivna framen.

Övning 3.4 Timeout-värdet

sidan 18

- a) Onödiga omsändningar och därmed fler dubletter, plus ökad bussbelastning precis när bussen redan är belastad.
- b) Långsam felupptäckt: systemet står och väntar på ett svar som aldrig kommer, och upplevs som hängt.
- c) Värstafallstiden för en tur och retur: sändningstid, mottagarens behandlingstid och kvittensens väg tillbaka. En nod känner sällan till den, eftersom den beror på hela systemets belastning och på hur snabb den andra noden är — vilket är just det som varierar.

Övning 3.5 Kedjan

sidan 18

Checksumma upptäcker att en frame är trasig, men säger inget om frames som aldrig kom fram. **ACK** bekräftar leverans, men hjälper inte när kvittensen själv uteblir. **Timeout** upptäcker den uteblivna kvittensen, men gör inget åt den. **Omsändning** åtgärdar den, och skapar därmed dubletter. **Dublettetetektering** städar upp efter omsändningen, och skapar inget nytt problem — det är därför kedjan slutar där.

Kapitel 4 — CAN, registerkartan och arkitekturen

Övning 4.1 Vem gör vad

sidan 25

Ramsynkronisering, längdfält, checksumma, kvittens och kollisionshantering görs alla av **hårdvaran**: SOF och EOF, DLC, CRC-15, ACK-biten inuti framen, och arbitreringen som bussen löser elektriskt.

De två raderna utan självklart svar är **adressering** och **sekvensnummer**. CAN har ingen destinationsadress och inget sekvensnummer alls: identifieraren beskriver innehållet, och behövs ett löpnummer får applikationen lägga det i nyttolasten.

Timeout och omsändning samt **dubblettetektering** är ett tredje fall: en fullständig CAN-kontroller sköter omsändningen i hårdvara, men den förenklade i den här konstruktionen gör det inte — se §4.3.4.

Övning 4.2 Packa en frame

sidan 25

Databyte 0 ligger i de mest signifikanta bitarna, och TX_DATA_L0 bär de *första* fyra bytesen:

```
TX_ID      = 0x000002A7
TX_DLC     = 0x00000005
TX_DATA_L0 = 0x01020304
TX_DATA_HI = 0x05000000
TX_SEND    = 0x00000001
```

Ordningen spelar roll därför att skrivningen till TX_SEND inte lagrar ett värde utan *utlöser en sändning*: kontrollern läser då de övriga registren. Skrivs triggern först skickas föregående frames innehåll.

Övning 4.3 Packa upp en frame

sidan 25

- a) 0x481.
- b) DE AD BE — tre bytes, eftersom RX_DLC är 3 och RX_DATA_L0 bär byte 0–3 med byte 0 överst.
- c) Noll. Drivrutinen ska maskera mot dlc; bytes ovanför den mottagna längden är inte framens innehåll, även om hårdvaran råkar nollställa dem.
- d) RX_ACK, med 0x1. Bit 0 måste vara satt.

Övning 4.4 Fyra påståenden

sidan 25

a, b och c beskriver buggar.

- a) Bit 0 betyder *porten är fri igen*, inte *framen kom fram* — en förlorad arbitrerings ger tillbaka samma bit. Läs ERROR_FLAGS efteråt.
- b) ERROR_FLAGS nollställs bara av ett *helt nollställt* ord. 0x1 gör ingenting, så felbiten går aldrig ned.
- c) Bytes ovanför RX_DLC hör inte till framen. Att kopiera alla åtta rapporterar upp till sex bytes som aldrig sändes.
- d) Korrekt. Utan kontrollen kan en skrivning ske mitt i en pågående sändning.

Övning 4.5 Lagergränsen

sidan 26

- a) Falskt. EchoNode ligger ovanför interfacet och får inte känna till någon konkret drivrutin.
- b) Sant. Spi *är* en Interface och måste inkludera den.
- c) Falskt. Stubben simulerar CAN i minnet och har ingen transport.
- d) Falskt. Transporten flyttar bytes och vet ingenting om register — det är hela poängen med att den är dum.
- e) Falskt. EchoNode testas genom Stub, som inte har någon transport.

Kapitel 5 — Framen i kod, och den första testsviten

Övning 5.1 Förutsäg

sidan 31

- a) Typiskt 12 bytes, inte 11. Kompilatorn lägger in utfyllnad så att id (std::uint16_t) hamnar på jämn adress, och avslutar strukten på en storlek som är en multipel av dess strängaste alignment.
- b) Ja. f.data[7] är 0: aggregate-initiering nollställer de medlemmar som inte anges.
- c) Raden slutar kompilera. En användardeklarerad konstruktor gör typen till en icke-aggregate, och då finns ingen konstruktor som tar (0x1230, 20, {...}). Det är precis skälet till att Frame inte har någon.

Övning 5.2 `constexpr`

sidan 31

- a) Att `isValid()` går att utvärdera vid kompileringstillfället — annars är den inte ett giltigt `static_assert`-villkor. Det bevisar samtidigt att funktionen är fri från sidoeffekter.
- b) Ingenting. Kontrollen sker när programmet byggs; i den färdiga binären finns ingen kod för den alls.

Övning 5.3 Läs sviten

sidan 31

- a) `0x7FF` och `0x800` för `id`, 8 och 9 för `d1c` — det största som ska accepteras och det minsta som ska avvisas. Buggar bor på gränserna: ett värde mitt i intervallet passerar både en korrekt och en felaktig jämförelse.
- b) Fallet som kontrollerar att `d1c = 8` accepteras skulle falla; alla andra vore gröna. Det är därför paret behövs och inte bara det ena värdet.
- c) Ja — att `MaxDataLen` är samma tal som arrayens storlek kontrolleras inte, och kan inte kontrolleras utifrån. En grön svit betyder att koden är rätt på de punkter sviten frågar om, inte att den är rätt.

Övning 5.4 Bryt koden

sidan 32

- a) Fallet som accepterar det största giltiga `id:t`, alltså `0x7FF`. Utskriften namnger uttrycket, filen och raden.
- b) Kortare än att gissa: testnamnet pekar ut kravet, Arrange-delen förutsättningen, och Assert-raden det förväntade värdet — först därefter behöver man öppna sin egen kod.
- c) Återställ koden — och notera att den felaktiga versionen passerade alla andra testfall. Ett enda tecken skilde en korrekt implementation från en som avvisar var åttonde giltig frame.

Övning 5.5 Var ligger valideringen

sidan 32

- a) Att ett fel fångas tidigare och närmare den som orsakade det.
- b) Att regeln nu finns på två ställen, och att båda måste hållas i synk.
- c) Bara det ena stället ändras. Den glömda kopian avvisar då frames som är giltiga, eller släpper igenom frames som inte är det — och felet ser ut att komma från hårdvaran.
Regeln uttrycks en gång, i `isValid()`, och används på ett ställe: i `Spi::send()`.

Kapitel 6 — Interfacet och stubben

Övning 6.1 Förutsäg kompilatorn

sidan 36

- a) Det kompilerar utan minsta invändning, och det är hela problemet. Utan `virtual` bestäms vilken destruktör som anropas av pekarens *statiska* typ, alltså `Interface`. Stubs destruktör körs aldrig, dess medlemmar destrueras aldrig, och standarden kallar det odefinierat beteende. För `Stub` märks det inte — den äger ingenting — men för en klass som håller en resurs läcker den, tyst, vid varje radering. Inget test fångar det.
- b) Utan attributet är `driver.send(frame);` en fullt laglig sats som kompilatorn inte har någon anledning att klaga på. Med `[[nodiscard]]` blir den en varning, och med `-Werror` ett fel. Det förvandlar den vanligaste drivrutinsbuggen — att tro att en `frame` gick iväg när hårdvaran var upptagen — från en felsökningssession till en kompilering som stannar.
- c) Med `override` är felstavningen ett kompileringsfel: det finns ingen virtuell metod med det namnet att överskugga. Utan `override` kompilerar det, och du har i stället *lagt till* en ny metod. Basklassens rena virtuella metod är då fortfarande oimplementerad, så klassen förblir abstrakt — och felmeddelandet du får handlar om att typen inte går att instansiera, långt ifrån den felstavade raden. Det är därför `override` är obligatoriskt.

Övning 6.2 En metod för mycket

sidan 36

- a) Ingenting meningsfullt. Stubben har inga register; den skulle behöva returnera en påhittad konstant, alltså ljuga.
- b) `EchoNode` blir bunden till att hårdvaran *har* register. Testet på om en metod hör hemma i interfacet är just detta: kan båda implementationerna uppfylla den ärligt? Kan de inte det, hör den inte hemma där.
- c) Varje anropare måste ändras, eftersom bitarnas betydelse är en del av registerkartan och inte av interfacet. Interfacet skulle alltså ärva hårdvarans versioner, vilket är precis vad det finns för att undvika.
- d) I `Spi`, som privat hjälpfunktion — eller, för felsökning, i en separat `PrintTransport` under `ByteTransport`-sömmen, där den inte syns för någon ovanför.

Övning 6.3 Polymorfismen

sidan 37

Uppgiften ber inte om kod; frågorna besvaras här.

- a) Nej. Funktionen tar en `driver::can::Interface&` och använder bara interfacets fyra metoder.
- b) Därför att den då kan köras mot vilken implementation som helst. Skriven mot `Stub` hade den varit ett test av `Stub`; skriven mot interfacet är den ett test av *kontraktet*, och kontraktet är det både `Stub` och `Spi` lovar.

c) Att Spi verkligen uppfyller samma kontrakt: samma returvärden i samma situationer. Framför allt måste `send()` returnera `false` när hårdvaran är upptagen och `receive()` returnera `false` när ingen frame finns — annars går funktionen igenom för stubben och fallerar för drivrutinen.

Övning 6.4 Stubbens gränser

sidan 37

- a) Fel (`hasError()` är alltid `false`), upptagen hårdvara (`send()` lyckas alltid), kö (en andra `send()` skriver över den första), och allt CAN gör: arbitrering, CRC, bitstoppning.
- b) Till exempel upptagen hårdvara: en räknare som får nästa `send()` att returnera `false`, satt av en testmetod. Det är den enklaste, eftersom den inte kräver något nytt tillstånd utöver en flagga.
- c) Varje simulerat beteende är kod som kan vara fel. En stubb som simulerar kö, fel och timing är en andra implementation av hårdvaran — och då vet du inte längre om ett fallerande test beror på drivrutinen eller på stubben. Gränsen går ungefär där stubben skulle behöva ett eget testfall: behöver den det, hör beteendet hemma i en riktigare dubblare, som BankTransport i kapitel 11.

Kapitel 7 — Komponenttest genom stubben

Övning 7.1 Skriv buggen, se testet fånga den

sidan 43

Uppgiften ber om en kodändring; frågorna besvaras här.

- a) Fallen som kontrollerar den ekade framens innehåll och räknaren fallerar — framen som ekas har fel `d1c`, och fallet som anropar `run()` två gånger på en injicerad frame avslöjar att tillståndet konsumerats. Fallet utan injicerad frame är fortfarande grönt, eftersom `receive()` returnerar `false` redan första gången.
- b) Nej, inget av dem. Stub och Frame gör exakt vad de lovar; ingen av dem är trasig.
- c) Ett enhetstest kan inte se en bugg som bor *mellan* enheterna. Det krävs ett test som kör flera delar tillsammans för att upptäcka att den ena använder den andra fel.

Övning 7.2 Lagergränsen

sidan 43

- a) Nej. EchoNode måste känna till interfacet — det är dess enda beroende.
- b) Nej. Testet håller i den konkreta typen; det är där `inject()` finns.
- c) Ja. Implementationsfilen ligger också ovanför interfacet, och får inte veta vilken drivrutin den kör mot.
- d) Nej, den bryter inte mot gränsen — `hasError()` och `clearError()` tillhör interfacet. Men det är ändå tveksamt: EchoNode kan inte göra något meningsfullt åt ett fel den inte kan tolka, och felflaggan är en bit för fyra orsaker. Felhanteringen hör hemma hos den som äger drivrutinen och känner systemets krav, alltså i factoryn eller i lagret ovanför.
- e) Ja, grovt. Casten namnger en konkret drivrutin *och* når en privat metod i den. Den skulle dessutom krascha så fort en Stub injicerades.

Övning 7.3 Ägande

sidan 43

- a) Det kompilarar inte. Interface är abstrakt och går inte att instansiera som värde; kompilatorn säger att typen är abstrakt. Vore den det inte skulle medlemmen *skiva* objektet — kopiera basklassdelen och kasta resten — vilket är värre än ett kompileringsfel.
- b) Det kompilarar. Referensen pekar då på ett förstört objekt, och `run()` är odefinierat beteende. Kompilatorn kan inte se det; det är den ena kostnaden för att komponenten inte äger sin drivrutin.
- c) Därför att EchoNode inte äger den. Drivrutinens livstid är längre än komponentens, den kan delas med andra komponenter, och ägandet ligger hos factoryn. En `unique_ptr` hade påstått motsatsen, och en `shared_ptr` hade lagt till räkning som ingen behöver.

Övning 7.4 Läs sviten

sidan 43

- a) Fallet som bara kontrollerar den ekade framens *innehåll* — rätt `id`, `dlc` och `data` — skulle vara grönt, eftersom ekot fortfarande sker. Det är därför sviten också kontrollerar räknaren.
- b) Trafik matas in med `stub.inject()` och läses tillbaka med `stub.receive()`. Ingen av dem finns i Interface: `inject()` kan Spi inte implementera, och `receive()` används här i en annan roll än interfacets — testet inspekterar vad komponenten *skickade*, vilket stubben råkar lagra i samma fält.
- c) Att `run()` konsumerar framen: efter första anropet finns ingenting kvar att eka, så räknaren ska stå på 1 och inte 2. Det är kravet som fångar dubbelanropet i övning 7.1.

Kapitel 8 — SPI och transaktionsprotokollet

Övning 8.1 Avkoda

sidan 48

Kommandobyten är **W** i bit 7, nollor i bitarna 6–4, och registerindexet i bitarna 3–0.

- a) $0x81$ = skrivbit satt, index 1: en **skrivning till TX_ID** med värdet $0x000007FF$ — elva bitar ettor, alltså det största värde registret kan hålla.
- b) $0x85$ = skrivning, index 5: en **skrivning till TX_SEND** med $0x1$, som *utlöser en sändning*. Registret lagrar ingenting.
- c) $0x8B$ = skrivning, index 11: en **skrivning till ERROR_FLAGS** med $0x0$, som nollställer felbiten.
- d) $0xC2$ är $1100\ 0010$: skrivbiten satt, men *också* bit 6, som är reserverad och måste vara noll. Kommandot är ogiltigt — ingen skrivning verkställs, ingen läsning låses — och slaven konsumerar ändå alla fem bytes innan den återgår till vila.

De två ovanliga är **c** och **d**. I **c** nollställer man genom att skriva *noll*, vilket är motsatt regel mot TX_SEND och RX_ACK som kräver bit 0 satt. I **d** är kommandot ogiltigt och slaven rapporterar det *inte*: det finns ingen felkanal på det här lagret, så en drivrutin som skickar skräp får tyst nonsens tillbaka.

Övning 8.2 Koda

sidan 48

- a) Läsning har bit 7 låg, så kommandobyten är bara indexet; RX_DLC har index 7 och de fyra databyten är dummybytes: $07\ 00\ 00\ 00\ 00$
- b) $82\ 00\ 00\ 00\ 04$
- c) $85\ 00\ 00\ 00\ 01$
- d) $8A\ 00\ 00\ 00\ 01$
- e) $8B\ 00\ 00\ 00\ 00$

Skillnaden mellan d och e är regeln för vad som räknas som en giltig skrivning. RX_ACK kräver att **bit 0 är satt** — en nolla gör ingenting. ERROR_FLAGS kräver att **hela ordet är noll** — $0x1$ gör ingenting. Två register bredvid varandra i samma karta, med motsatta regler, och att blanda ihop dem är en av projektets vanligaste buggar.

Övning 8.3 Vad kommer tillbaka

sidan 49

- a) Ingen. Under en skrivning är allt slaven driver på MIS0 meningslöst.
- b) Fyra: de sista fyra. Byten som klockas in *under kommandobyten* är rester från förra gången, eftersom slaven laddar sin svarsskiftare först i slutet av kommandobyten.
- c) Returvärdet blir skiftat en byte: den första riktiga databyten hamnar i bitarna 23–16 i stället för 31–24, och den sista databyten faller bort helt. I felsökningen ser det ut som att varje registervärde är 256 gånger för litet, eller — för STATUS — som att biten man letar efter aldrig sätts. Det är samma symptom som när SPI0.DATA inte läses i `transfer()`, vilket gör de två lätta att förväxla.

Övning 8.4 SS och avbrott

sidan 49

- a) Transaktionen överges helt, utan sidoeffekter: TX_ID ändras inte, ingen trigger går, och slaven återgår till vila redo för nästa fallande SS-flank. Det är vad som gör slaven självläkande efter en glitch.

- b) Slaven tar de fem första byten som en transaktion och **kastar de fem följande**. En transaktion börjar bara på den första byten efter att SS fallit, så den sjätte byten tolkas aldrig som en ny kommandobyte.

Det farliga är att det misslyckas *tyst*. Vid bring-upen ser det ut som att varannan register-skrivning försvinner: den första kommer fram, den andra inte, och ingenting rapporterar fel. Man misstänker kopplingen, klockfrekvensen och drivrutinens bitpackning — långt innan man misstänker SS.

- c) `0x91` är `1001 0001`: skrivbiten satt, index 1, men bit 4 är satt och reserverad. Kommandot är ogiltigt och ignoreras helt. Det ska förhindras redan när kommandobyten byggs: `CmdWrite | index` med ett index i intervallet 0–12 kan aldrig sätta bitarna 6–4, vilket är skälet till att indexet är en namngiven konstant och inte ett tal från en beräkning.

Övning 8.5 Tidsbudget

sidan 49

- a) Sex transaktioner — STATUS, RX_ID, RX_DLC, två dataregister och RX_ACK — alltså omkring $6 \times 45 = 270 \mu\text{s}$.
- b) En maximal frame är omkring 130 bitar, alltså cirka **130 μs** vid 1 Mbit/s.
- c) De skriver över den föregående. Kontrollern håller exakt en mottagen frame, och den nyaste vinner.

Det är **inte** din bugg: det är hårdvarans dokumenterade begränsning, och den är avsiktlig. Du kan minska risken genom att polla ofta och inte göra något långsamt mellan anropen, men du kan inte komma förbi att en `receive()` tar dubbelt så lång tid som framen den hämtar. Vid tillräcklig trafikintensitet tappas frames oavsett vad drivrutinen gör.

Kapitel 9 — Transportsömmen och registeråtkomsten

Övning 9.1 Förutsäg

sidan 56

- a) RegTxDataHi har index 4. Skrivning: `CmdWrite | 4 = 0x84`. Läsning: bara indexet, alltså `0x04`.
- b) `83 AA BB 00 00` — kommandobyten `CmdWrite | 3`, sedan värdet MSB först.
- c) `0x12345678`. Den första byten, `0xFF`, är svaret på kommandobyten och kastas; de fyra följande sätts ihop.
- d) Du får `0xFF123456`: värdet skiftas en byte, den sista databyten faller bort, och skräpbyten hamnar överst. Symptomet är registervärden som ser ut att vara 256 gånger för stora med en oförklarlig `0xFF` i toppen.

Övning 9.2 Teckenutvidgningen

sidan 56

Uppgiften ber om en kodändring; frågorna besvaras här.

- a) `value` blir `0xFFFFFFFF80000000` trunkerat till 32 bitar, alltså `0x80000000` — men vägen dit går genom odefinierat beteende. `transfer()` returnerar en `std::uint8_t`, som befordras till `int`. `0x80` skiftat 24 steg åt vänster sätter teckenbiten i en 32-bitars `int`, och att skifta in i teckenbiten är odefinierat i C++17. I praktiken blir resultatet ett negativt tal som sedan konverteras till `std::uint32_t` med ettor där det inte ska finnas några.
- b) För varje byte där bit 7 är *låg*, alltså alla värden under `0x80`. Koden fungerar därför perfekt för små identifierare och små DLC-värden.
- c) Drivrutinen hade fungerat för `id` upp till `0x7F` i den översta databyten och gått sönder däröver. Konkret: `0x123` fungerar, `0xAABB` i datafältet gör det inte. Vid bring-upen ser det ut som att vissa frames kommer fram och andra inte, beroende på *innehållet* — vilket är ett av de mest förvirrande symptom som finns, eftersom man letar efter ett fel i överföringen när felet ligger i en typkonvertering.

Övning 9.3 En transport att felsöka med

sidan 56

Uppgiften ber om kod. Utskriften ska gå att jämföra rad för rad mot exemplet i kapitel 8: fem bytes mellan ett `select` och ett `deselect`, kommandobyten först, värdet MSB först.

Två saker är värda att bygga in från början: skriv ut *både* den skickade och den mottagna byten på varje rad, så att en läsning går att följa, och räkna transaktionerna, så att en `send()` går att kontrollera mot de sex den ska kosta.

Övning 9.4 Lagergränsen

sidan 56

- a) Nej. ByteTransport flyttar bytes och vet ingenting om frames. Ett beroende till `frame.h` vore det första steget mot att den vet vad den transporterar.
- b) Nej. Då vore det inte längre en transport utan en registeråtkomst, och den hör hemma i `Spi` — som är det enda lager som ska känna till registerkartan.
- c) Nej, och det här är den som låter mest rimlig. Argumentet för är att transaktionen ändå alltid är fem bytes, så varför inte låta transporten sköta den? Argumentet emot är att `SS`-ramningen och `femBytes`-formatet tillhör *protokollet*, inte överföringen. Läggs de i transporten måste varje transportimplementation — även den skriptade testdubblaren — kunna protokollet, och då finns ingen söm kvar: man kan inte längre byta ut hur bytes flyttas utan att samtidigt byta ut vad de betyder.
- d) Nej. Att testa en privat metod direkt är nästan alltid ett tecken på att den borde vara publik, eller på att testet testat fel sak. Här testas den genom `send()` och `receive()`, och genom att inspektera vilka bytes som kom ut — vilket är precis vad som ska vara sant.

Kapitel 10 — Drivrutinen

Övning 10.1 Bryt koden med flit

sidan 62

Uppgiften ber om kodändringar; utfallen beskrivs här.

- a) Triggern skrivs innan dataregistren, så kontrollern läser föregående frames innehåll. På bussen syns en nod som ligger exakt en frame efter — den skickar alltid det du bad om *förra* gången.
- b) Databyte 4–7 hamnar i `TX_DATA_L0`. En frame med `d1c = 2` och data `AA BB` skickas som två nollbytes; mottagaren får rätt längd och fel innehåll.
- c) `AA BB CC DD` kommer fram som `DD CC BB AA`. Framen kommer fram, med rätt längd — bara bakvänd.
- d) Felbiten går aldrig ned, så `hasError()` returnerar `true` för alltid efter första felet och all felhantering ovanför slutar fungera.
- e) `STATUS` bit 1 står kvar. Varje `receive()` returnerar `true` med samma frame, i en oändlig loop, och ingen ny frame kan tas emot.
- f) Ingenting syns i utskriften, och det är poängen. `PrintTransport` visar vilka bytes som gick ut, och de är korrekta — felet ligger i vad drivrutinen *lämnar ifrån sig* till anroparen, alltså i `frame.data` ovanför `d1c`. Det fångas av ett testfall som levererar en frame med `d1c = 2` efter en med `d1c = 8` och kontrollerar att de sex översta bytesen är nollställda, vilket kräver

en dubblare som modellerar hårdvarans beteende — alltså BankTransport i kapitel 11, inte den skriptade.

g) Skrivningar sker mitt i en pågående sändning och skriver över data kontrollern använder. Utfallet beror på timingen, vilket gör felet sporadiskt.

Övning 10.2 Räkna transaktioner

sidan 62

- a) Sex: en STATUS-läsning och fem skrivningar.
- b) En — STATUS-läsningen. Därefter returnerar den false.
- c) Noll. Valideringen sker före allt annat, så en ogiltig frame kostar inte en enda transaktion.
- d) En.
- e) Sex: STATUS, RX_ID, RX_DLC, två dataregister och RX_ACK.

Fråga c är poängen med ordningen: en anropare som råkar skicka ogiltiga frames i en loop belastar inte bussen alls.

Övning 10.3 Den oändliga loopen

sidan 63

Uppgiften ber om kod; utfallen beskrivs här.

- a) Loopen snurrar tills hårdvaran blir ledig, och avslutas sedan normalt. Det är det avsedda beteendet.
- b) Loopen snurrar för alltid. isValid() returnerar false oavsett hur länge man väntar, och ingenting i systemet kommer att ändra på det.
- c) Validera före loopen och returnera direkt vid en ogiltig frame, och räkna försök i loopen så att även en hårdvara som aldrig blir ledig ger upp. Se §10.5.

Övning 10.4 Designfrågor

sidan 63

- a) Till exempel en uppräkningsvärde i stället för bool: Accepted, Busy, Invalid. Kostnaden för Stub är liten — den skulle returnera Accepted eller Invalid — men kostnaden för *interfacet* är att det växer, och varje anropare måste hantera fler fall. Avvägningen är att en bool är enklare att implementera och svårare att använda rätt.
- b) Ett testfall som levererar en frame med dlc = 8 och därefter en med dlc = 2. Utan maskering bär den andra ramen kvar sex bytes från den första, och applikationen kan inte skilja dem från riktig data. På riktig hårdvara krävs dessutom bara att hårdvarugruppen ändrar när dataackumulatorn nollställs, eller att du lånar ett kort byggt av en annan grupp.
- c) Drivrutinen observerar att STATUS bit 0 går låg vid triggern och kommer tillbaka strax därefter — exakt som vid en lyckad sändning — och att felflaggan är satt. Den kan *inte*

observera att det var en arbitrerings som förlorades snarare än ett stoppfel, en trasig CRC eller en mottagen fjärrframe, och den kan inte se vilken nod som vann eller vilken frame som gick igenom i stället.

d) Att polla oftare, alltså att göra mindre arbete mellan `receive()`-anropen. Gränsen du inte kommer förbi är att en `receive()` tar omkring 270 μs medan en frame tar 130 μs : en sändare som kör kontinuerligt hinner alltid skicka fler frames än du hinner hämta.

Kapitel 11 — Testning utan hårdvara

Övning 11.1 Få sviten grön

sidan 69

Uppgiften ber om kod. Loggen är däremot poängen: skriv ned vad testnamnet sa, var buggen låg, och hur lång tid det tog.

Det man brukar se är att tiden faller kraftigt under uppgiftens gång, och skälet är inte att buggarna blir enklare utan att man slutar gissa. Arbetsgången i §11.5 — testnamn, index, förväntat värde, och *först därefter* sin egen kod — är hela skillnaden.

Övning 11.2 Vilken dubblare fångar vad?

sidan 69

a) **Scripted.** De inspelade byten hamnar i fel transaktion, vilket syns direkt. `BankTransport` skriver lydigt in värdena i de register kommandobyten pekar ut och märker ingenting.

b) **Båda.** `Scripted` ser att den skickade byten är 01 där testet väntar 00; `Bank` ser att felbiten aldrig går ned.

c) **Bara Bank.** Ur den skriptade transportens synvinkel skickades exakt rätt bytes — felet ligger i vad drivrutinen lämnar ifrån sig i `frame.data`, inte i vad den skickade.

d) **Båda.** `Scripted` saknar `RX_ACK`-transaktionen i inspelningen; `Bank` ser att `RX`-giltig står kvar och att nästa `receive()` ger samma frame.

e) **Bara Bank,** indirekt. `Scripted` kontrollerar vad som *skickades*, och den byten skickas korrekt — felet är att svaret sätts ihop fel. `Bank` ger tillbaka värden som inte stämmer med vad som skrevs, så någon läsning blir fel.

f) **Båda.** `Scripted` ser att transaktioner går ut trots att ramen är ogiltig; `Bank` ser att registren skrivs.

g) **Ingen.** Och det är precis därför raden är med: ett kopplingsfel är osynligt för varje test som körs på laptopen, oavsett hur bra testdubblaren är. Det är vad kapitel 13 finns för, och skälet till att steg 1 i bring-up-stegen är en loopback.

Övning 11.3 Läs sviten som kravspecifikation

sidan 70

- a) Testfallet som kontrollerar TX_DATA_L0-transaktionen vid en `send()` av `id = 0x123`, `d1c = 2`, data AA BB. Dess fem EXPECT_EQ är exempelvis fem bytes, skrivna som ett krav.
- b) Fallet som skickar en frame med `d1c = 9` och kontrollerar att den inspelade byteströmmen är *tom*, alltså att inga transaktioner alls gick ut. Det kontrollerar inte bara returvärdet utan att valideringen sker *först*.
- c) Fördelningen ska luta mot BankTransport för beteende och ScriptedTransport för protokoll — ungefär som tabellen i §11.1. Ser man bara den ena i sviten saknas en hel klass av buggar.

Övning 11.4 Hela stacken

sidan 70

Uppgiften ber om att köra ett testfall; frågorna besvaras här.

- a) Noll. EchoNode är oförändrad sedan kapitel 7.
- b) Att EchoNode beror på `driver::can::Interface` och ingenting annat. Både Stub och Spi uppfyller det kontraktet, så bytet är en enda rad i testets Arrange-del.
- c) Allt. EchoNode hade behövt inkludera `spi.h`, och därmed dragit in ByteTransport och hela transportlagret. Testet hade behövt en transport för att kunna konstruera komponenten, och komponenttestet i kapitel 7 — som körs helt utan transport — hade inte gått att skriva.

Övning 11.5 Vad återstår

sidan 70

Minst dessa, och den sista är den som brukar glömmas:

- Att **tolkningen** av kontraktet är rätt. Sviten är skriven av en människa som läste samma dokument som du.
- Att **hårdvarugruppen** tolkat det likadant. Deras testbänkar är skrivna mot samma dokument, av någon annan.
- Att **kopplingen** är rätt — en omkastad MOSI/MISO är osynlig här.
- Att **tidsbudgeten** håller. BankTransport svarar omedelbart.
- Att du **pollar tillräckligt ofta** för att inte tappa frames.
- Att AvrSpiTransport fungerar. Den är den enda klassen i projektet som inget automattest täcker, eftersom den pratar med ett fysiskt register.

Kapitel 12 — Hårdvaran, transporten och factoryn

Övning 12.1 Se optimeraren arbeta

sidan 77

Uppgiften ber om kod; utfallen beskrivs här.

- a) Loopen försvinner, eller blir ett ovillkorligt hopp till sig själv. Kompilatorn läser flag en gång, ser att ingenting i loopen ändrar den, och drar slutsatsen att villkoret är konstant.
- b) Läsningen flyttas in i loopen igen: en laddning från minnet per varv.
- c) Statusflaggan ändras av SPI-periferin, inte av programmet. Utan `volatile` har kompilatorn samma rätt att läsa den en gång och anta att den står still — och då väntar `transfer()` antingen för alltid eller inte alls.

Övning 12.2 Förutsäg, ändra, observera

sidan 77

Uppgiften ber om kodändringar; symptomen beskrivs här.

- a) Ingenting händer på de kopplade ledningarna. Periferin ligger på standardmuxen, alltså en annan port, och programmet kör vidare utan att rapportera något. Det mest förvirrande symptom som finns: ingen felkod, inga klockpulser, ingen ledtråd.
- b) Mastern lämnar masterläget vid första `select()`, eftersom den tolkar den låga SS-nivån — som din egen kod just skapade — som att en annan master tagit över bussen. Överföringen avstannar direkt.
- c) Klockan hamnar över protokollets tak på 1 MHz. Det kan mycket väl fungera på en kort kabel, och det är det farliga: FPGA-slaven garanterar ingenting däröver, så felen kommer sporadiskt och blir värre med längre ledningar.
- d) Statusflaggan nollställs aldrig. Nästa `transfer()` ser en flagga som redan står kvar, returnerar omedelbart, och läser en byte som inte hunnit komma in — så all data ligger **en byte efter** genom hela transaktionen. Det *ser* ut att fungera: bytes kommer, antalet stämmer, bara innehållet är förskjutet, och det är lätt att misstänka bitordningen i stället.
- e) Klockpinnen drivs inte, så slaven ser inga flanker och ingenting överförs. Till skillnad från a) är MOSI och SS aktiva, vilket gör att en logikanalysator visar aktivitet på tre av fyra ledningar — och pekar rätt direkt.

Övning 12.3 Arkitekturtestet

sidan 78

Uppgiften ber om att kompilera; frågorna besvaras här.

- a) Det ska gå. Går det inte är arkitekturen bruten.
- b) Kompilatorfelet namnger filen, och den filen ligger per definition ovanför `driver/can/interface.h` och har lärt sig något om hårdvaran — typiskt en `#include` av `spi.h` eller en referens till `ByteTransport`.
- c) Precis samma sak som förut, mot en array i minnet: den ekar injicerade frames och räknar upp sin räknare. Att beteendet är oförändrat är hela poängen — applikationen kan inte se skillnad på en FPGA och en stubb.

Övning 12.4 Friställt

sidan 78

Uppgiften ber om att bygga; frågorna besvaras här.

- a) Ja. Ingenting i projektet använder undantag eller RTTI.
- b) Ingenting ska hittas. Hela stacken — `Frame`, `Interface`, `Stub`, `ByteTransport`, `Spi`, `EchoNode` — är byggd på statiskt allokerade objekt med känd storlek.
- c) Därför att kodkonventionerna var skrivna för ett friställt mål från början: returkoder i stället för undantag, aggregates i stället för dynamiska behållare, virtuella funktioner i stället för nedcastning. Det var inte tur, och det är det här kapitlet som visar vad vanan var värd.

Övning 12.5 Konstruktionsordningen

sidan 78

- a) Det kompilerar, och kompilatorn varnar inte. Medlemmar konstrueras i *deklarationsordning*, oavsett vilken ordning initialiseringslistan råkar ha.
- b) `myDriver` konstrueras först och binder en referens till `myTransport`, som ännu inte konstruerats. Själva bindningen är ofarlig — det är bara en adress — men första anropet till `transfer()` använder ett objekt som inte hunnit initieras, alltså odefinierat beteende. På ett friställt mål brukar det yttra sig som att SPI-periferin aldrig konfigureras.
- c) Att en medlem som beror på en annan måste deklarerars *efter* den. Initialiseringslistan uttrycker inte ordningen; den följer bara deklarationsordningen, och en kompilator som varnar för det gör det bara när listan och deklARATIONERNA står i olika ordning — inte här.

Kapitel 13 — Bring-up

Övning 13.1 Stegen

sidan 82

- a) Steg 1 bevisar att *din* SPI-master fungerar: klocka, bitordning, timing och konfiguration. FPGA:n är inte inkopplad och kan inte påverka utfallet. Steg 2 bevisar att hela kedjan fungerar — kopplingen, slavens synkronisering, transaktionsbryggan och registerbanken — och att båda sidor tolkar kommandobyten likadant.
- b) Därför att en loopback också ger tillbaka det du skrev. Skriver du 0x00000123 och läser 0x00000123 kan det lika gärna vara din egen MOSI som kommit tillbaka på MISO via en felkoppling. **Maskeringen är beviset:** skriv 0xFFFFFFFF till TX_ID och läs tillbaka 0x000007FF. Ett eko kan inte maskera bort de övre bitarna; bara en registerbank kan.
- c) Steg 1. AvrSpiTransport är den enda klassen i projektet som inget automattest täcker, eftersom den pratar med ett fysiskt register — den kan inte köras på värddatorn, och en testdubblare av den hade testat dubblaren.

Övning 13.2 Från symptom till hypotes

sidan 82

- a) Till exempel: MISO inte kopplad eller kopplad till fel pinne; pinnmuxen inte satt, så periferin ligger på fel port; VDDI02 inte matad, så PORTC inte fungerar; FPGA:n inte programmerad; klockan för snabb; SS släpps inte mellan transaktioner; eller gemensam jord som saknas.
- b) Snabbast först: VDDI02 (läs MVIO-statusen i programmet — ingen mätning alls), pinnmuxen (läs koden), FPGA:n programmerad (titta på kortet), sedan mätningarna: klockpulser på SCK, SS-beteendet, och sist kopplingen ledning för ledning.
- c) En loopback: koppla MOSI direkt till MISO och kör steg 1 igen. Får du tillbaka det du skickade fungerar din master och felet ligger i kopplingen mot FPGA:n eller i FPGA:n. Får du inte det ligger felet på din sida, och FPGA:n är utesluten.

Övning 13.3 Vems fel?

sidan 82

- a) Det går inte att avgöra från symptomet ensamt — alla tre är möjliga, vilket är hela svårigheten med att felsöka ett system två grupper byggt varsin halva av.
- b) Klättra nedåt i stegen tills något ger rätt kvitto. Ger steg 2 och 3 rätt svar fungerar SPI-kedjan, alltså är felet i CAN-sidan: kopplingen till transceivern, termineringen, eller kontrollern. Ger steg 3 0x00000001 men STATUS bit 0 aldrig går låg efter en TX_SEND-skrivning, når triggern inte kontrollern — vilket pekar på registerbanken eller på att din skrivning inte har bit 0 satt.
- c) Den ska innehålla vad du gjorde, vad du observerade och vad du redan uteslutit. Till exempel: *"Steg 2 ger rätt maskering för TX_ID (0xFFFFFFFF läses tillbaka som 0x000007FF), så SPI-kedjan och registerbanken fungerar. STATUS läses som 0x00000001. Efter en skrivning av 0x1*

till TX_SEND går bit 0 aldrig låg, och inget syns på tx_bus. Vi har verifierat att vår kommandobyte är 0x85 och att bit 0 är satt i datat.” Det är en rapport någon kan agera på; ”det fungerar inte” är det inte.

Övning 13.4 Termineringen

sidan 83

- a) Ungefär 60 Ω : två 120-ohmsmotstånd parallellt, ett i vardera änden.
- b) Det sitter tre motstånd i stället för två — tre parallella 120 Ω ger 40 Ω . Troligen har en nod mitt på bussen sin terminering påslagen.
- c) Det sitter bara ett motstånd; det andra saknas eller är inte anslutet. Det visar sig när kabeln blir längre, hastigheten högre eller miljön störigare — alltså typiskt när konstruktionen flyttas från labbänken, vilket är det sämsta tänkbara tillfället.

Kapitel 14 — Bussanalys

Övning 14.1 Läs din egen trafik

sidan 88

Uppgiften ber om mätning; frågorna besvaras här.

- a) De ska stämma exakt med vad du skrev till TX_ID, TX_DLC och dataregistren. Gör de inte det ligger felet i packningen, inte i överföringen — analysatorn avkodar det som faktiskt ligger på ledningarna.
- b) Ordningen är det analysatorn tillför. En förväxling av TX_DATA_L0 och TX_DATA_HI ger en frame med rätt identifierare och rätt längd men fel byteordning, och det är först här den blir synlig: alla tester på laptopen kan ha varit gröna om testet gjorde samma fel som koden.
- c) Bland annat SOF, RTR, IDE, r0, CRC-fältet, ACK-luckan och EOF. Registerkartan exponerar bara identifierare, DLC och data — resten sköter kontrollern, och analysatorn är det enda stället där du ser dem.

Övning 14.2 Åt andra hållet

sidan 88

Uppgiften ber om mätning; frågorna besvaras här.

- a) Ja, om RX-giltig sattes och du läste registren. Gör den inte det: kontrollera att identifieraren är en din nod tar emot, och att bithastigheten är densamma i analysatorn.
- b) Bit 1 ska gå hög när framen tas emot och låg först när du skriver 0x1 till RX_ACK. Står den kvar har kvittensen inte gått igenom — kontrollera att bit 0 verkligen är satt i det skrivna ordet.

c) Den andra framen skriver över den första om du inte hunnit kvittera den. Kontrollern håller exakt en mottagen frame, och den nyaste vinner; det är dokumenterat och avsiktligt. Med en `receive()` som tar omkring $270\ \mu\text{s}$ och frames som tar $130\ \mu\text{s}$ hinner du inte.

Övning 14.3 Kvittensen

sidan 88

- a) Ja. Analysatorn är en riktig CAN-kontroller och drar ned ACK-luckan på varje frame den tar emot med korrekt CRC. I tracen syns det som att framen registreras utan kvittensfel.
- b) Felflaggan förblir *orörd*. Den här konstruktionen läser aldrig tillbaka ACK-luckan, så den kan inte upptäcka att ingen kvitterade — sändningen fullbordas precis som om allt gått bra.
- c) En fullständig CAN-kontroller hade upptäckt kvittensfelet, ökat sin sändarfelräknare och **skickat om framen**, om och om igen tills någon svarade eller räknaren nådde bus off. En ensam nod på en riktig CAN-buss sänder alltså samma frame i all oändlighet. Den här kontrollern gör det inte, därför att den saknar både ACK-återläsning och automatisk omsändning — två av de dokumenterade förenklingarna.

Övning 14.4 Flaskhalsen

sidan 88

- a) Beror på hur ofta din applikation anropar `send()`. Med en frame per varv i en tät loop blir belastningen låg i CAN-mått — några procent — eftersom SPI-länken bromsar dig långt innan bussen blir full.
- b) En `send()` är sex SPI-transaktioner à omkring $45\ \mu\text{s}$, alltså cirka **$270\ \mu\text{s}$** . En maximal CAN-frame är omkring 130 bitar, alltså cirka **$130\ \mu\text{s}$** vid 1 Mbit/s.
- c) **SPI-länken**. Den tar ungefär dubbelt så lång tid som framen den skickar, så CAN-bussen står och väntar på dig. Det stämmer med räkningen i §8.3 — och det är här du kan se med egna ögon att den stämde, vilket är skälet till att kapitlet ber dig mäta i stället för att lita på tabellen.

Om duggan

Duggan är skriftlig och genomförs med papper och penna. Tillåtna hjälpmedel är en fusklapp på ett A4-blad, båda sidor, och en hexadecimal miniräknare. **Registerkartan delas ut med duggan** — indexen behöver inte memoreras, men att läsa kartan och veta vad den betyder behövs desto mer.

Byte-sekvenser, checksumberäkningar, registerpackning och SPI-transaktioner ska kunna göras för hand.

Upplägg

Duggan har två delar, ungefär lika stora:

Del	Innehåll	Poäng
A	Det egna protokollet: framedesign, parsning, adressering, felmodell och tillförlitlighetskedjan (kapitel 1–kapitel 3).	12
B	CAN och drivrutinen: arbitrering, registerkartan, SPI-transaktionen, arkitekturen och begränsningarna (kapitel 4–kapitel 12).	13

Godkänt kräver poäng i *båda* delarna. Det går alltså inte att kompensera en tom del B med en perfekt del A.

G- och VG-uppgifter

G-uppgifterna kontrollerar grundläggande förståelse: bygga och läsa en frame, räkna en checksumma, följa en parser genom en byte-ström, läsa registerkartan, packa en frame i register, avkoda en SPI-transaktion, och förklara arbitrering.

VG-uppgifterna kräver resonemang: tillförlitlighetskedjan och dess konsekvenser, analys av fel i en byte-ström, varför klibbiga statusbitar behövs, lagerarkitekturens gränser, och vad konstruktionens begränsningar gör omöjligt för drivrutinen.

Övningsduggan

Nästa kapitel är en fullständig övningsdugga i samma format och med samma poängfördelning som den skarpa. Svaren står i kapitlet därefter — med uträkningarna, inte bara resultaten.

Gör den med papper och penna, och läs svaren först efteråt.

Övningsdugga

Totalt 25 poäng. **G:** minst 12 poäng, med minst 4 i vardera delen. **VG:** minst 19 poäng.

Svara kort och tydligt. Visa uträkningarna där du räknar.

Del A — det egna protokollet (12 p)

Checksumman är en 16-bitars summering av **alla bytes fram till CHK-fältet**. Fält större än en byte (SOF, SEQ, CHK) skickas big-endian.

SOF (2) | LEN (1) | TYPE (1) | DST (1) | SRC (1) | SEQ (2) | DATA (N) | CHK (2)

SOF = 0xA5F7. Frametyper: Ping = 0x00, Pong = 0x01, StatusRequest = 0x02, StatusResponse = 0x03, Ack = 0x04, Nack = 0x05.

Övning C.1 Bygg två frames

4 P

- En nod med adress 0x17 skickar en StatusRequest till en sensor med adress 0x25. Sekvensnumret är 0x7F05. Skriv upp hela ramen byte för byte. (2 p)
- Sensorn svarar med ett StatusResponse innehållande temperaturen 0x16 (en byte). Skriv upp den ramen. (2 p)

Övning C.2 Frame-parsern

2 P

- Beskriv kort vad parsern gör i WaitForSof1 respektive WaitForSof2. (1 p)
- Parsern står i WaitForSof2 och tar emot en byte som inte är 0xF7. Vad ska den göra? Det finns ett svar som är nästan rätt och ett som är rätt — motivera vilket som är vilket. (1 p)

Övning C.3 Buss och routing

2 P

- a) Sker routingen i bussen eller i noden? Motivera. (1 p)
- b) En nod tar emot en frame med korrekt checksumma, men DST är inte nodens adress. Vad ska noden göra, och varför ska den *inte* skicka Nack? (1 p)

Övning C.4 Tillförlitlighet

2 P

- a) Nod A skickar en StatusRequest och får ingen Ack. Beskriv flödet med timeout och omsändning. Ska SEQ vara samma eller nytt, och varför? (1 p)
- b) Ack-ramen tappas på vägen tillbaka, så A skickar om. B tar emot samma frame en andra gång. Vad ska B göra, och vad ska B *inte* göra? (1 p)

Övning C.5 Byte-ström och felmodell

2 P

En nod tar emot:

```
32 74 00 FF A5 F7 03 03 25 17 7F 05 10 20 30 02 C2
```

- a) Vid vilket index börjar ramen, och vad ska parsern göra med datan före? (1 p)
- b) Antag i stället att byten 0x20 tappas. Vad händer med tolkningen, och i vilket tillstånd hamnar parsern? (1 p)

Del B — CAN och drivrutinen (13 p)**Övning C.6 CAN-bussen**

3 P

- a) Vad är ett CAN-ID, och vilka **två** roller har det samtidigt? (1 p)
- b) Två noder börjar sända samtidigt. Nod A har 0x100, nod B har 0xFF. Vilken vinner, vad gör förloraren, och varför krävs ingen busmästare? (1 p)
- c) Räkna upp **tre** mekanismer från del A som du inte behövde implementera i mjukvara, och säg var i systemet de utförs i stället. (1 p)

Övning C.7 Registerkartan

4 P

- a) Du ska skicka en frame med id = 0x2A7, dlc = 5 och databytesen 01 02 03 04 05. Skriv upp de fem registervärdena som ska skrivas, i rätt ordning. Varför spelar ordningen roll? (2 p)
- b) Du läser RX_ID = 0x00000481, RX_DLC = 0x00000003, RX_DATA_LO = 0xDEADBE00, RX_DATA_HI = 0x00000000. Vilken identifierare kom ramen från, vilka databytes innehåller den, och vad ska frame.data[3] till frame.data[7] innehålla efter receive()? (1 p)

- c) Vilket register måste skrivas innan nästa frame kan tas emot, och med vilket värde? Vad händer om drivrutinen glömmer det? (1 p)

Övning C.8 SPI-transaktionen

3 P

Följande bytes går ut på MOSI. För var och en: läsning eller skrivning, vilket register, och vilket värde? (2 p)

- a) 81 00 00 07 FF
- b) 85 00 00 00 01
- c) 8B 00 00 00 00
- d) C2 00 00 00 03

- e) Skriv upp den kompletta femBytestransaktionen för att **läsa** RX_DLC. (1 p)

Övning C.9 Klibbiga bitar och begränsningar

3 P

- a) Kontrollern signalerar med pulser på en klockcykel, 20 ns vid 50 MHz. En SPI-transaktion tar omkring 40 μ s. Förklara varför registerbanken behövs, och varför STATUS bit 0 måste nollställas av en *skrivning* i stället för automatiskt vid läsning. (1 p)
- b) Din nod sänder en frame och förlorar arbitreringen. STATUS bit 0 kommer tillbaka precis som vid en lyckad sändning. Vad ska drivrutinen göra för att ta reda på hur det gick, och vad kan den fortfarande *inte* ta reda på? (1 p)
- c) `clearError()` skriver 0x0 till `ERROR_FLAGS`, medan `receive()` skriver 0x1 till `RX_ACK`. En student tycker att det är inkonsekvent och skriver 0x1 i båda. Vad blir symptomet, och när upptäcks det? (1 p)

Svar på övningsduggan

Uträkningarna står med, inte bara resultaten.

Del A

Övning C.1 Bygg två frames

sidan 119

a) $LEN = 0$ eftersom en StatusRequest saknar nyttolast, $TYPE = 0x02$, $DST = 0x25$, $SRC = 0x17$, $SEQ = 0x7F05$.

Checksumman: $A5 + F7 + 00 + 02 + 25 + 17 + 7F + 05 = 165 + 247 + 0 + 2 + 37 + 23 + 127 + 5 = 606 = 0x025E$.

```
A5 F7 00 02 25 17 7F 05 02 5E
```

b) Nu byter adresserna plats: $DST = 0x17$, $SRC = 0x25$. $LEN = 1$, $TYPE = 0x03$, och en databyte $0x16$. SEQ är detsamma.

Checksumman: $606 - 0x02 + 0x03 + 0x01 + 0x16 = 606 - 2 + 3 + 1 + 22 = 630 = 0x0276$.

```
A5 F7 01 03 17 25 7F 05 16 02 76
```

Övning C.2 Frame-parsern

sidan 119

a) I `WaitForSof1` kastar parsern varje byte som inte är $0xA5$. Den står stilla tills strömmen börjar se ut som en frame. I `WaitForSof2` väntar den på $0xF7$, och går vidare till `WaitForLen` om den kommer.

b) Det *nästan* rätta svaret är att gå tillbaka till `WaitForSof1` och kasta byten. Det *rätta* är att stanna kvar i `WaitForSof2` om byten är $0xA5$.

Skälet: den byte som just kom kan vara den första byten i en riktig frame. Kastar parsern den missar den en giltig frame som börjar direkt efter en falsk SOF-start — vilket händer så fort $0xA5$ förekommer i en nyttolast strax före en verklig framestart.

Övning C.3 Buss och routing

sidan 120

a) I noden. Bussen är en fysisk ledning som levererar varje byte till varje nod; den kan inte läsa adresser, och den känner inte till att det finns frames. Varje nod måste därför själv jämföra DST med sin egen adress.

b) Ignorera framen helt. Den ska *inte* skicka Nack, eftersom framen inte är felaktig — den är bara adresserad till någon annan. Skickade varje nod Nack på varje frame den inte var mottagare för skulle varje meddelande på en buss med n noder generera $n - 2$ felaktiga negativa kvittenser, och sändaren skulle dessutom inte kunna skilja dem från ett verkligt fel.

Övning C.4 Tillförlitlighet

sidan 120

a) A startar en timer när framen skickas. Uteblir Ack tills timern löper ut skickar A om framen och startar timern på nytt, upp till ett bestämt antal försök; därefter rapporteras ett permanent fel.

SEQ ska vara **samma**. Ett nytt sekvensnummer skulle göra omsändningen till ett nytt meddelande, och B skulle inte kunna känna igen den som en upprepning — vilket är exakt vad dubblettdetekteringen behöver.

b) B ska **skicka Ack igen**, och **inte** köra applikationslogiken en andra gång.

Att inte köra logiken igen är poängen med dubblettdetekteringen: kommandot är redan utfört. Att ändå kvittera är den del som glöms — A väntar fortfarande på sin kvittens, och tystnad leder bara till ännu en omsändning och därmed till en oändlig cykel.

Övning C.5 Byte-ström och felmodell

sidan 120

a) Vid index 4 (nollindexerat): 32 74 00 FF är skräp, och A5 F7 på index 4–5 är SOF.

Parsern kastar bytesen före: de tas emot i `WaitForSof1` och förkastas en och en eftersom ingen av dem är 0xA5.

b) LEN är fortfarande 3, så parsern samlar in tre databytes — men nu blir de 10 30 02 i stället för 10 20 30. Därefter läser den C2 som den första checksummabyten och väntar på den andra.

Parsern hamnar alltså i `WaitForChk2` och står kvar där tills nästa byte kommer, vilken den än är. När den kommer är framningen klar men checksumman stämmer inte, så `extractFrame()` returnerar `false` och framen förkastas.

Notera att `isFrameReady()` mycket väl kan bli sann — framningen lyckades. Det är valideringen som fångar felet, och det är därför de två frågorna är åtskilda.

Del B

Övning C.6 CAN-bussen

sidan 120

a) Identifieraren beskriver **vad framen innehåller** — inte vem den är till. De två rollerna är:

1. **Innehåll:** mottagarna filtrerar på den och avgör själva om framen angår dem.
2. **Prioritet:** den avgör vem som vinner en arbitrering, där lägre numeriskt värde vinner.

b) **B vinner**, eftersom 0x0FF är numeriskt lägre än 0x100. Binärt, elva bitar: A sänder 001000000000 och B 000111111111, och vid bit 3 sänder A recessivt där B sänder dominant.

Förloraren A upptäcker det omedelbart — den sänder en recessiv bit och läser tillbaka en dominant — slutar sända, blir mottagare för B:s frame, och försöker igen när bussen är ledig.

Ingen bussmästare krävs eftersom bussen själv är en OCH-funktion: en dominant bit skriver över en recessiv, så konflikten löses av elektriken medan den uppstår. Ingen ram förstörs och ingen tid går förlorad.

c) Tre av följande: ramsynkronisering (kontrollern, SOF och EOF), längdfält (kontrollern, DLC), checksumma (kontrollern, CRC-15), kvittens (kontrollern, ACK-biten inuti framen), kollisionshantering (bussen själv, genom arbitrering).

Övning C.7 Registerkartan

sidan 120

a) Databyte 0 ligger i de mest signifikanta bitarna, och `TX_DATA_L0` bär de *första* fyra bytesen:

<code>TX_ID</code>	= 0x000002A7
<code>TX_DLC</code>	= 0x00000005

```
TX_DATA_LO = 0x01020304
TX_DATA_HI = 0x05000000
TX_SEND    = 0x00000001
```

Ordningen spelar roll därför att skrivningen till TX_SEND inte lagrar ett värde utan *utlöser en sändning*. Kontrollern läser då de övriga registren. Skrivs triggern först skickas föregående frames innehåll.

b) RX_ID = 0x481 ger identifieraren **0x481** (inom 11 bitar, alltså giltig). RX_DLC = 3 betyder tre databytes, och RX_DATA_LO = 0xDEADBE00 ger byte 0–3 som DE AD BE 00. Framen innehåller alltså **DE AD BE**.

frame.data[3] till frame.data[7] ska vara **noll**. Drivrutinen ska maskera mot dlc: byte-sen ovanför den mottagna längden är inte ramens innehåll, även om hårdvaran råkar nollställa dem.

c) RX_ACK, med värdet 0x1 — bit 0 måste vara satt.

Glöms den står STATUS bit 1 kvar för alltid. Varje receive() returnerar då true med samma frame, i en oändlig loop, och ingen ny frame kan tas emot.

Övning C.8 SPI-transaktionen

sidan 121

Kommandobyten är W i bit 7, nollor i bitarna 6–4, och registerindexet i bitarna 3–0.

a) 0x81 = skrivbit satt, index 1: en **skrivning till TX_ID** med värdet 0x000007FF. Det är det största värde registret kan hålla — elva bitar ettor.

b) 0x85 = skrivning, index 5: en **skrivning till TX_SEND** med värdet 0x1. Den **utlöser en sändning**; registret lagrar ingenting.

c) 0x8B = skrivning, index 11: en **skrivning till ERROR_FLAGS** med värdet 0x0. Det är den *enda* skrivning som nollställer felbiten — ett nollskilt värde hade ignorerats.

d) 0xC2 = 1100 0010. Bit 7 är satt (skrivning) *och bit 6 är satt*, men bitarna 6–4 är reserverade och måste vara noll. Kommandot är därför **ogiltigt**: ingen skrivning verkställs, ingen läsning låses. Slaven konsumerar ändå alla fem bytes och återgår till vila, och rapporterar ingenting — det finns ingen felkanal på det här lagret.

e) En läsning har bit 7 låg, så kommandobyten är bara indexet. RX_DLC har index 7, och de fyra databyten är dummybytes:

```
07 00 00 00 00
```

Övning C.9 Klibbiga bitar och begränsningar

sidan 121

a) På 50 MHz är en klockcykel 20 ns, och 40 μ s motsvarar $40 \cdot 10^{-6} \cdot 50 \cdot 10^6 = 2000$ cykler. Sannolikheten att en pollning råkar träffa just den enda cykel pulsen varar är alltså ungefär 1 på 2000: mjukvaran skulle missa nästan varje händelse. Registerbanken gör om pulsen till en **nivå som står kvar**, så att den går att polla.

Nollställningen måste vara en skrivning därför att en automatisk nollställning vid läsning gör det omöjligt att läsa statusen två gånger — och därmed omöjligt att först kontrollera en bit och sedan agera på den. En uttrycklig nollställning skiljer dessutom *jag har sett händelsen från jag har hanterat den*, och det är mjukvaran, inte hårdvaran, som vet när det senare är sant.

b) Drivrutinen ska vänta tills STATUS bit 0 kommer tillbaka — porten är fri — och **därefter läsa ERROR_FLAGS**. Är felbiten satt gick sändningen inte igenom.

Den kan fortfarande **inte** ta reda på *varför*. Felflaggan är en enda bit för fyra orsaker: förlorad arbitrering, stoppfel, misslyckad CRC och mottagen fjärrframe. Retry-logik som behandlar dem olika går därför inte att skriva mot det här gränssnittet.

c) ERROR_FLAGS är en läs-och-nollställ-på-noll-konstruktion: **bara** en skrivning där hela ordet är noll nollställer läsningen. En skrivning av 0x1 gör ingenting alls.

Symptomet: felbiten går aldrig ned. `hasError()` returnerar `true` för alltid efter det första felet, och all felhantering ovanför slutar fungera — en retry-loop som läser felflaggan kommer att tro att varje sändning misslyckas.

Det upptäcks **inte** av ett test som bara kontrollerar vilka bytes som skickades, eftersom transaktionen ser korrekt ut. Det upptäcks av ett test mot en simulerad registerbank, som modellerar nollställningsregeln — eller, betydligt senare och dyrare, vid bring-upen.